OVERVIEW

Development Approach

How a team chooses the way it builds software, in four parts. Methodology concepts: process vs. process model vs. methodology, what a methodology addresses, its ten elements, and the two life cycles of an IT project (PLC for project requirements, SDLC for product requirements). Software development processes: plan-driven waterfall (when it fits and how it fails) vs. evolutionary/iterative development with its four phases (inception, elaboration, construction, transition). Agile: the 4 values and 12 principles, incremental development, XP practices and Scrum (roles, artifacts, sprints, backlog). DevOps: closing the gap between development and operations, and how project management adapts to a CI/CD pipeline.

The exam angle is choosing an approach for a situation and justifying it: match stable, known requirements to waterfall and evolving, uncertain ones to iterative or agile, and say why.

Process → model → methodology

Process = the activities; process model (life cycle) = an abstract shape like waterfall or iterative; methodology = the concrete "how we work around here".

PLC vs. SDLC

The project life cycle targets project requirements (time, cost); the SDLC targets product requirements. You need both, integrated.

Waterfall fits the known

Clear stable requirements, proven technology, low complexity, adequate time. Fails late: late integration, late risk resolution.

Iterative fits the uncertain

Unclear or evolving requirements, new technology, high complexity, short schedule. Phases: inception → elaboration (most critical) → construction → transition.

Agile = 4 values, 12 principles

Individuals over process, working software over documents, collaboration over contracts, responding to change over the plan. Waterfall fixes scope; agile fixes time and cost.

XP vs. Scrum

XP prescribes engineering practices (TDD, CI, refactoring, pairing); Scrum is a planning framework (backlog, sprints, Product Owner, ScrumMaster) with no technical practices.

HOW THE LECTURE IS ORGANIZED

Slides 1–14 Methodology concepts, elements of a methodology, PLC vs. SDLC

Slides 15–32 Plan-driven waterfall vs. evolutionary/iterative development

Slides 33–47 Agile: manifesto, principles, incremental development

Slides 48–55 Extreme Programming practices

Slides 56–74 Scrum: roles, backlog, sprints, agile vs. waterfall

Slides 75–98 DevOps and DevOps project management

EXAM FOCUS

Midterm 251 (both versions) asked when an iterative approach is most beneficial (MCQ 6) and, in the Edugate case, whether to recommend Agile, Waterfall or Hybrid and why (Case Q2, from the instructor's key: Agile/Scrum because requirements evolve). Final 251 Q1.4 asked for a PM tool or technique for a complex software project; Final 242 Q1.2 needs the milestone definition from slide 10.

HOW TO READ THIS PAGE

Every original slide is shown unchanged. Yellow marks on a slide point to a numbered note: hover or tap a mark to jump to its note, and hover a note to see its marks. Questions tagged **Past exam** appeared on a real paper; **Prediction** questions are likely angles, not confirmed. Pick an answer to each question: a wrong pick tells you why it is wrong, so you can try again.

SLIDE 1 / 98 **Slide 1**

CCIS كلية علوم الحاسب والمعلومات COLLEGE OF COMPUTER & INFORMATION SCIENCES



جامعة الأمير سلطان PRINCE SULTAN UNIVERSITY

Development Approach

SE423: Software Project Management

1

NOTES

1 The deck has four parts: **methodology concepts**, **software development processes** (waterfall vs. iterative), **agile** project management (XP and Scrum), and **DevOps**. The middle two carry most of the exam questions.

SLIDE 2 / 98 **Outline**

Outline

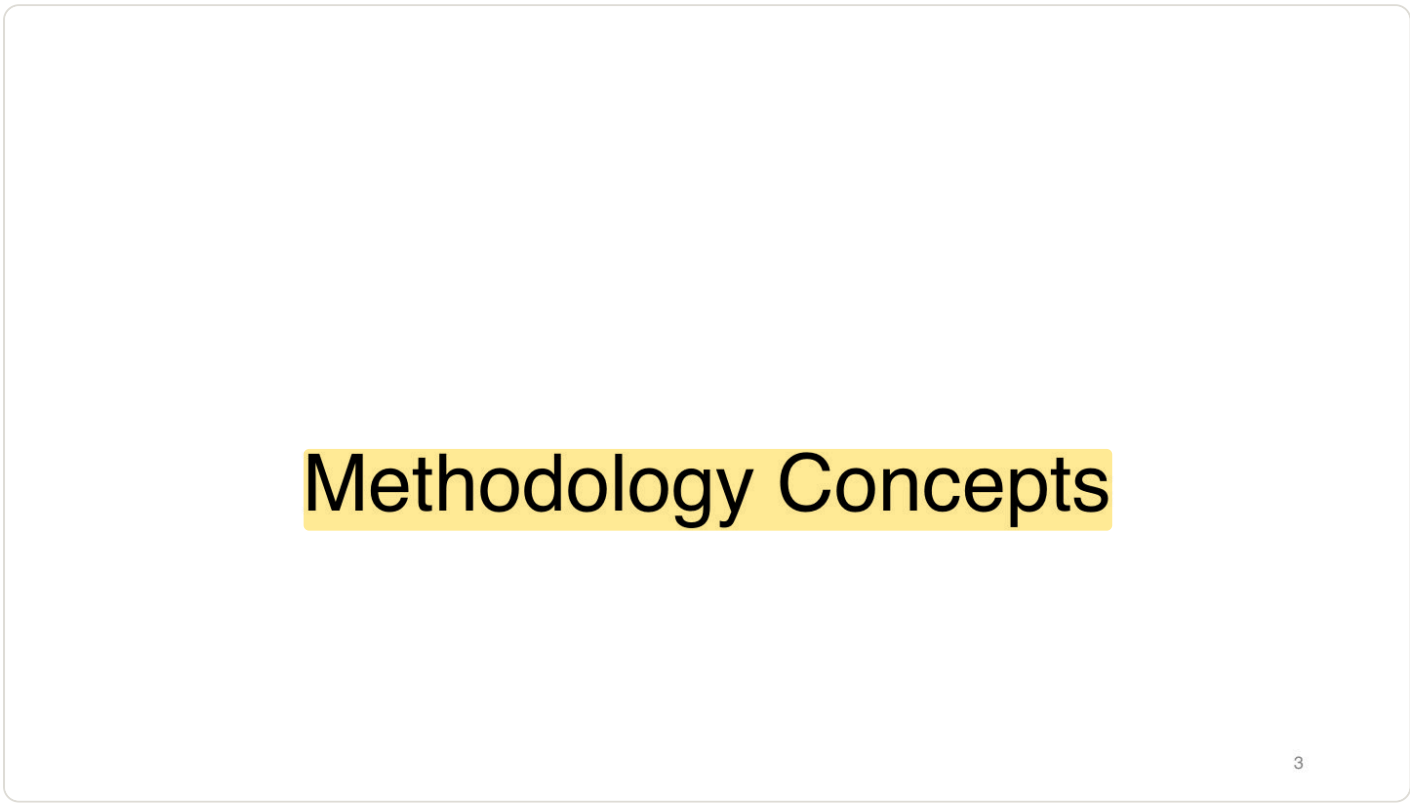
- Methodology Concepts
- Software Development Process
- Agile Project Management
- DevOps

2

NOTES

1. **Methodology concepts**: the vocabulary (process, model, methodology) and what a methodology contains.
2. **Software development process**: plan-driven waterfall vs. evolutionary/iterative.
3. **Agile project management**: the manifesto, XP and Scrum.
4. **DevOps**: joining development and operations, and what that means for the PM.

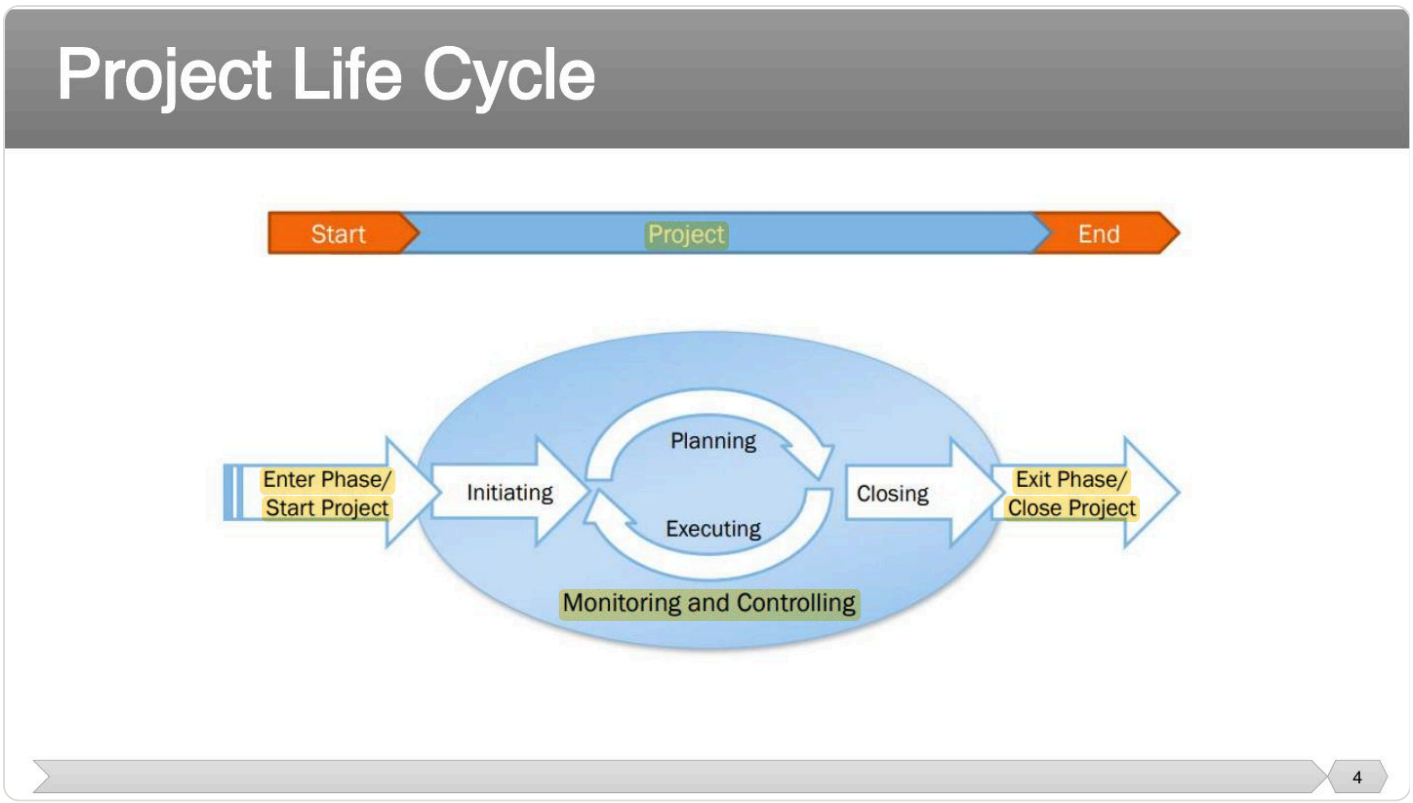
SLIDE 3 / 98 **Slide 3**



NOTES

1. Section divider: **Methodology Concepts**.

 Project Life Cycle



NOTES

1. The **project life cycle** as process groups. **Initiating** starts the phase or project (charter, stakeholders).
2. **Planning** and **executing** loop around each other, wrapped by **monitoring and controlling**, which compares progress with the plan and triggers re-planning.
3. **Closing** ends the phase or project (acceptance, lessons learned). Every development approach in this deck still sits inside this cycle.

 Methodology Concepts

Methodology Concepts

- Process
  - A collection of work activities, actions and tasks that are performed when some work product is to be created
- Process Model or Life Cycle
  - An abstract description of a software process that presents one view of that process
  - Waterfall, Iterative, Spiral, Evolutionary, etc.
- Process Methodology or Methodology
  - Instantiations of process models – tend to be prescriptive.
  - The conventions that a group agrees to – "How we work around here"

5

NOTES

1. A **process** is what actually gets done to produce something. Writing and reviewing a requirements document is a process: it's the real work, not a description of it.
2. A model is a general shape, like a recipe type: "iterative" tells you the idea without saying how your company does it. That's why it's called an **abstract description**.
3. A **methodology** is the model made specific, with your team's rules, roles and habits. So Scrum at your company is a methodology, while "iterative" is the model behind it.

EXAM ANGLE

Prediction Q1

Waterfall, iterative and spiral are examples of:

A Tools

B Milestones

C Methodologies

D **Process models (life cycles)**

SLIDE 6 / 98 **What a Methodology Addresses?**

What a Methodology Addresses?

- Introducing new people to the process
- Substituting people
- Delineating responsibilities
- Demonstrating visible progress
  - McConnell's definition for visibility: "The ease & accuracy with which it is possible to assess the status of a project's cost, schedule, functionality, or other characteristic."

6

NOTES

1. If a new hire can't find out how things work, they're slow for months. A methodology writes down how work is done, so **new people** ramp up faster, and a replacement can step in.
2. Managers need to know how a project is doing without guessing. **Visibility** means status is easy to see and accurate, so problems get noticed while they can still be fixed.

SLIDE 7 / 98 **Evaluating a Methodology**

Evaluating a Methodology

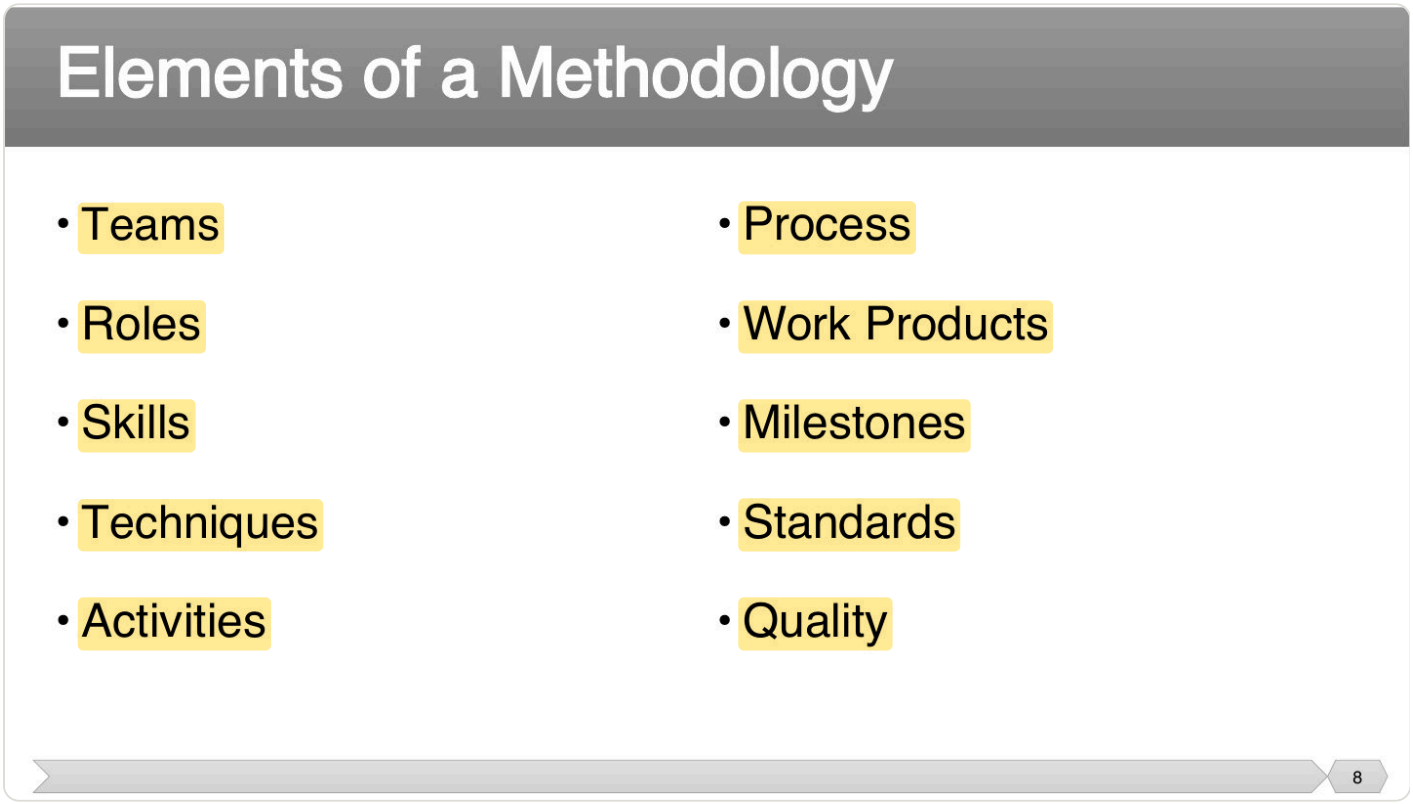
- How rapidly you can substitute or train people.
- How great an effect it has on the sales process.
- How much freedom (or how constraining) it is to people on the project.
- How fast it allows people to respond to changing situations.
- How well it "protects" the organization – legally or from other damages.

7

NOTES

1. If one person leaves and work stops, the method is failing you. A good one makes roles clear enough that you can **substitute** someone without losing weeks...
2. ...its **effect on the sales process** (can you show clients a credible way of working?)...
3. Too many rules slow people down and frustrate them, while too few cause chaos. This criterion asks you to judge that balance, or how much **freedom** people have...
4. If each change means a long approval process, you can't react when something shifts. How well you **respond to change** is one of the main differences between methods...
5. Records of who approved what help if a dispute goes to court. A method's paperwork can be annoying day to day and valuable when it **protects** the organization.

SLIDE 8 / 98 **Elements of a Methodology**

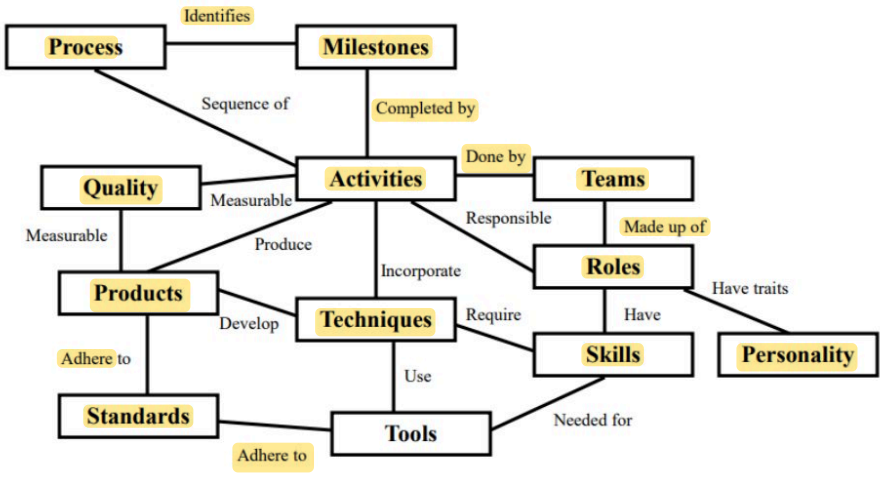


NOTES

1. You don't need to memorize these as one list. The left set is about people: who does the work and what they know. The **ten elements** are easier to recall in those groups.
2. This set is about the work itself: what order it's done in, what comes out of it, and what counts as good enough. Slides 10–13 define each.

 Cockburn Methodology Structure

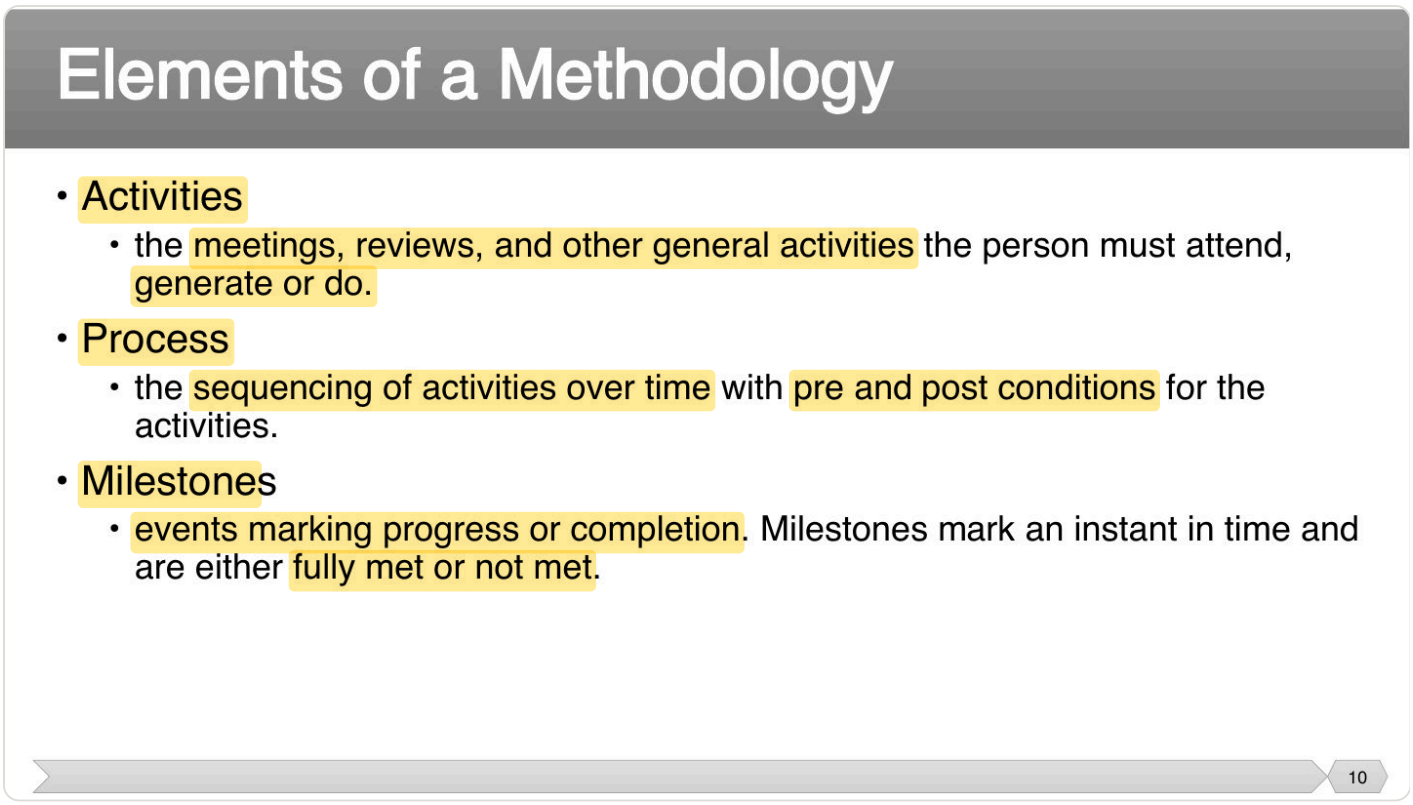
Cockburn Methodology Structure



NOTES

1. Read the diagram as sentences: the **process** is a sequence of **activities**, completed by **milestones**; activities produce **products** and incorporate **techniques**.
2. **Teams** are made of **roles**; roles have **skills** and are matched to **personalities**; techniques require skills and use **tools**.
3. Products adhere to **standards**, and **quality** is measurable on products and activities.

 Elements of a Methodology



NOTES

1. Think of your calendar: reviews and meetings count as **activities** in the methodology too, since they take real time and need planning.
2. Activities alone don't tell you the order. **Process** says what must be finished before something can start, such as code review before merging.
3. A **milestone** isn't work that takes time. It's a moment you pass or don't. That's why "60% met" makes no sense: either the design was approved today or it wasn't.

TIPS & TRICKS

- **MAP**: Milestones, Activities, Process.

EXAM ANGLE

Past exam · Final 242 · Q1.2 Q1

Explain what is a Gantt chart and describe its significance in planning and tracking project activities. In your answer, define the terms milestone and dependency, and provide one example of each as they would appear in a Gantt chart.

Answer: A Gantt chart is a bar chart with activities down the side and a time scale across, where each bar shows an activity's start, finish and duration; it makes the schedule easy to plan and to track, because actual progress is shaded against the planned bars. A milestone is a significant event with zero duration, either fully met or not, shown as a diamond, e.g. "Requirements signed off". A dependency is a logical relationship where one activity can't start (or finish) until another does, shown as an arrow, e.g. "Coding" can't start until "Design approved" finishes.

Prediction Q2

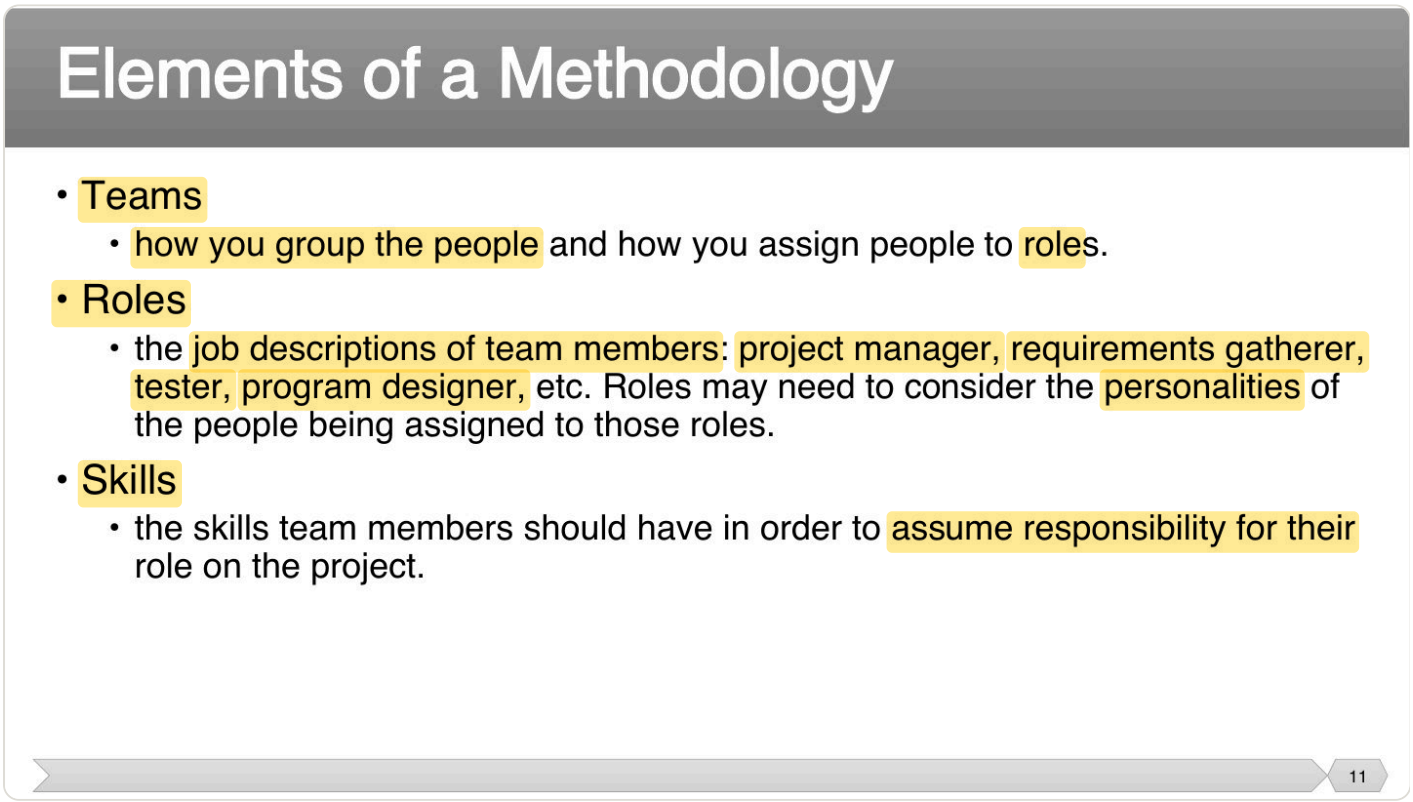
A milestone can be 60% met.

T True

F **False**

> Milestones mark an instant in time and are either fully met or not met.

SLIDE 11 / 98 **Elements of a Methodology**



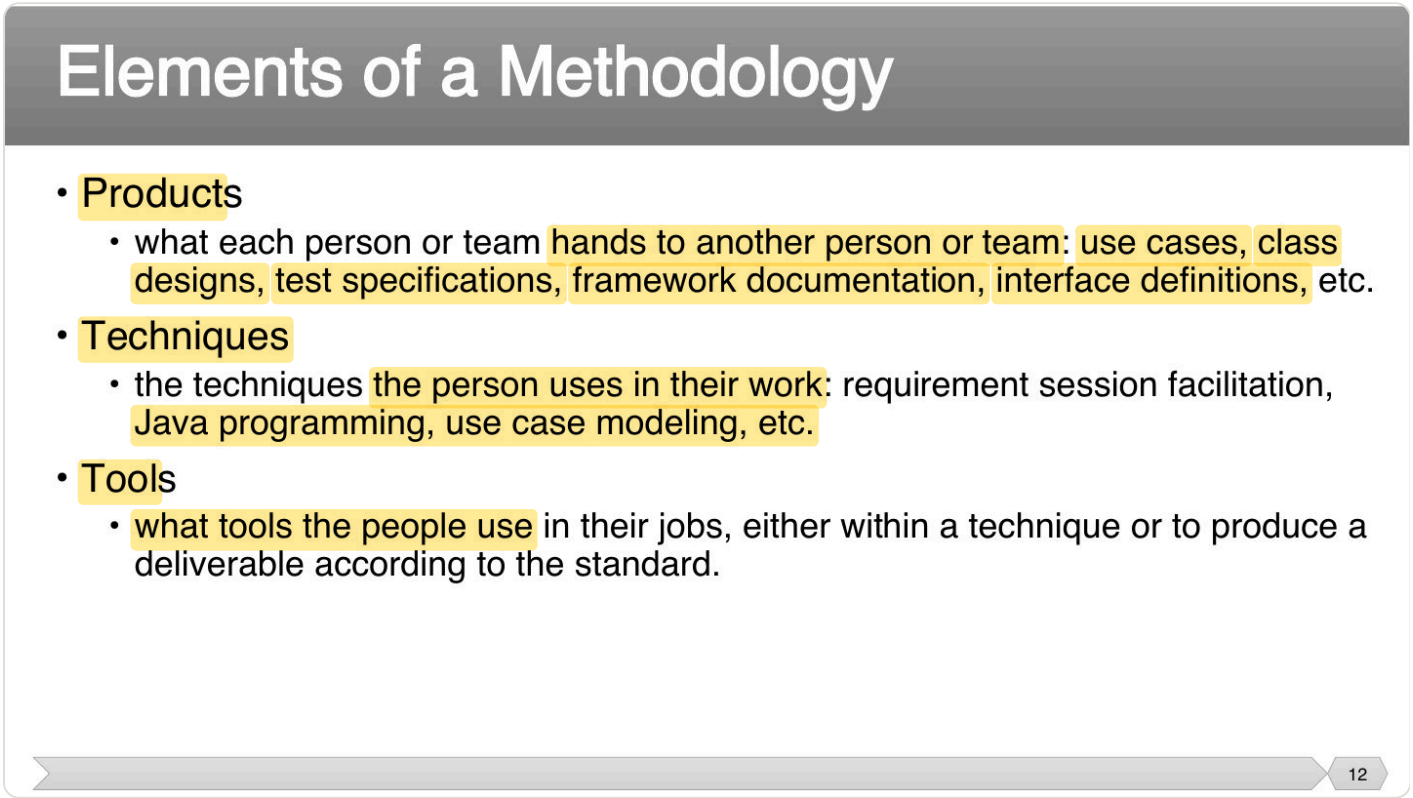
NOTES

1. Who works with whom affects speed and quality. Putting testers with developers gives different results from keeping them apart, so **teams** are a real design choice.
2. A **role** is a job to be done, not a person, and one person may have several. Matching roles to **personalities** matters because someone who hates conflict may struggle as the negotiator.
3. Giving someone a role they can't do helps nobody. Listing **skills** per role tells you who needs training and who to hire.

TIPS & TRICKS

- **STaR**: Skills, Teams, Roles.

SLIDE 12 / 98 Elements of a Methodology



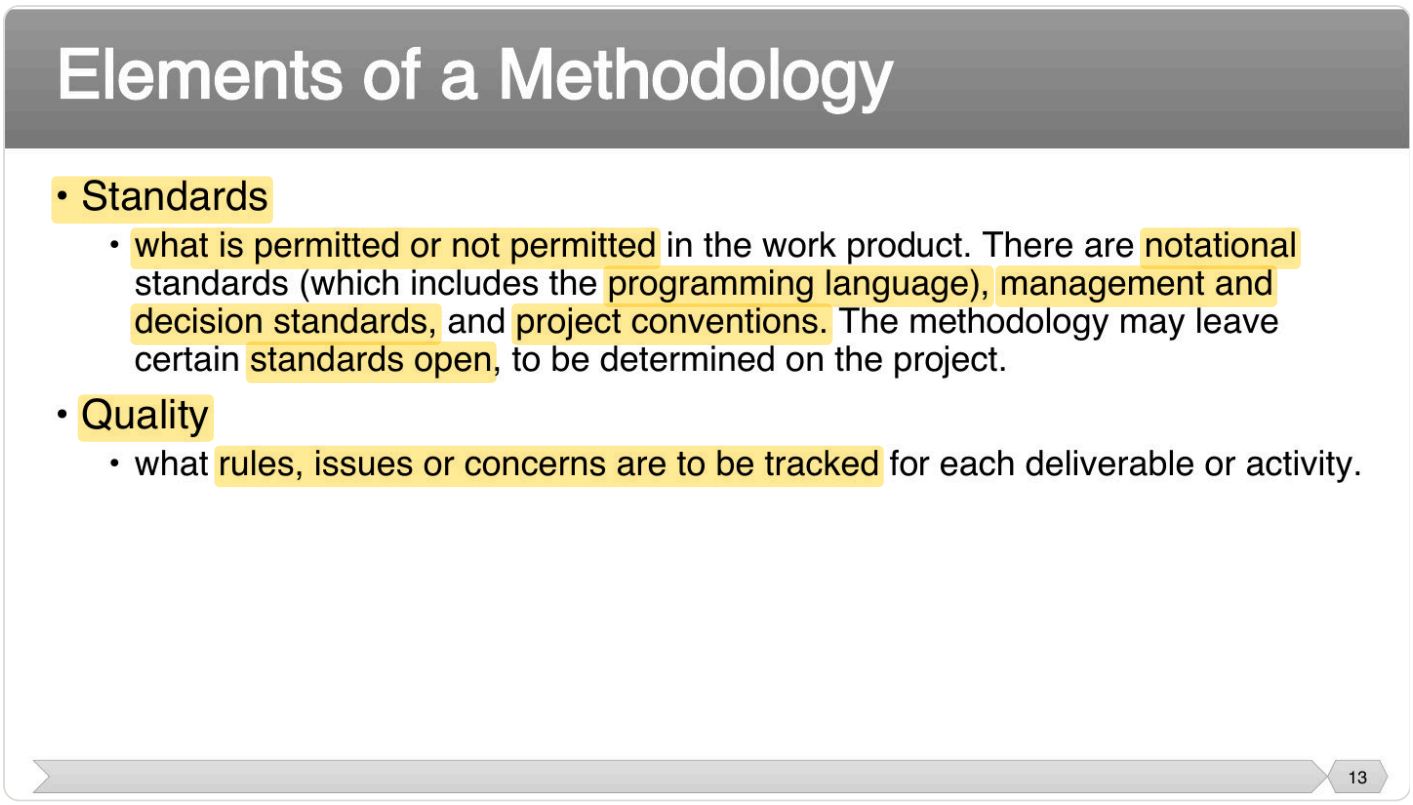
NOTES

1. Ask "what does the next person need from me?". The tester needs test specifications, and that hand-over is a **product**. If it's poor, the next person's work suffers.
2. Two people can make the same product differently. **Techniques** are how the work is done, like modeling use cases, so a methodology can prescribe skills rather than only results.
3. A technique is the method, and a **tool** is what helps you apply it. Use case modeling is a technique, while the diagram software is a tool.

TIPS & TRICKS

- **PuTT**: Products, Techniques, Tools.

 Elements of a Methodology

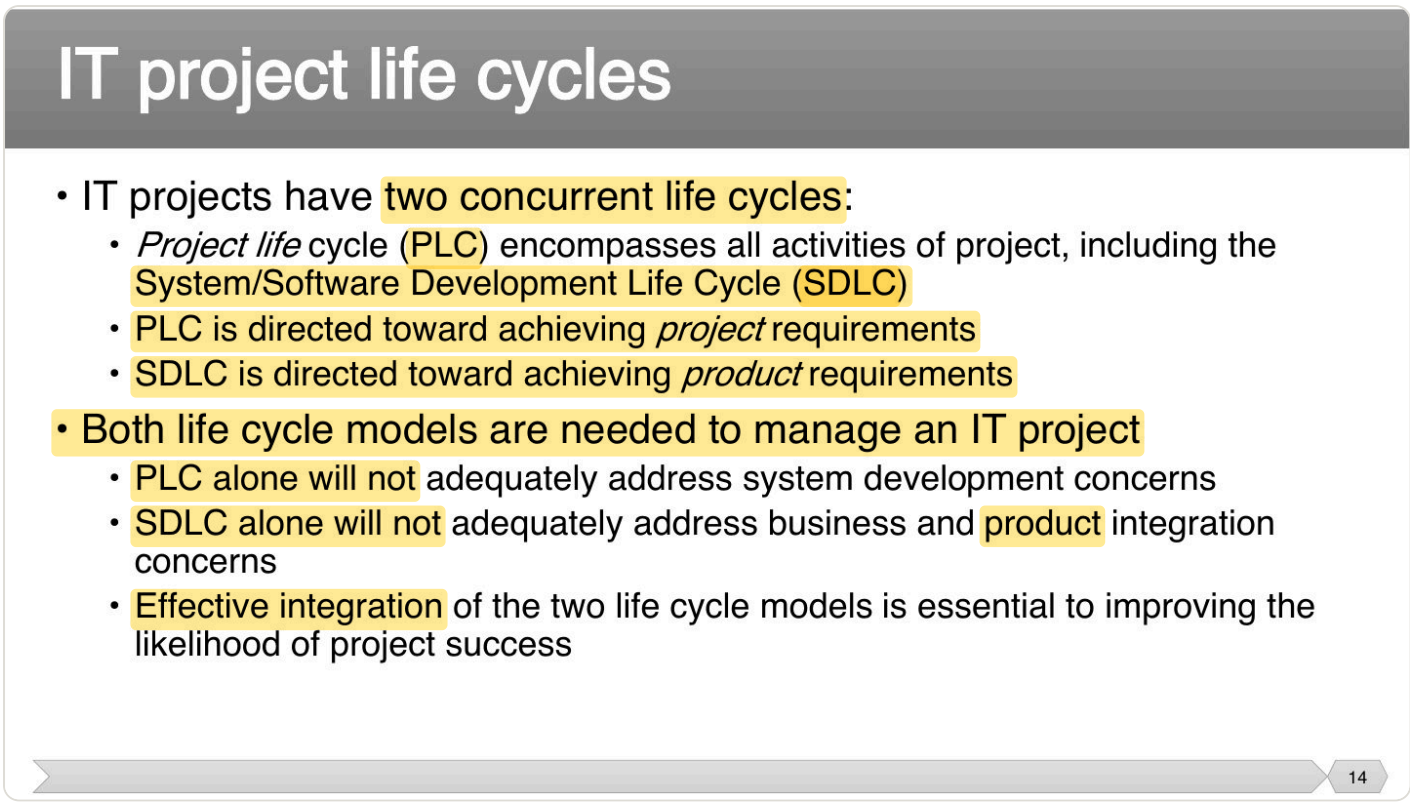


NOTES

1. Why have rules on the work product? Everybody can read and combine each other's work more easily. **Notational** standards are about how things are written, like which language or diagram style. Leaving some **open** allows projects to decide for themselves.

2. **Quality**: which rules, issues or concerns are tracked for each deliverable or activity (e.g. code review required, test coverage).

 IT project life cycles



NOTES

1. A software project is more than coding: someone also handles contracts, budgets and approvals. So the **PLC** contains the development work, the **SDLC**, plus everything around it.
2. The **PLC** targets **project** requirements (scope, time, cost, approvals); the **SDLC** targets **product** requirements (features, quality). A common trap: don't swap them.
3. A perfect product can still fail if nobody arranged the budget, and a well-run project can still build broken software. So you need both, because **neither is enough alone**.
4. The two must line up. A PM who plans a release date without knowing how development works will set dates that can't be met, which is why **integrating** them matters.

EXAM ANGLE

Prediction Q1

The SDLC is directed toward achieving:

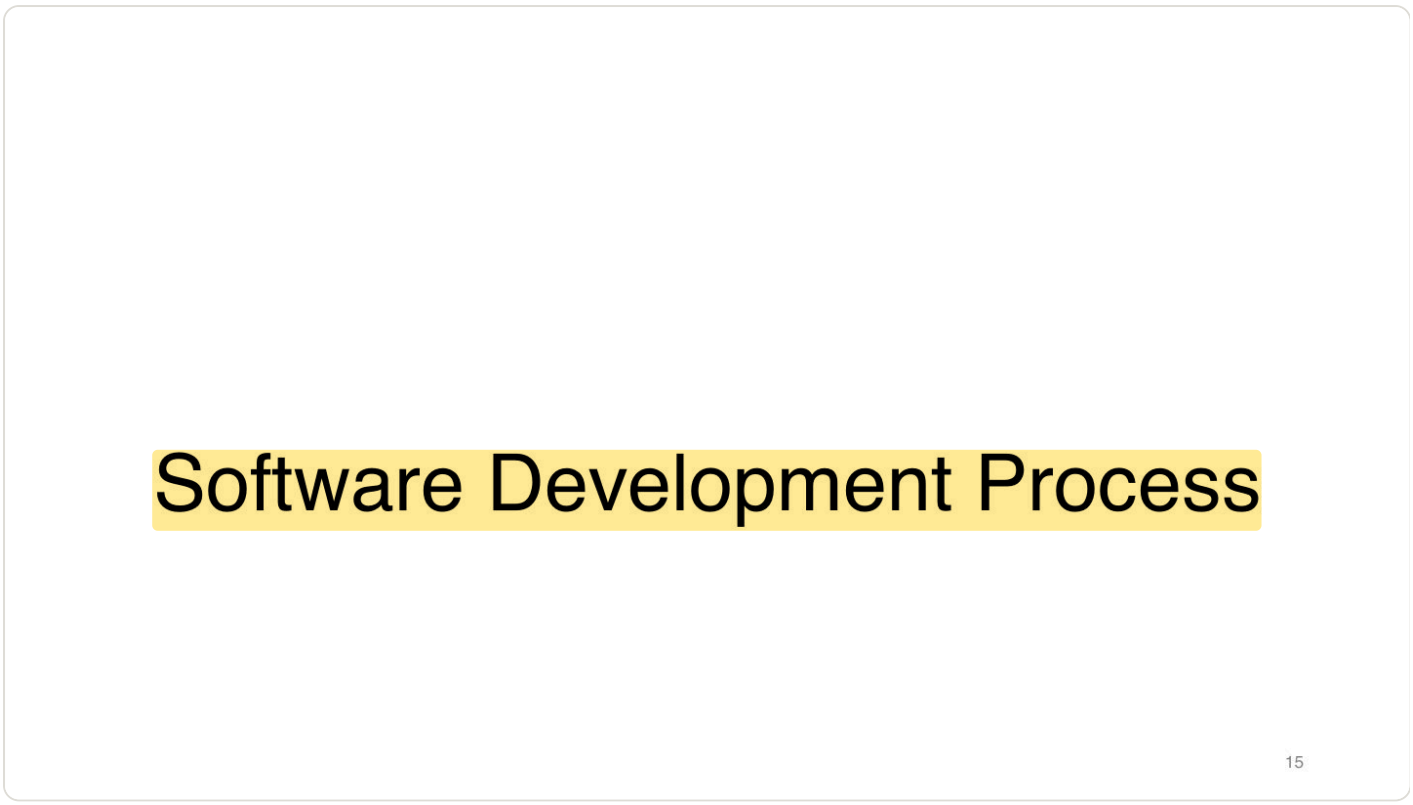
A Procurement

B Project requirements

C **Product requirements**

D Business case approval

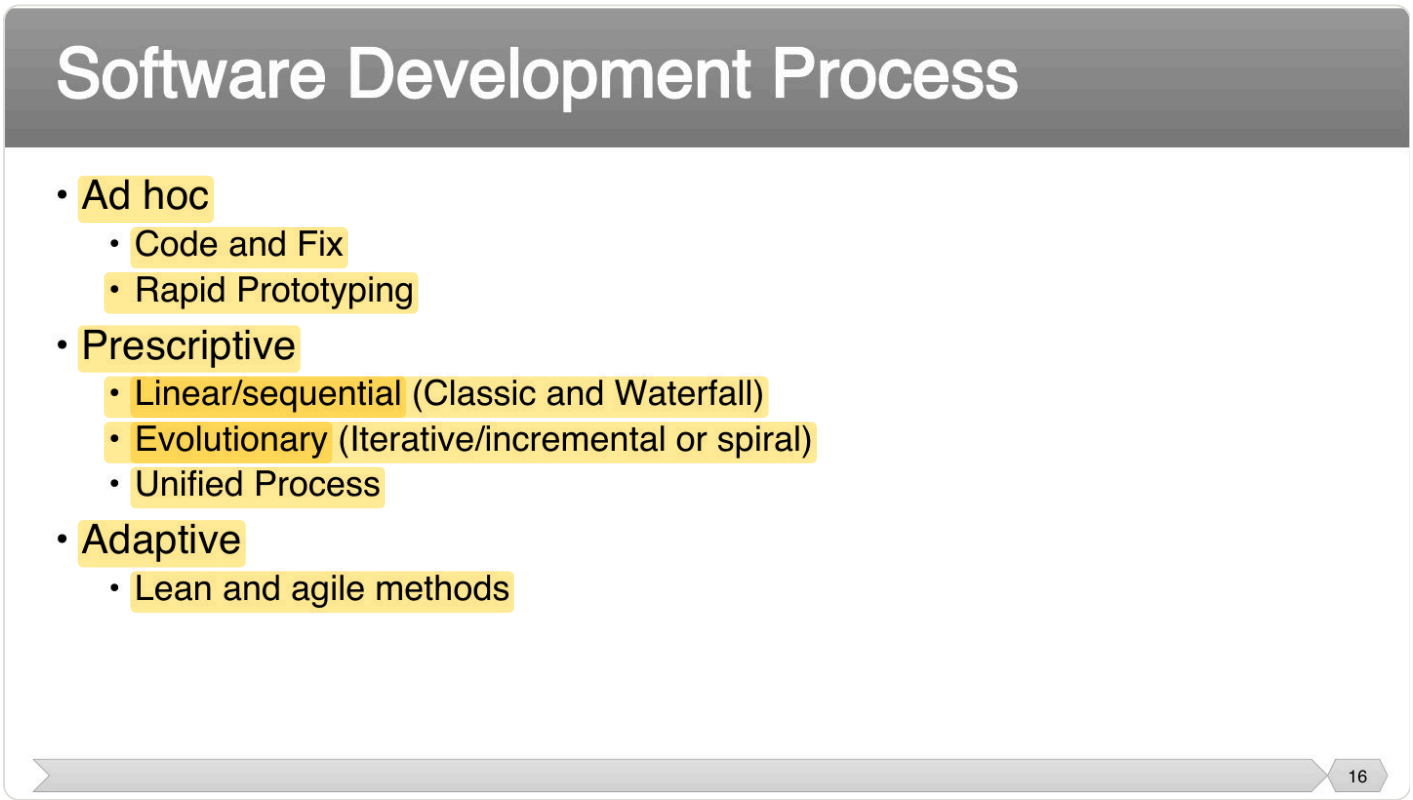
 Slide 15



NOTES

1 Section divider: **Software Development Process**.

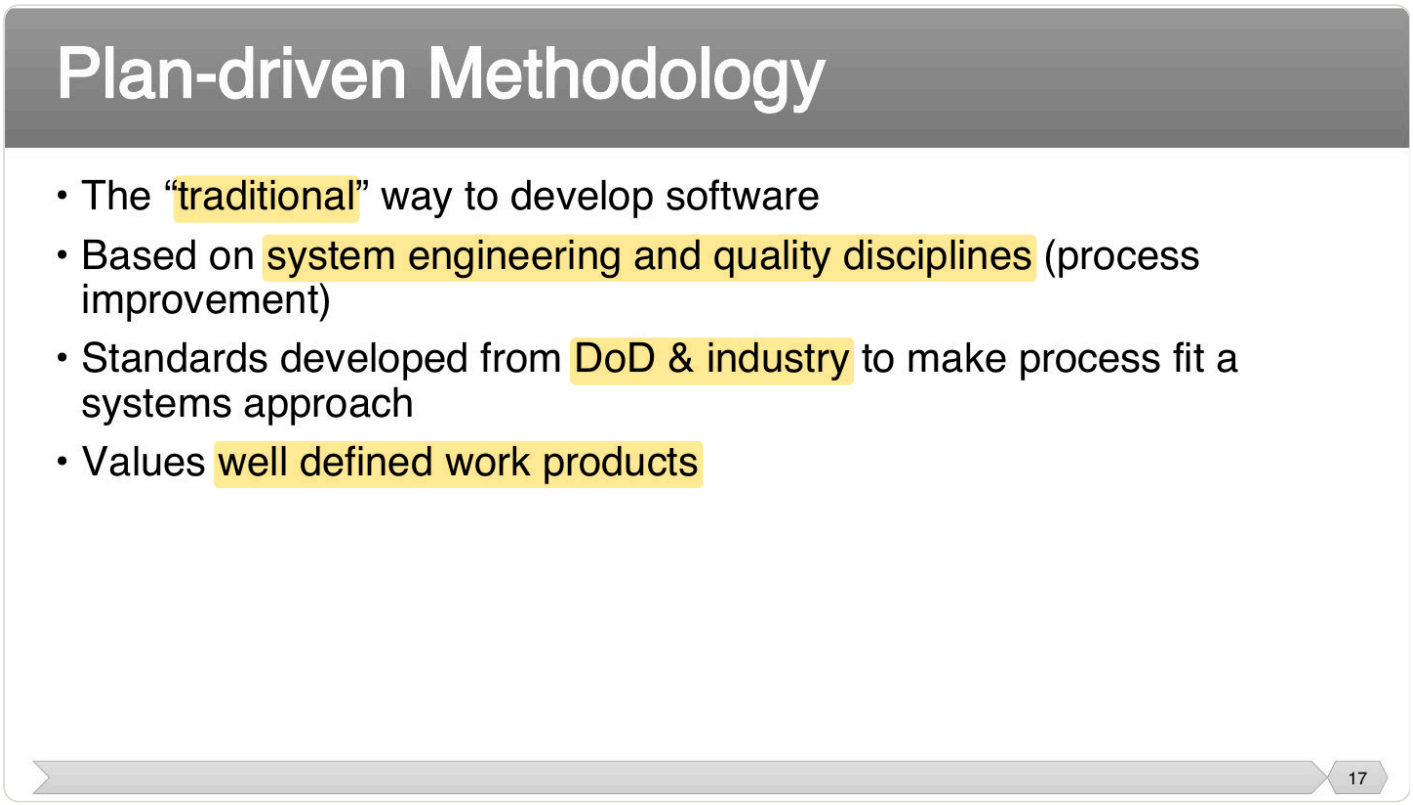
 Software Development Process



NOTES

1. "We just start coding and fix things" is how many small projects begin. **Ad hoc** can work for tiny tools, but it falls apart as projects grow, since nothing is defined.
2. These set out in advance what to do and when. **Prescriptive** doesn't have to mean waterfall: evolutionary approaches are prescriptive too, and it's the defined process that matters.
3. Rather than a detailed plan up front, these expect to change course. **Adaptive** methods like lean and agile treat what you learn as the main input for planning.

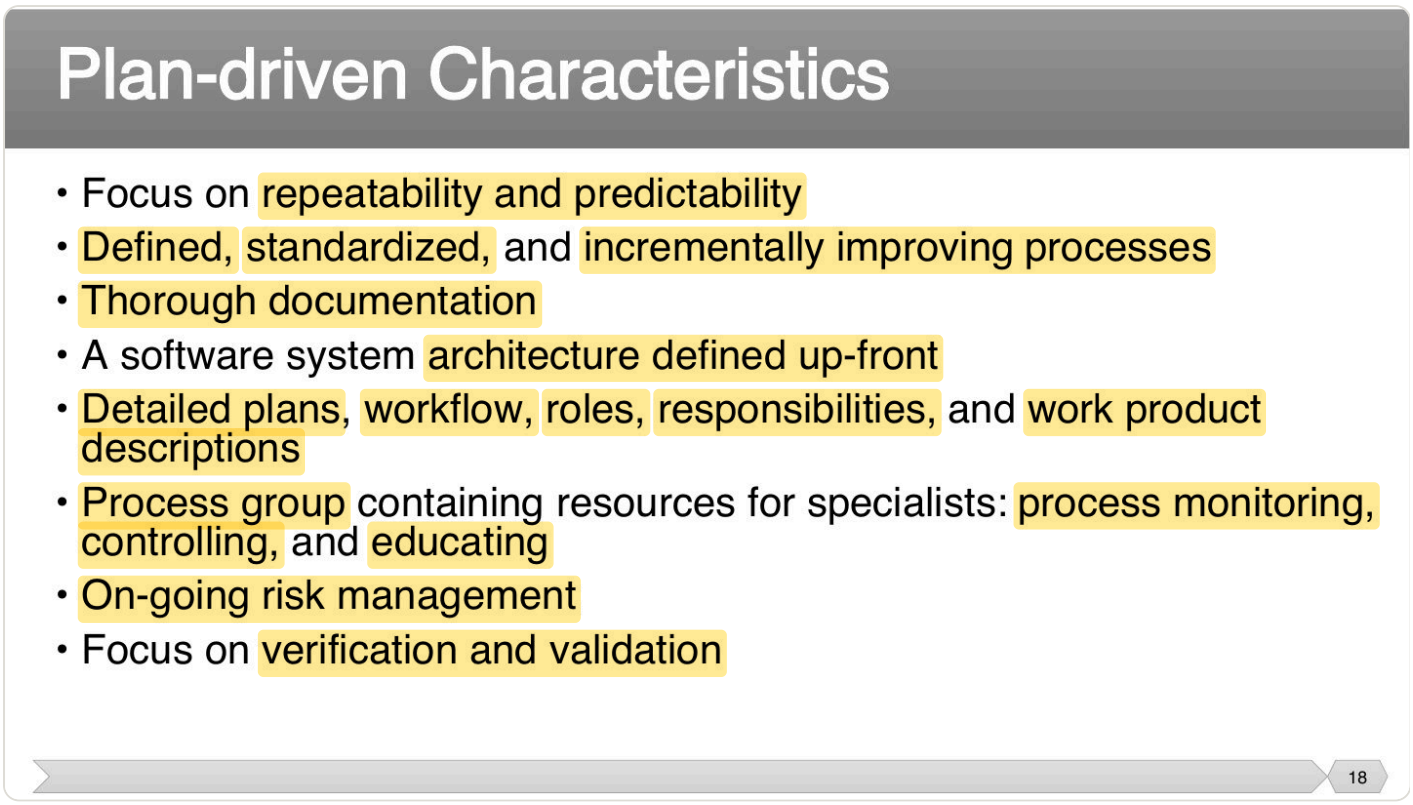
SLIDE 17 / 98 Plan-driven Methodology



NOTES

1. Traditional doesn't mean bad. **Plan-driven** comes from fields where you plan carefully because rework is very expensive...
2. ...such as building aircraft. It borrows from those fields' way of thinking, which is why it's **rigorous**: define it, measure it and improve it...
3. ...and when you're a big customer, like the US **Department of Defense**, you want every contractor working the same way, so you can compare and control them...
4. ...which is why it **values documents**. They let different teams, even different companies, hand work to each other, and they leave a record to check.

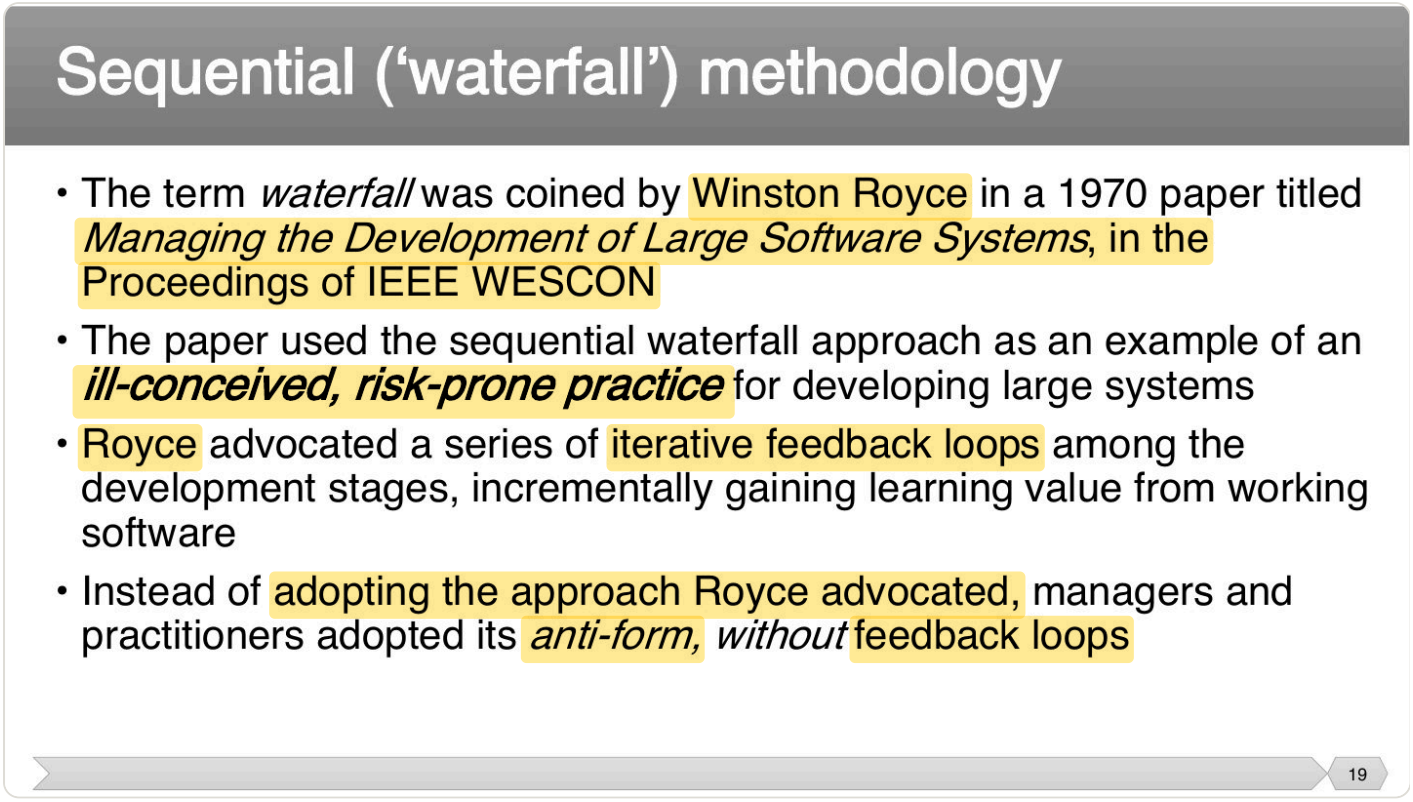
 Plan-driven Characteristics



NOTES

1. If you run the same process twice, you want similar results. That's what **repeatability** is for: being able to predict cost and date, which is exactly what big contracts demand.
2. Writing everything down lets others take over and lets you check against it. An **architecture up front** avoids expensive redesign, provided requirements don't change much.
3. Everyone knows exactly what's expected, which is valuable on very large teams, though it's heavy for small ones. A **process group** exists just to keep that machine running.
4. Because you've committed early, surprises are costly, so risk gets watched throughout. **Verification** asks "did we build it right?" and **validation** asks "did we build the right thing?".

 Sequential (‘waterfall’) methodology



NOTES

1. It’s an irony worth remembering: the name most people use comes from a paper that warned against it. **Royce** is the only historical name you need here.
2. He didn’t think testing only at the end was sensible for big systems. By then, fixing a design flaw is very expensive. So he called the pure version **risk-prone**.
3. His idea was to go back and improve earlier stages using what you learn. So **feedback loops** are the part of his paper people forgot.
4. Industry adopted its **anti-form**, the version without feedback loops. A common trap: “Royce recommended the pure waterfall” is false.

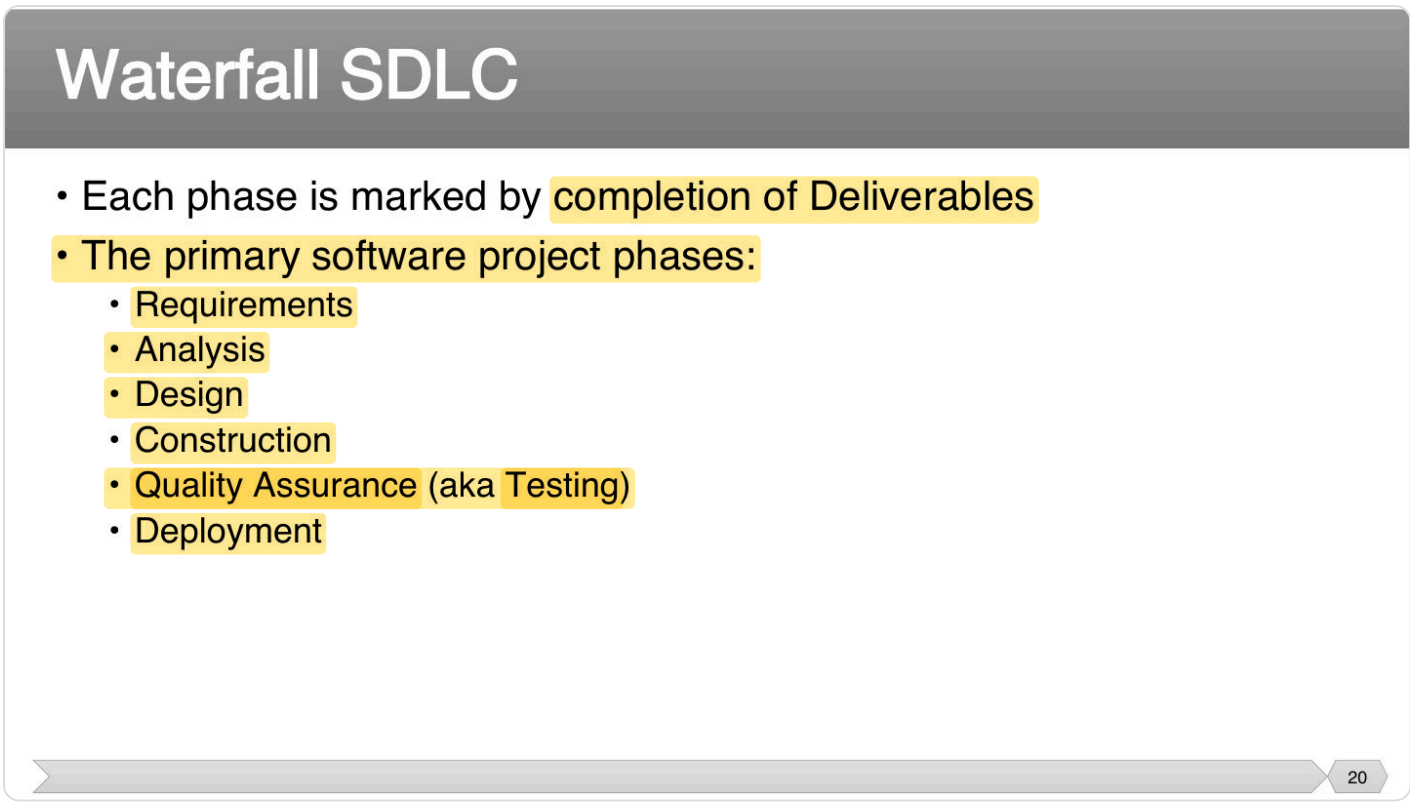
EXAM ANGLE

Prediction Q1

Winston Royce’s 1970 paper presented the pure sequential waterfall as an ill-conceived, risk-prone practice.

- T **True**
- F False

 Waterfall SDLC



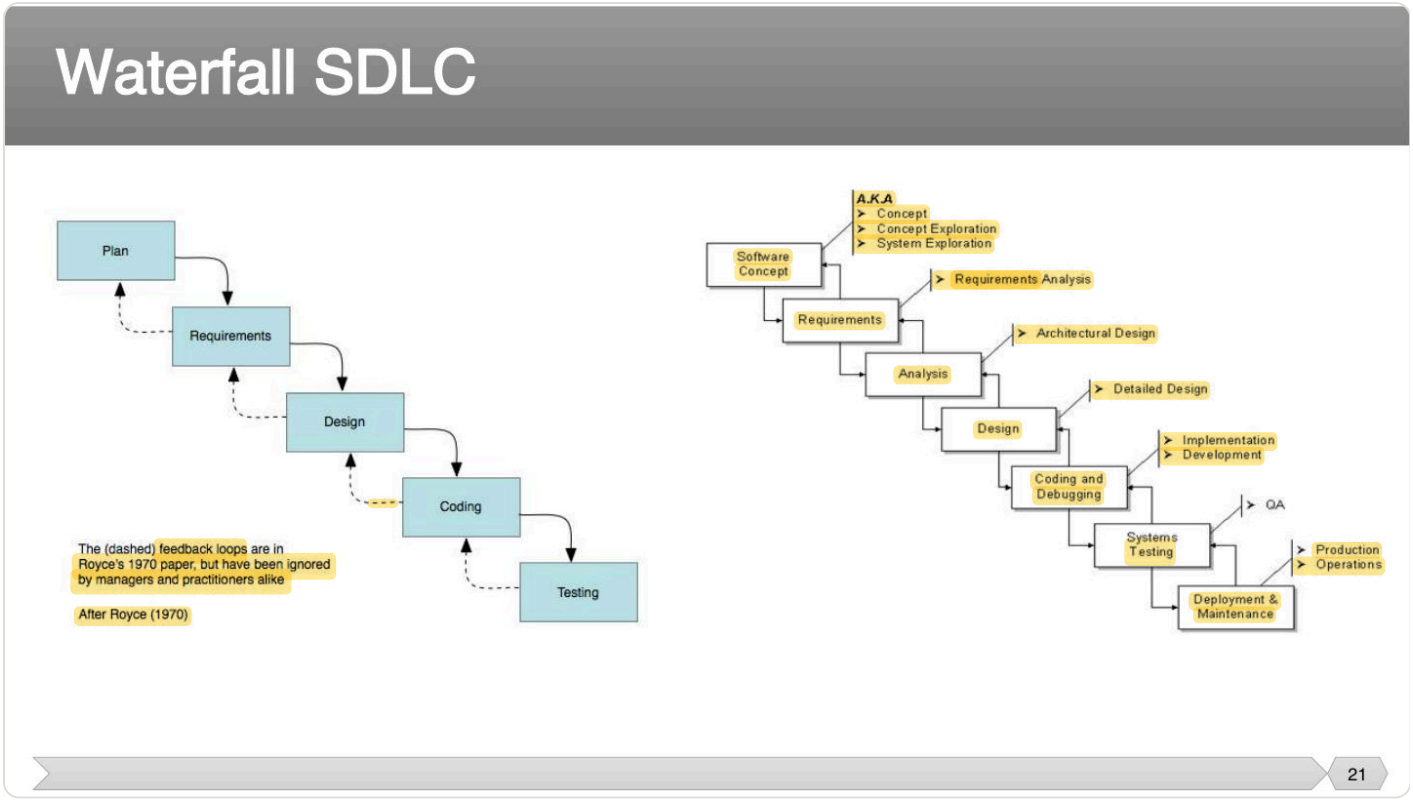
NOTES

1. The idea is that you don't start design until requirements are signed off. Finished **deliverables** keep control tight, but they also make later changes expensive.
2. Each stage depends on the last: you can't design what hasn't been analyzed. Mind the order, and that **testing** comes near the end: that's the main weakness.

TIPS & TRICKS

- The order matters here: **R**eal **A**rchitects **D**raw, **C**onstruct, **Q**uality-check, **D**eploy. Requirements, Analysis, Design, Construction, Quality assurance, Deployment.

 Waterfall SDLC

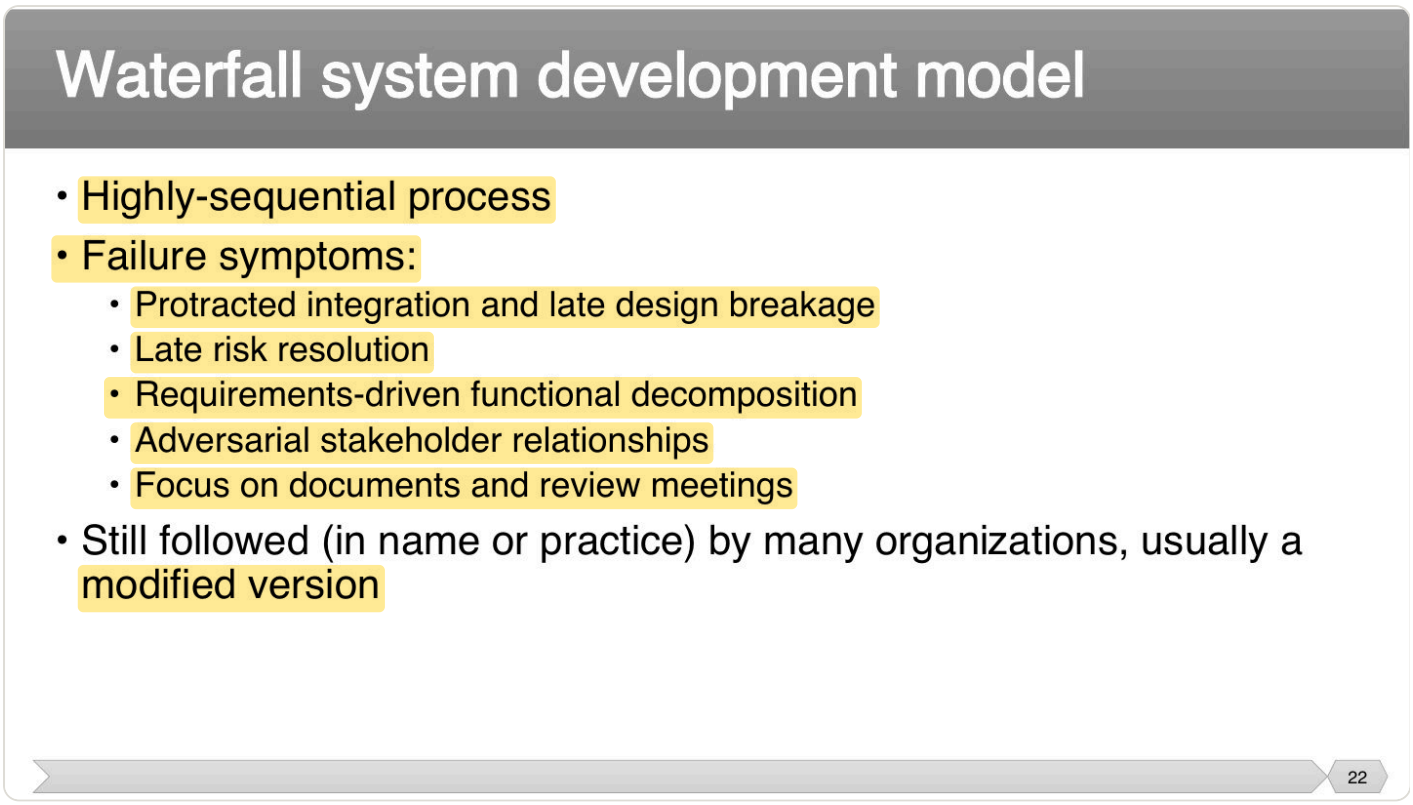


NOTES

1. Royce's waterfall: plan → **requirements** → design → coding → testing, with dashed **feedback loops** back to the previous step. The note on the left says those loops were in Royce's paper but have been ignored by managers and practitioners alike.

2. On the right, a more detailed version (concept → requirements → architecture → detailed design → coding → **testing** → deployment). Either way, testing comes last, which is why problems are found late.

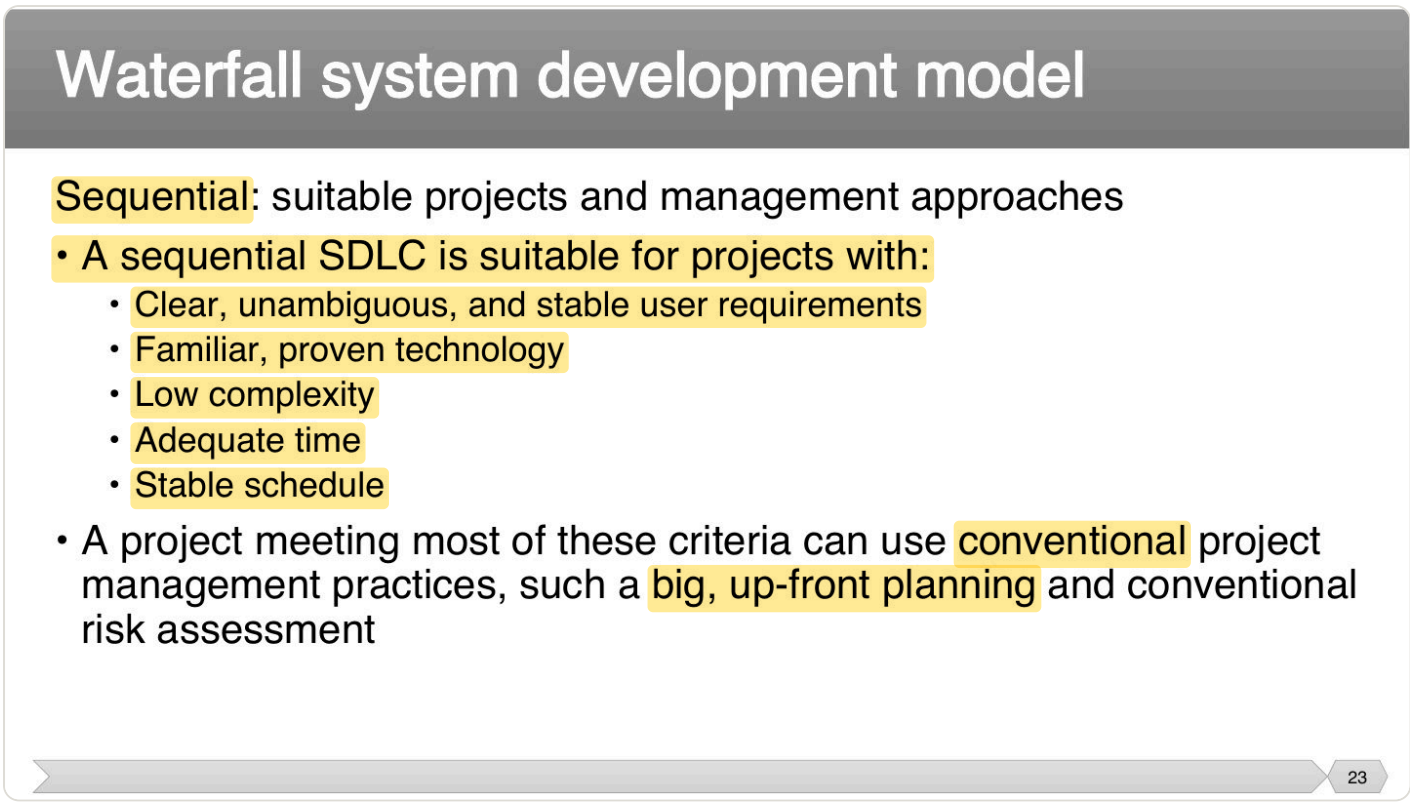
 Waterfall system development model



NOTES

1. Waterfall is a **highly sequential** process: no phase overlaps.
2. Nothing is combined until late, so the first real proof the parts fit comes at the end. That's when design flaws appear and are most expensive, hence **late design breakage** and **late risk resolution**.
3. **Adversarial stakeholder relationships** (arguing over signed documents) and a **focus on documents and reviews** instead of working software.
4. Why does it survive? It's easy to explain and to contract for, and some projects suit it. Most organizations use a **modified version**, adding some iteration, rather than the pure form.

 Waterfall system development model



NOTES

1. Every item is a form of "we already know what we're doing". If requirements and technology are both familiar, there's little to discover, so a fixed plan works. If one is missing, **sequential** gets riskier.
2. When there's little to learn along the way, planning it all at the start is efficient. **Conventional** methods are the right call here, so don't assume agile is always better.

TIPS & TRICKS

- **FALConS**: Familiar technology, Adequate time, Low complexity, Clear requirements, Stable schedule.

EXAM ANGLE

Past exam · Midterm 251 · V1 · Case Q2 (also V2) Q1

Given the evolving requirements (student usability, advisor requests, IT integration rules), which development approach (Agile, Waterfall, or Hybrid) would you recommend, and why would it be the best fit for this project?

Answer: Agile (Scrum) (0.5), because it handles the evolving requirements from students, advisors and IT, delivers iteratively so each sprint gets feedback, and integrates with DevOps pipelines for frequent tested releases (0.5). Justify it against the slide criteria: the requirements are reasonably clear but not stable, so waterfall's "clear, unambiguous, stable requirements" condition fails.

Prediction Q2

Which project suits a sequential (waterfall) SDLC?

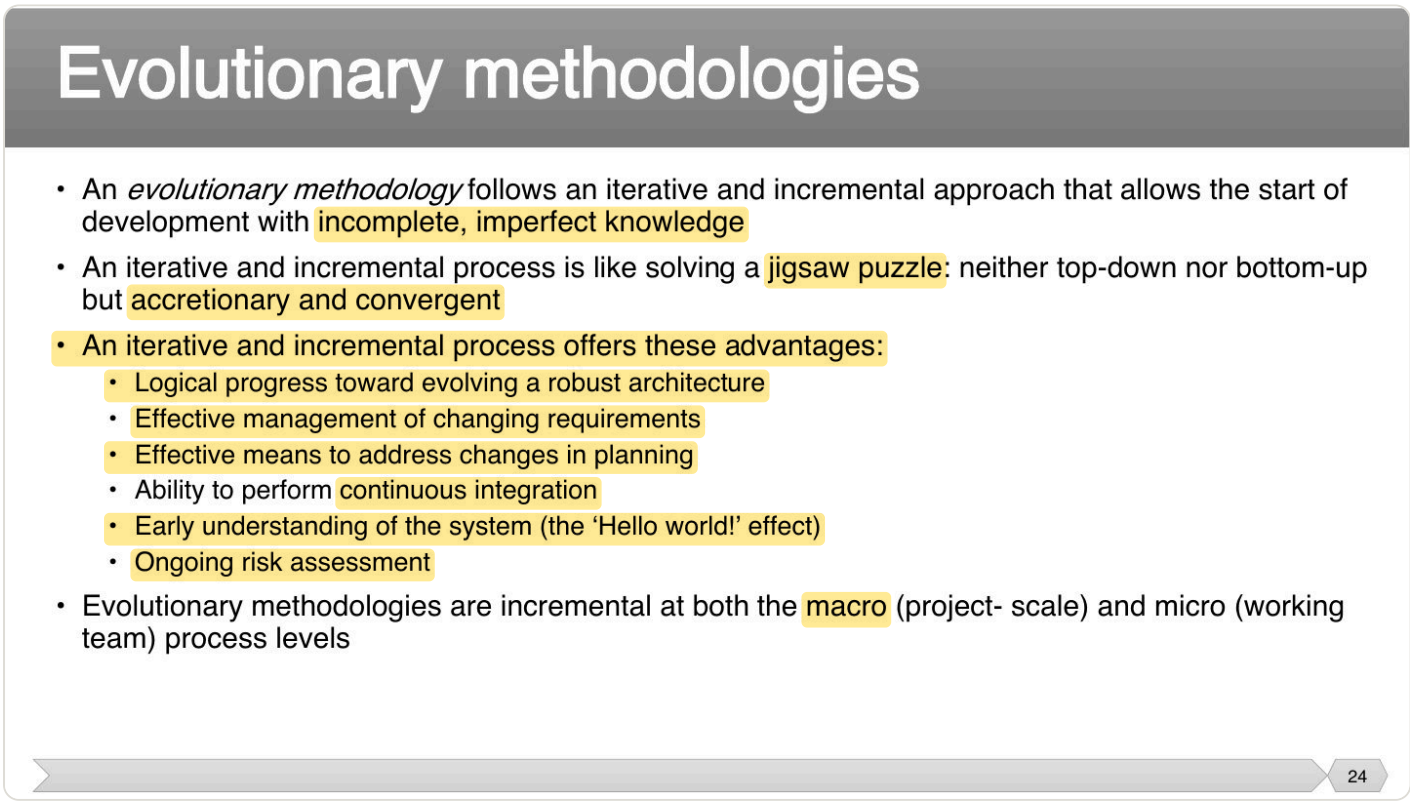
A Unclear requirements and new technology

B High complexity with a very short schedule

C **Stable requirements, familiar technology, low complexity**

D Requirements expected to change weekly

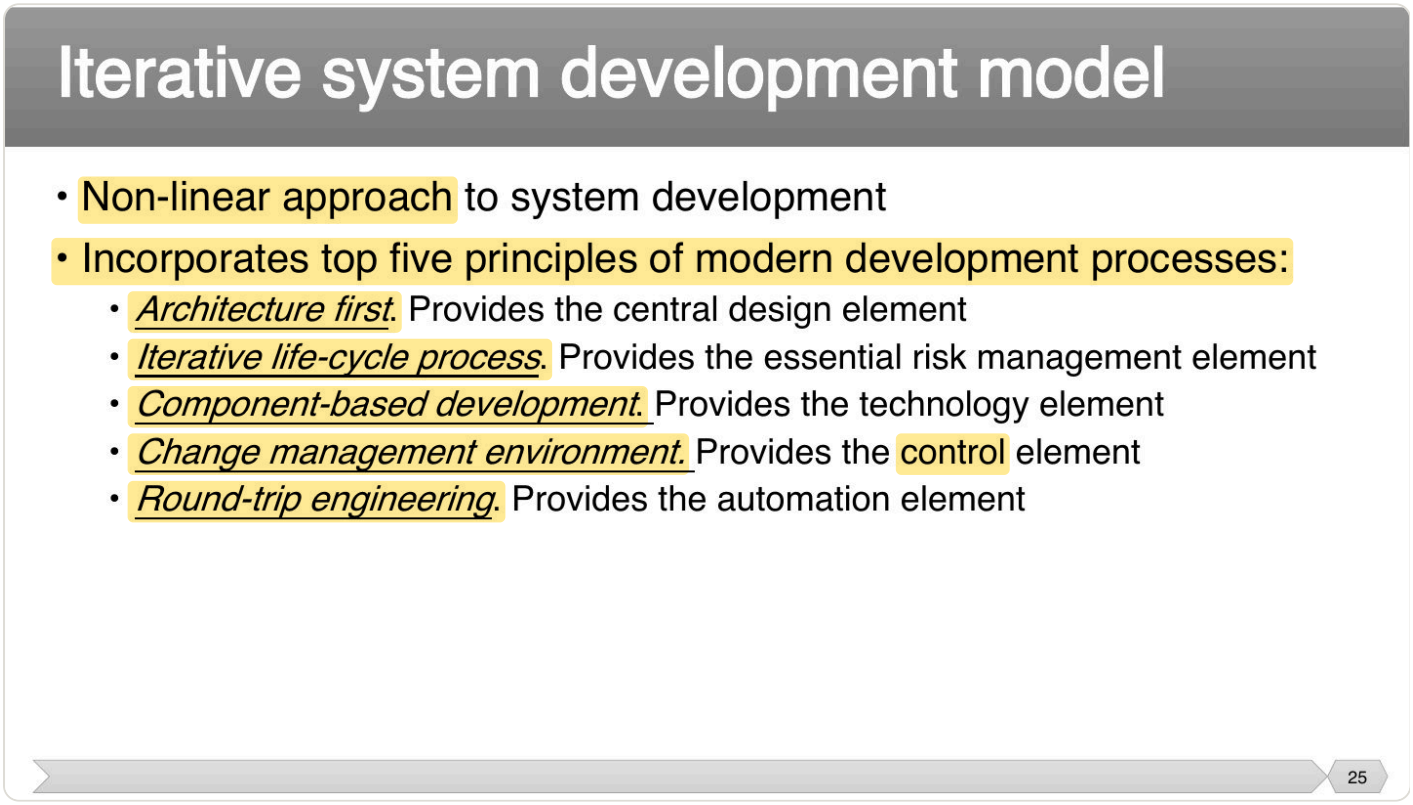
SLIDE 24 / 98 **Evolutionary methodologies**



NOTES

1. Waterfall needs you to know a lot before starting. **Evolutionary** approaches accept you don't, so you begin with what you know and learn the rest by building.
2. It is like a **jigsaw puzzle**: neither top-down nor bottom-up, but **accretionary and convergent** (pieces accumulate until the picture emerges).
3. Each benefit comes from working in small pieces. Because you integrate often, you spot problems fast, and early working results, even a "Hello world!", show the team that things connect. That's **continuous integration** in practice.
4. The same pattern works at two scales. The whole project grows in stages (**macro**), and within each stage, teams build small parts bit by bit (micro).

SLIDE 25 / 98 Iterative system development model



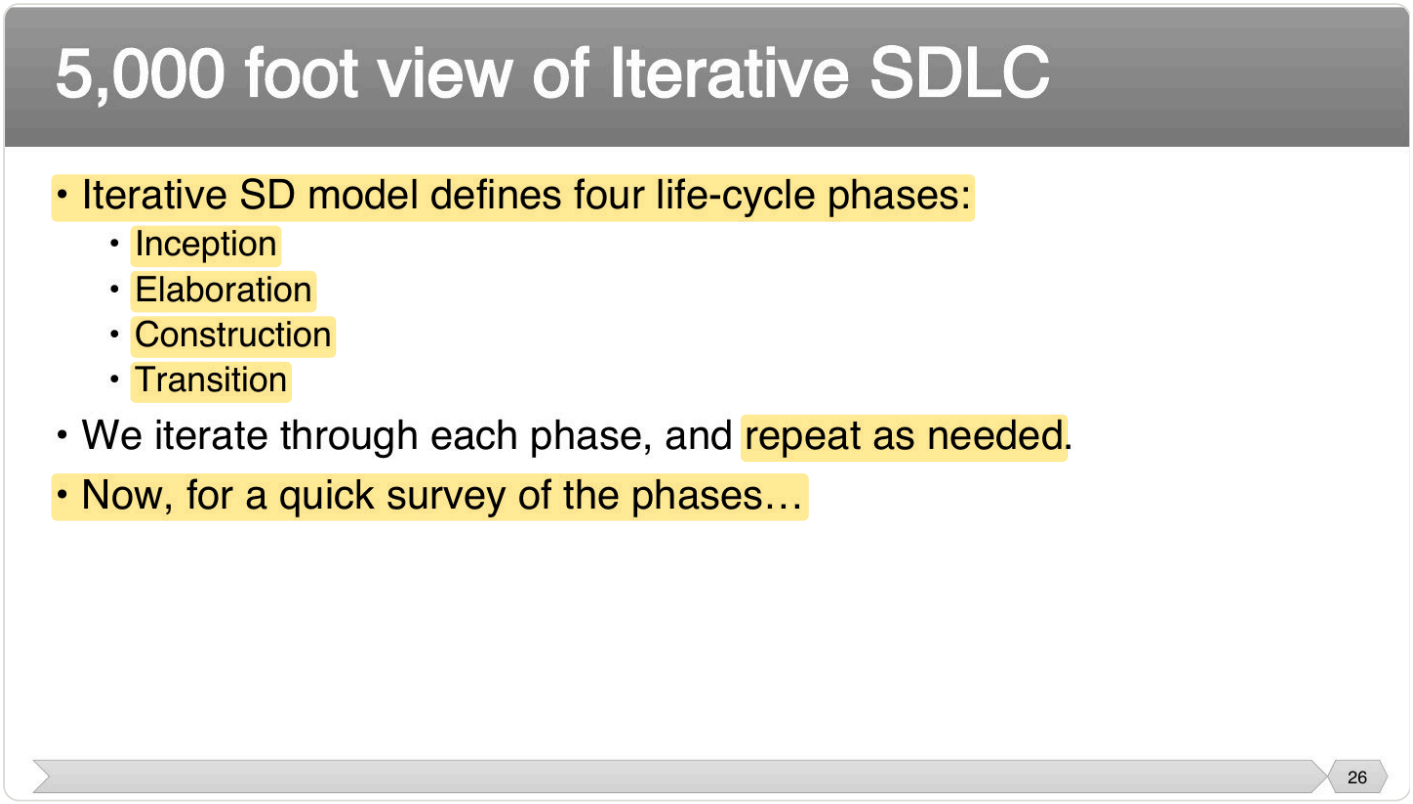
NOTES

1. You don't pass through each stage once and leave it behind. **Non-linear** means you can return to earlier work as you learn, which fixes waterfall's main weakness.
2. Why **architecture first**? It's the riskiest and hardest thing to change later, so get it stable early. And working in iterations surfaces **risks** sooner, which is why it's called the risk-management element.
3. Reusing components saves building everything from scratch. If changes happen often, you need a system to **control** them, and automation keeps models and code matching, so nobody works from outdated designs.

TIPS & TRICKS

- **CIRCA**: Component-based development, Iterative life cycle, Round-trip engineering, Change management environment, Architecture first.

SLIDE 26 / 98 **5,000 foot view of Iterative SDLC**



NOTES

1. Each phase answers a different question. Should we build it? Can we build it? Build it. Can they use it? That's a good way to recall **inception, elaboration, construction, transition**.
2. A phase isn't one pass, because you may need several rounds before the risk is low enough to move on. That's the point of **repeated as needed**.

TIPS & TRICKS

- **InfECT**, already in order: Inception, Elaboration, Construction, Transition.

EXAM ANGLE

Prediction Q1

Which is the correct order of the iterative SDLC phases?

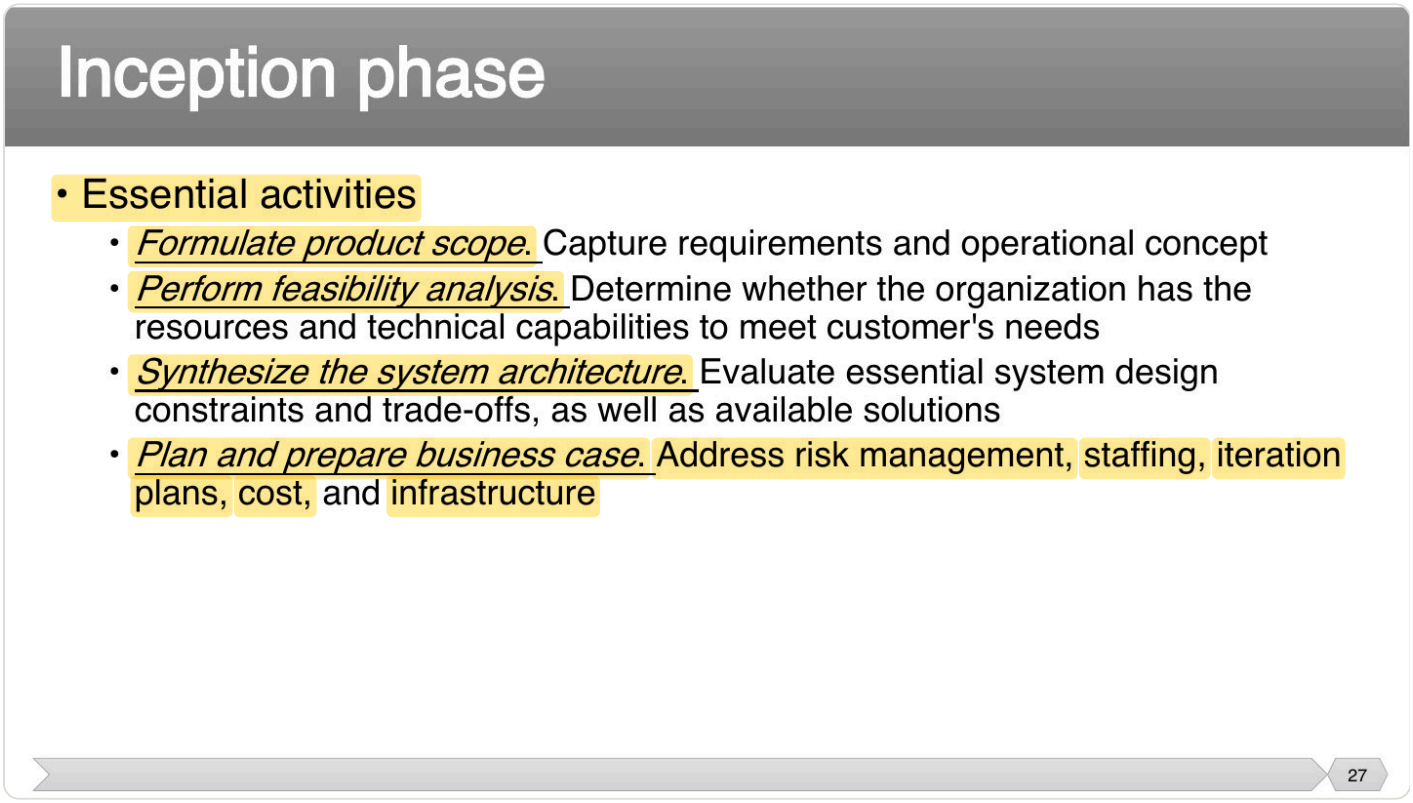
A Inception, construction, elaboration, transition

B Planning, design, build, deploy

C Elaboration, inception, construction, transition

D **Inception, elaboration, construction, transition**

SLIDE 27 / 98 **Inception phase**



NOTES

1. Before spending much, work out what you're actually building and for whom. **Product scope** at this stage is a rough outline, not every detail...
2. ...and check it's possible with what you have. **Feasibility** can stop a bad project at the cheapest moment...
3. ...and sketch how it could work. You're not designing it all yet, only checking that one workable **architecture** exists...
4. ...and finally justify it. The **business case** is what decision-makers use to agree whether to go on.

 Elaboration phase

Elaboration phase

- Most critical phase of the four
- Essential activities
  - *Elaborate the vision.* Detail elements of the vision that drive architectural or planning decisions
  - *Elaborate the process and infrastructure.* The construction process and environment are established here
  - *Elaborate the architecture and select reusable (internal or COTS) components.* Baseline the architecture as quickly as possible and demonstrate that the architecture will support the vision at reasonable cost in reasonable time

28

NOTES

1. **Elaboration** is the **most critical** of the four phases: it removes the biggest risks before heavy spending starts.
2. You can't detail everything yet, so focus on what shapes big decisions. If the vision says "millions of users", that detail changes the **architecture**, and the color of a button doesn't.
3. Everything construction needs is set up now: tools, environments and ways of working. Getting the **infrastructure** right here avoids slowing down a much larger team later.
4. To **baseline** means to fix it as the reference, and only after showing that it works. Reusing **COTS** (off-the-shelf) parts saves effort, as long as they fit the architecture.

EXAM ANGLE

Prediction Q1

Which iterative phase is described as the most critical?

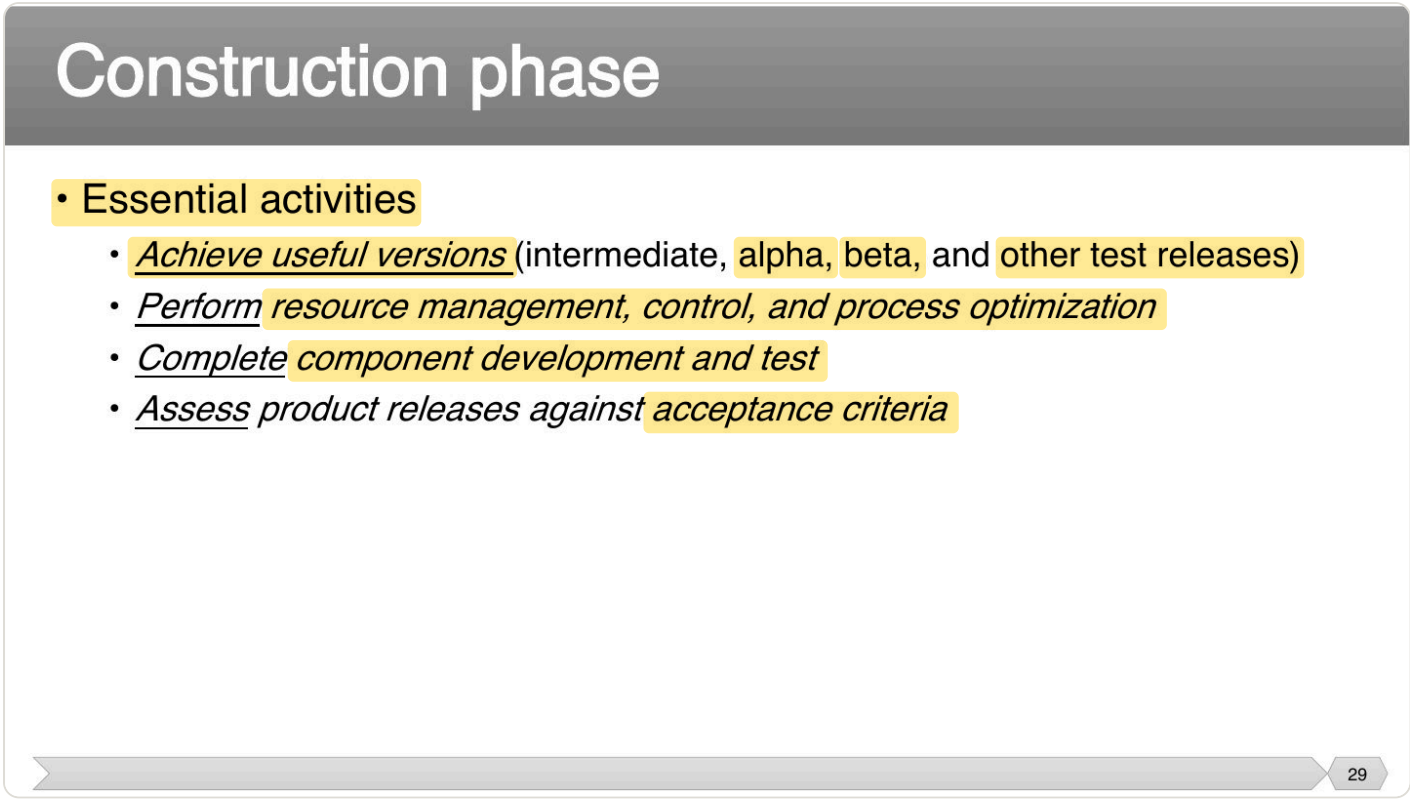
A **Elaboration**

B Transition

C Construction

D Inception

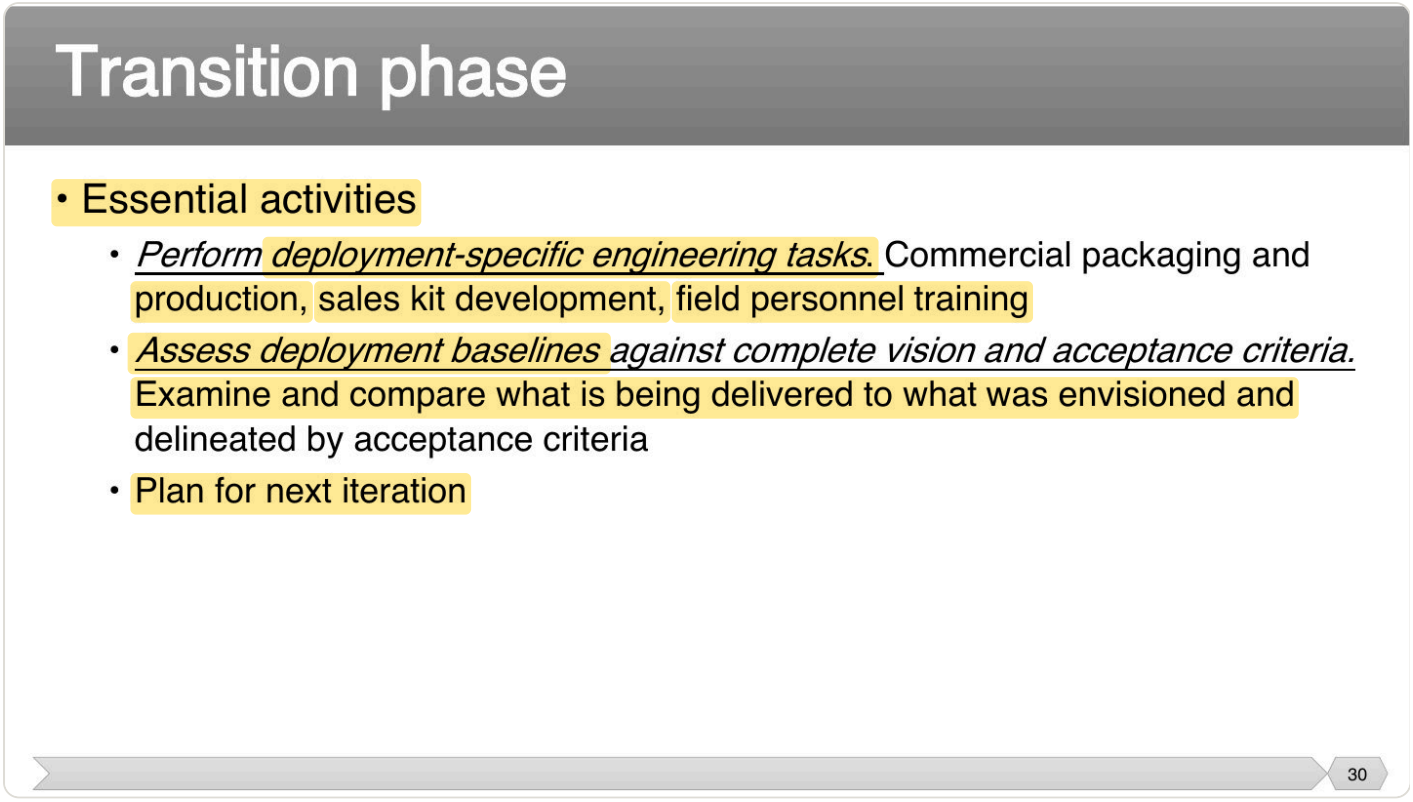
 Construction phase



NOTES

1. Unlike waterfall, you don't wait until the end to see something. Each **useful version** can be tried, and an alpha or beta gets real feedback early...
2. ...this is when most people and money are in use. So the PM's work shifts to **resource management** and keeping the flow efficient...
3. ...it's also the phase where the bulk of **development and testing** gets done, with the hard design risks already handled...
4. ...and "done" isn't just an opinion. Each release gets checked against **acceptance criteria**, agreed in advance, so that everybody knows when it's good enough.

 Transition phase



NOTES

1. Working software isn't yet a usable product for customers. Someone still has to package it, and train users and support staff, which is what **deployment-specific** tasks mean.
2. This is the final reality check: did we get what we set out to get? **Comparing** the result with the original vision reveals gaps before you call it finished.
3. Most products carry on after a release, so the last phase feeds into the next. **Planning the next iteration** is why the model is drawn as a repeating cycle.

 Comparative expenditure profiles

Comparative expenditure profiles

Waterfall		Iterative	
Activity	Cost	Cost	Activity
Management	5%	10%	Management
Requirements	5%	10%	Requirements
Design	10%	15%	Design
Code & Unit Testing	30%	25%	Implementation
Integration & Test	40%	25%	Assessment
Deployment	5%	5%	Deployment
Environment	5%	10%	Environment
Total	100%	100%	Total

NOTES

1. **Waterfall** spends most of its budget late: 30% on code and unit test and **40% on integration and test**, because problems surface only when everything comes together.
2. It costs more up front, as more is spent on **management** and design. But you pay less for late surprises, since testing and integration are spread throughout. Iterative moves the spending earlier, when mistakes are cheaper.

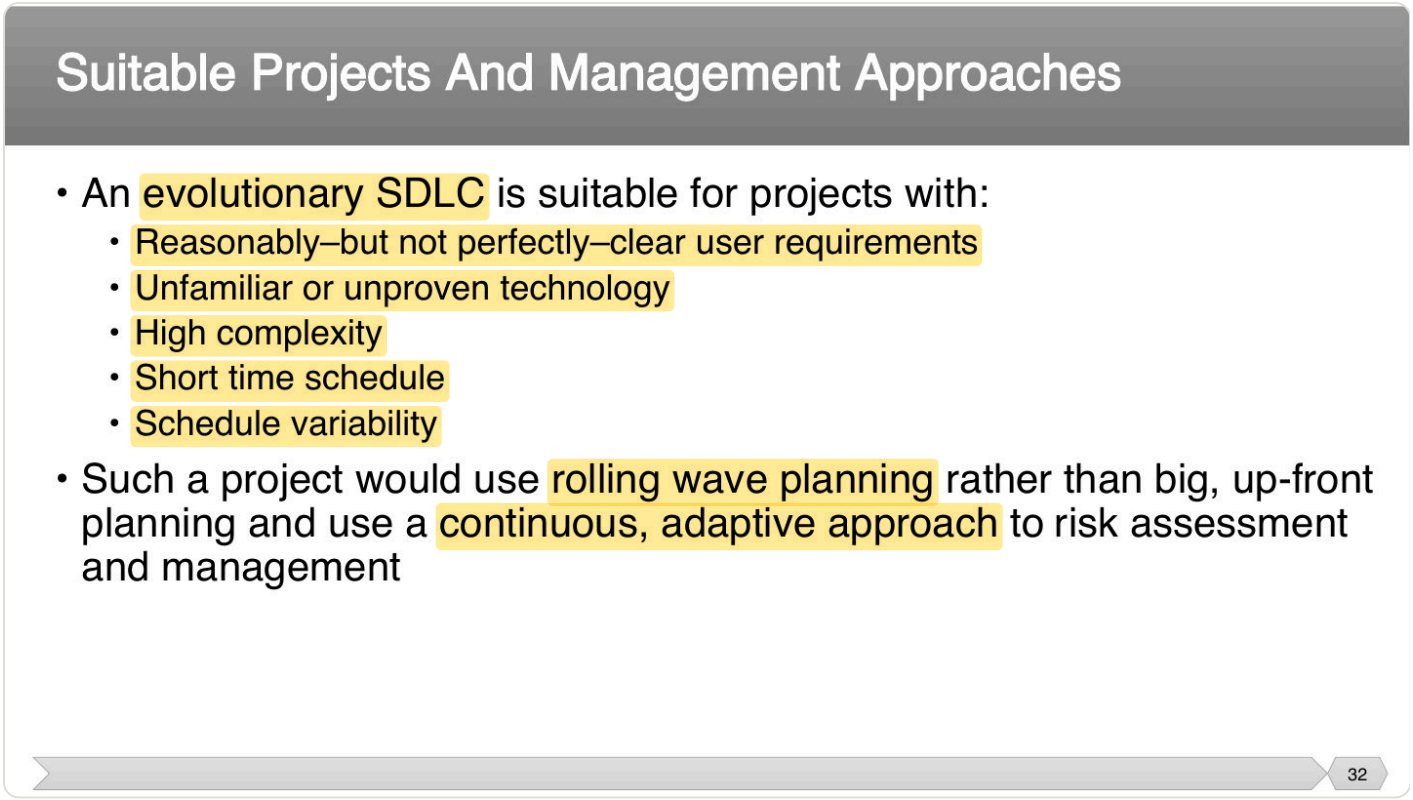
EXAM ANGLE

Prediction Q1

In the waterfall expenditure profile, which activity takes the largest share of cost?

- A **Integration and test (40%)**
- B Requirements (5%)
- C Design (10%)
- D Management (5%)

 Suitable Projects And Management Approaches



NOTES

1. Compare with slide 23: everything flips. You don't fully know what's wanted or how to do it, and time is tight. So an **evolutionary SDLC** lets you learn and deliver as you go. Remember **RUSHeS**.
2. Such a project uses **rolling wave planning** (detailed plans for the near term, rough plans for later) instead of big up-front planning, and a **continuous, adaptive** approach to risk.

TIPS & TRICKS

- **RUSHeS**, the opposite of slide 23: Reasonably clear requirements, Unfamiliar technology, Short time, High complexity, Schedule variability.

EXAM ANGLE

Past exam · Midterm 251 · V1 · MCQ 6 (also V2) Q1

In which type of project would an iterative development approach be most beneficial?

A **A project where requirements are unclear and expected to evolve over time.**

> Iterative development refines ideas through feedback, which suits unclear or evolving requirements.

B A project with well-defined, unchanging requirements from the start.

C A project that must deliver the entire product only at final completion.

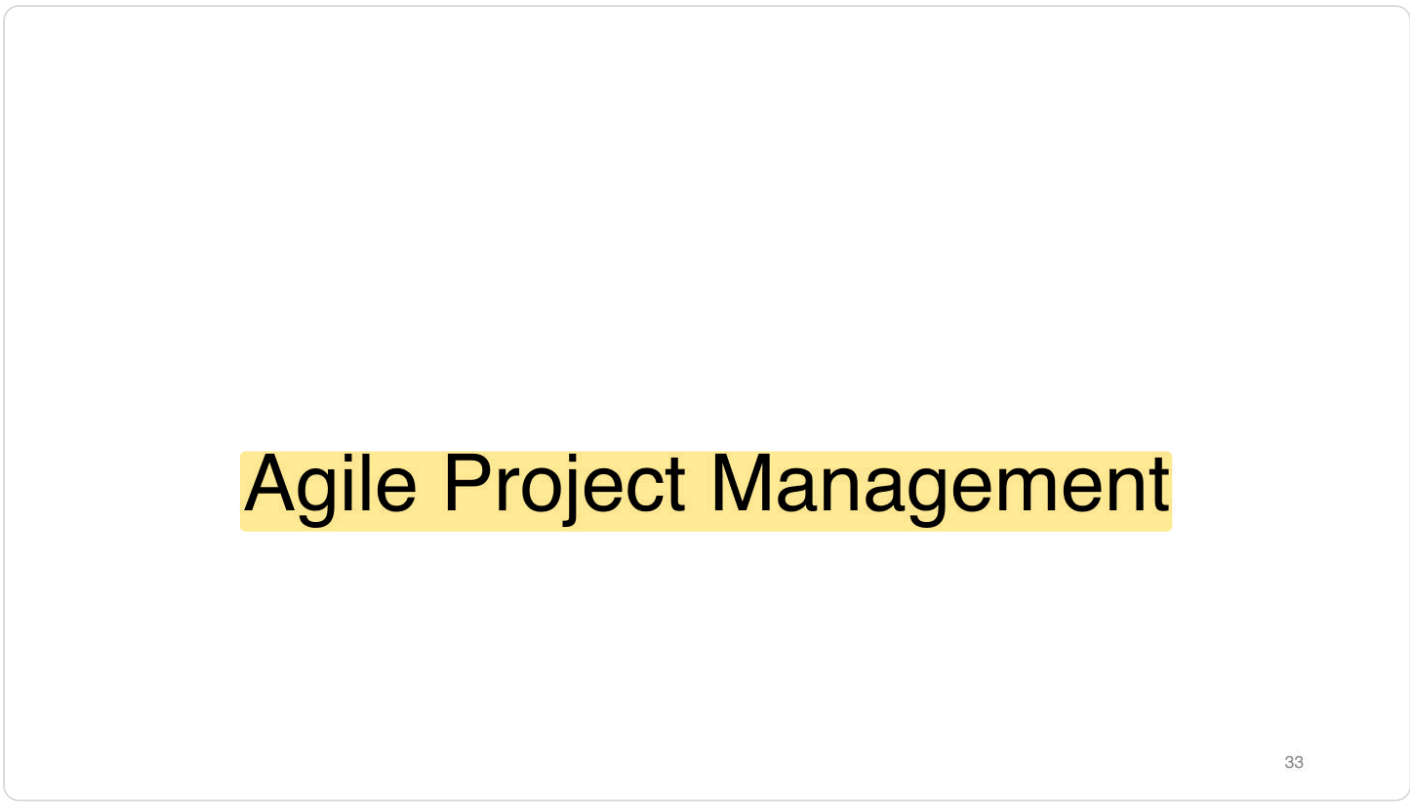
D A project that cannot accommodate stakeholder feedback during execution.

Past exam · Midterm 251 · V1 · Case Q2 (also V2) Q2

Given the evolving requirements (student usability, advisor requests, IT integration rules), which development approach (Agile, Waterfall, or Hybrid) would you recommend, and why would it be the best fit for this project?

Answer: Agile (Scrum) (0.5), because it handles the evolving requirements from students, advisors and IT, delivers iteratively so each sprint gets feedback, and integrates with DevOps pipelines for frequent tested releases (0.5). Justify it against the slide criteria: the requirements are reasonably clear but not stable, so waterfall's "clear, unambiguous, stable requirements" condition fails.

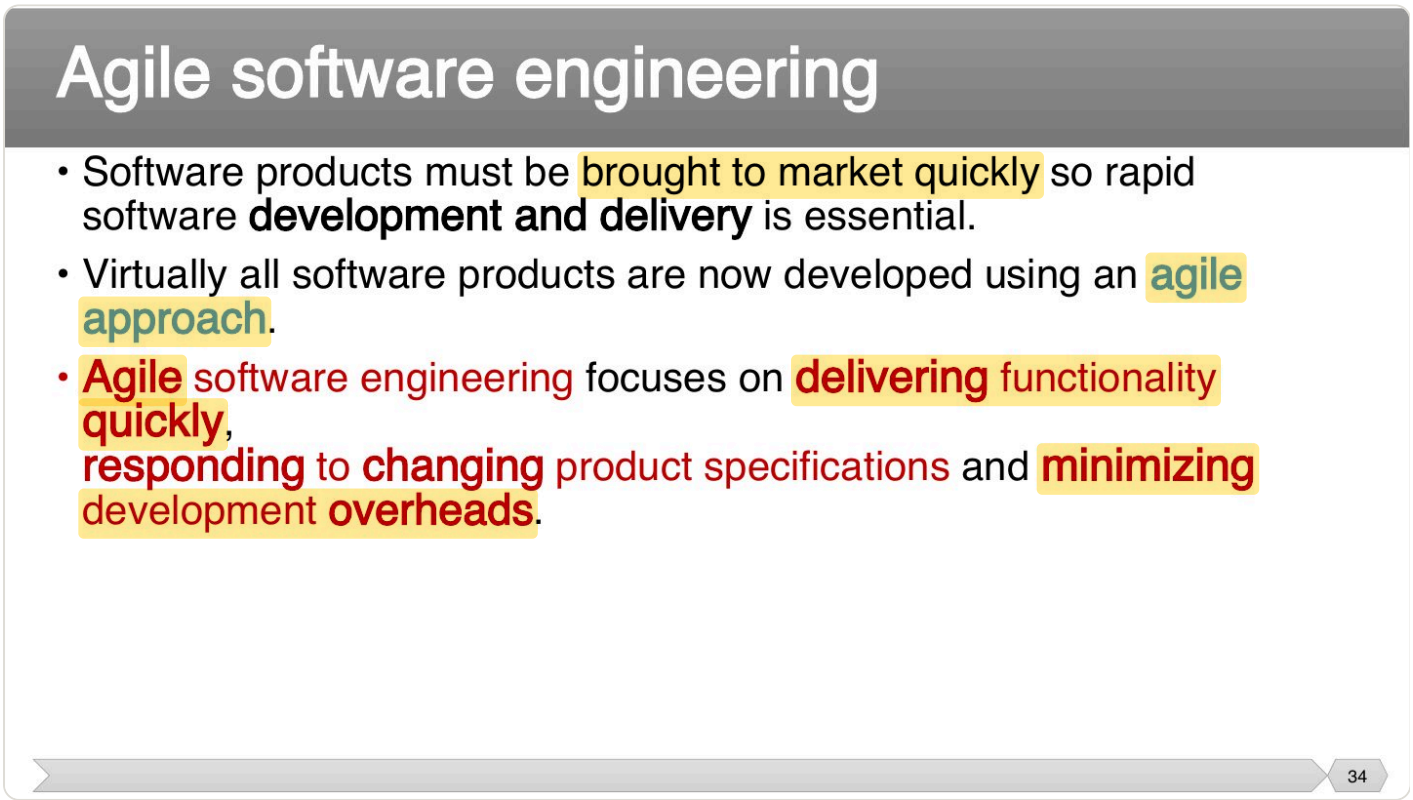
SLIDE 33 / 98 **Slide 33**



NOTES

1 Section divider: **Agile Project Management**.

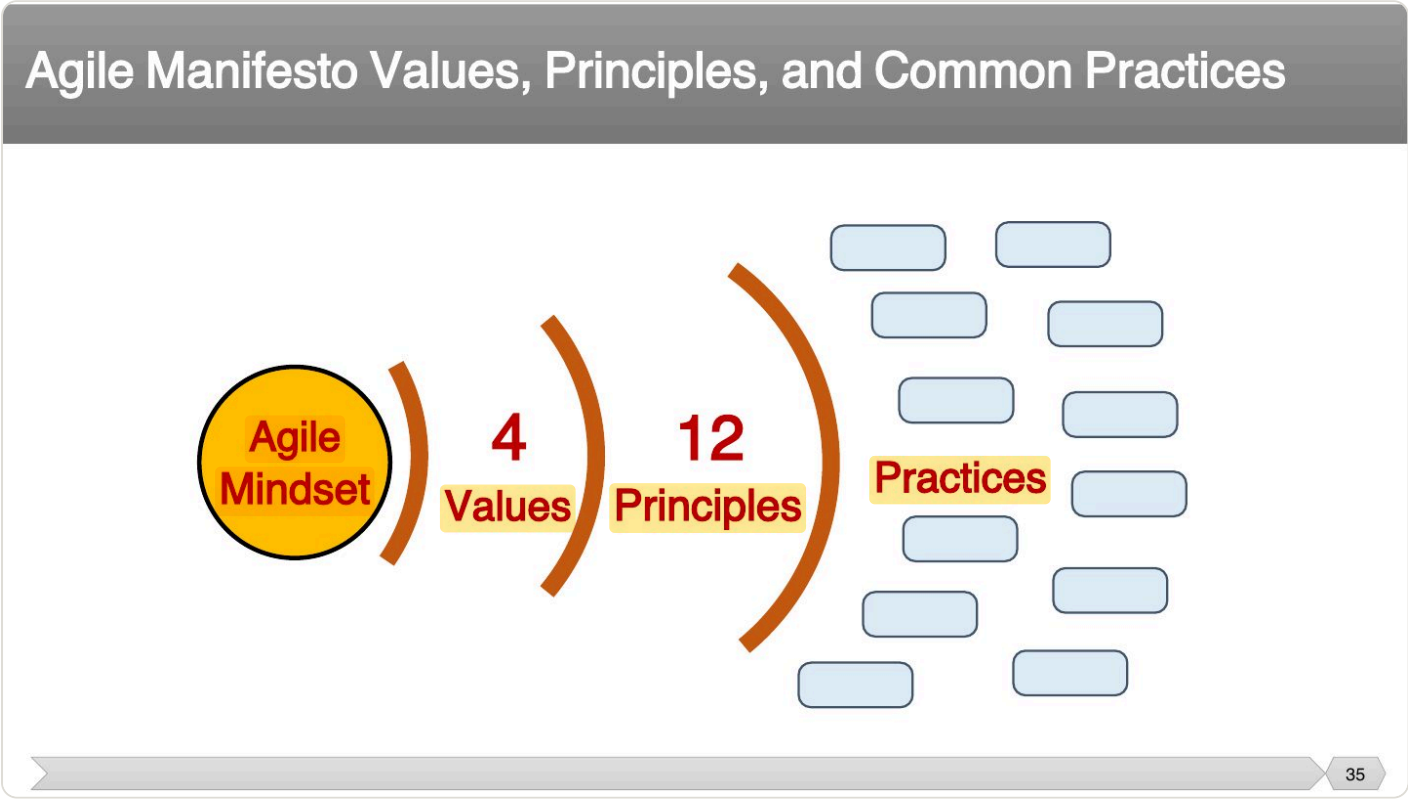
SLIDE 34 / 98 **Agile software engineering**



NOTES

1. Competitors may launch first, and a late product can miss its market. That's what makes speed so valuable, and why **quickly** comes up on this slide.
2. It's now the normal way to build most products. But "virtually all" is a broad claim: safety-critical systems may still favor heavier methods. Still, you should treat **agile** as the default for products.
3. Three goals with a common idea: avoid effort that doesn't reach the customer. Long documents that nobody reads are **overhead**, and they make change expensive.

SLIDE 35 / 98 Agile Manifesto Values, Principles, and Common Practices

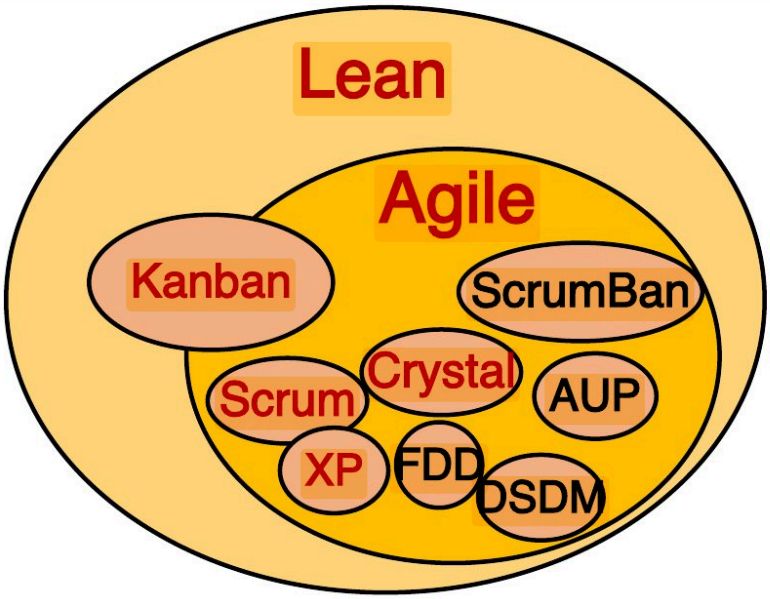


NOTES

1. Agile is first a way of thinking. You can hold daily stand-ups and still not be agile, if you won't change the plan. The **4 values** say what matters most...
2. ...and the **12 principles** turn those values into more specific guidance, such as welcoming late changes...
3. ...and expressed through many **practices** (stand-ups, user stories, TDD), which a team chooses to fit its needs.

SLIDE 36 / 98 **Lean**

Agile is a Blanket Term for Many Approaches



36

NOTES

1. **Agile** isn't a single method, so "we use agile" says little by itself. Scrum and Kanban both count, and they work quite differently.
2. **Lean** is broader still, and agile sits inside it.

SLIDE 37 / 98 **Agile Manifesto and Mindset**

Agile Manifesto and Mindset

Agile is a mindset defined by 4 values, guided by 12 principles, and manifested through many different practices.

Agile practitioners select practices based on their needs.

37

NOTES

1. Three layers, from general to specific. The **values** rarely change, while teams choose practices as they see fit. That's why two agile teams can look very different.
2. Practitioners **select practices** to fit their needs; there is no single "agile process".

EXAM ANGLE

Prediction Q1

Agile is best described as:

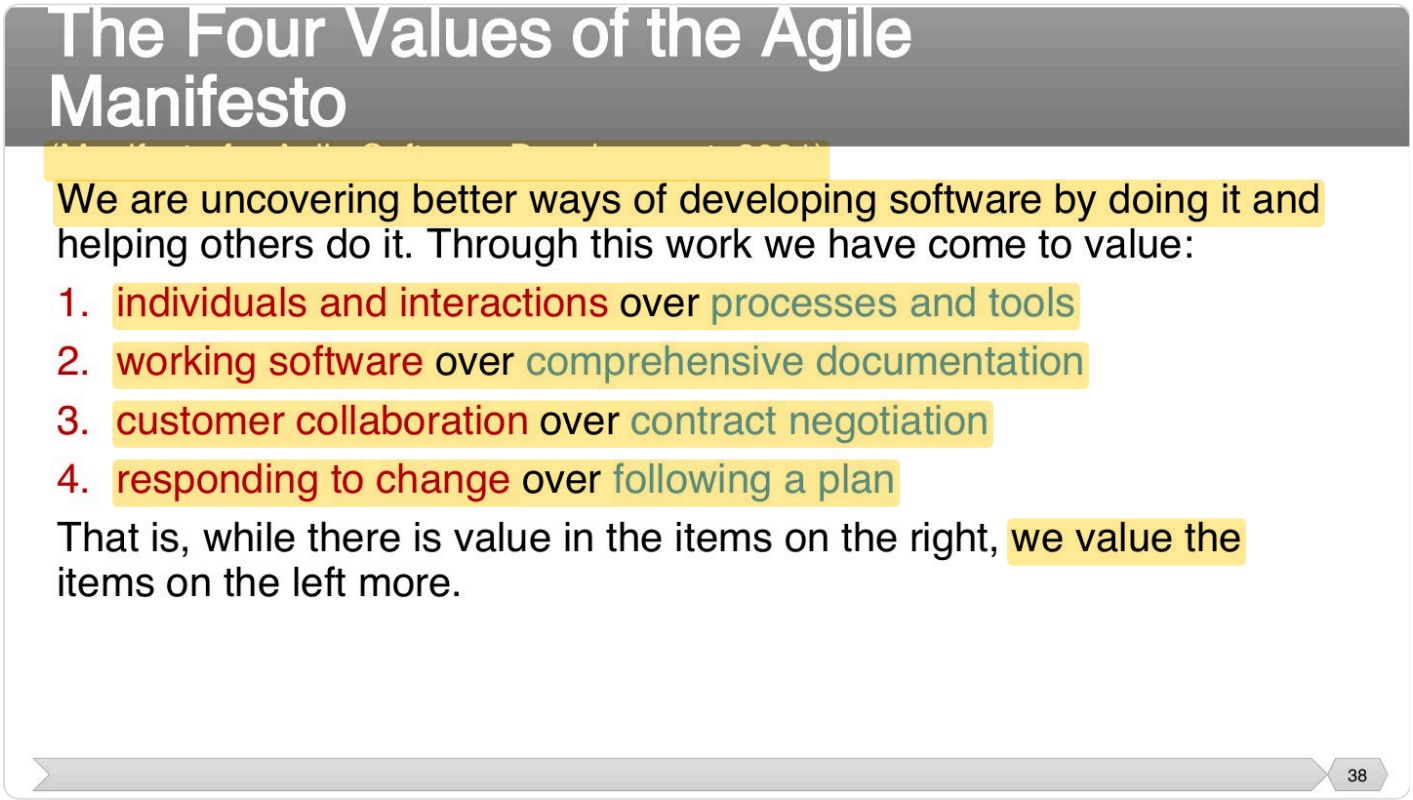
A A documentation standard

B A type of contract

C **A mindset defined by 4 values and 12 principles, expressed through many practices**

D A single fixed method

SLIDE 38 / 98 **Manifesto**



NOTES

1. A team that talks can solve most problems quickly, while one that only follows a process may not notice it's doing the wrong thing. So the value says prefer **individuals and interactions**.

2. A 200-page plan can't show whether anything works, but running software can. That's why **working software** is the real test of progress.
3. A contract can't foresee everything. If you treat the customer as a partner, you solve surprises together, rather than arguing over the wording, so **collaboration** comes first.
4. A plan is your best guess at the time. If the market changes, following it perfectly means delivering something no longer wanted, so **responding to change** beats following the plan.
5. The items on the right still have value; the **left is valued more**. A common trap: agile does not say "no documentation".

TIPS & TRICKS

- The four values, in order: **I W**ill **C**hange **R**eadily. Individuals and interactions, Working software, Customer collaboration, Responding to change.

EXAM ANGLE

Prediction Q1

Which is one of the four values of the Agile Manifesto?

A **Customer collaboration over contract negotiation**

B Following a plan over responding to change

C Processes and tools over individuals and interactions

D Comprehensive documentation over working software

Prediction Q2

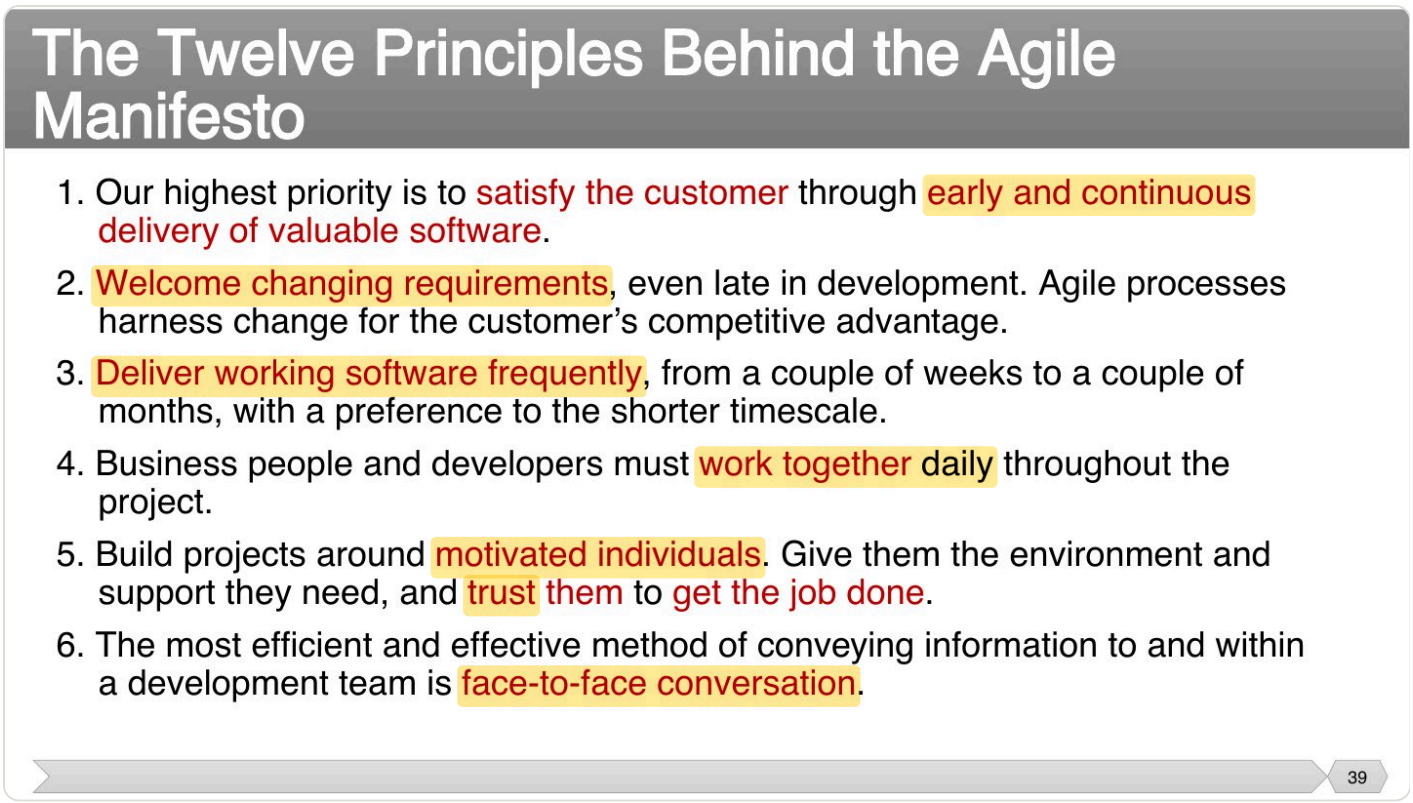
The Agile Manifesto says documentation has no value.

T True

F **False**

> There is value in the items on the right; the items on the left are valued more.

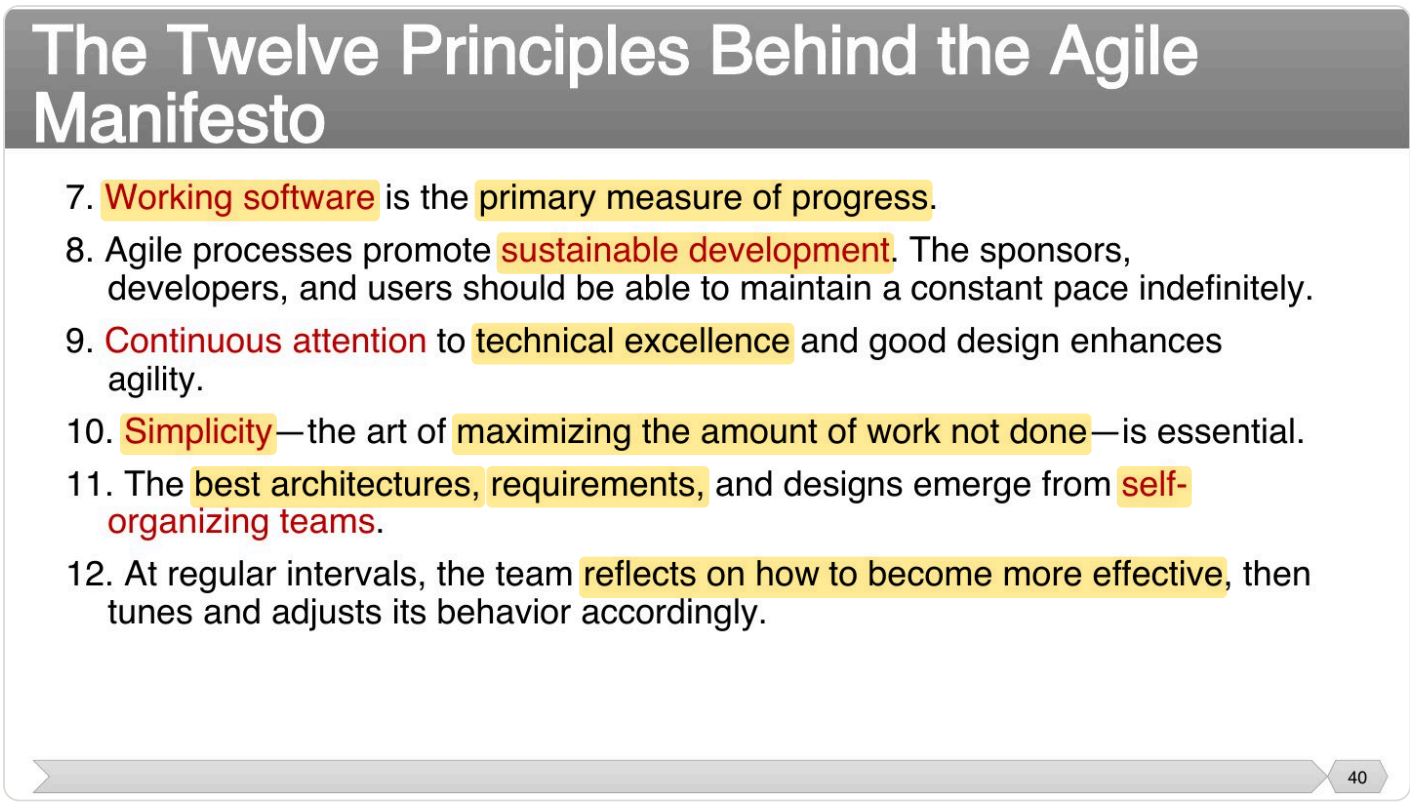
 Manifesto



NOTES

1. Early delivery gets value to users sooner and gets real feedback, not guesses. That's why **early and continuous delivery** is the top priority.
2. This is a big shift, as traditional projects treat late change as a failure. Agile sees it as a sign you've learned something, so it **welcomes changing requirements** if they make the customer more competitive.
3. The shorter the cycle, the sooner you find a mistake, and the less you've wasted on it. So **frequent delivery** reduces risk as well as adding value.
4. If developers only hear from the business occasionally, they have to guess. **Working together daily** replaces guesswork with a quick question.
5. You can't control people into caring. Hire good people, give them what they need, and **trust** them. Micromanaging skilled people wastes their ability.
6. A quick talk settles what could take days of emails, because you can ask and clarify on the spot. That's why **face-to-face** is named as the most effective.

 Manifesto



NOTES

1. Hours spent and documents written don't tell you whether it works. Only **working software** does, so "90% done" with nothing running means little.
2. Overtime can win a week, but tired people make more mistakes and leave. **Sustainable** means a pace you could keep up forever, without crunches.
3. Messy code gets slower to change over time, so skipping design makes you less agile later. **Technical excellence** is therefore what keeps change cheap.
4. The best feature might be one you never had to build. **Simplicity** means asking "do we really need this?", because work not done has no bugs and no maintenance.
5. People doing the work see problems first and care more about what they designed themselves. So **self-organizing teams** tend to produce better designs than plans handed down.
6. A process that's never reviewed never improves. The team stops regularly to **reflect** on how it's working, and agrees on something to change. That's a retrospective.

EXAM ANGLE

Prediction Q1

According to the agile principles, the primary measure of progress is:

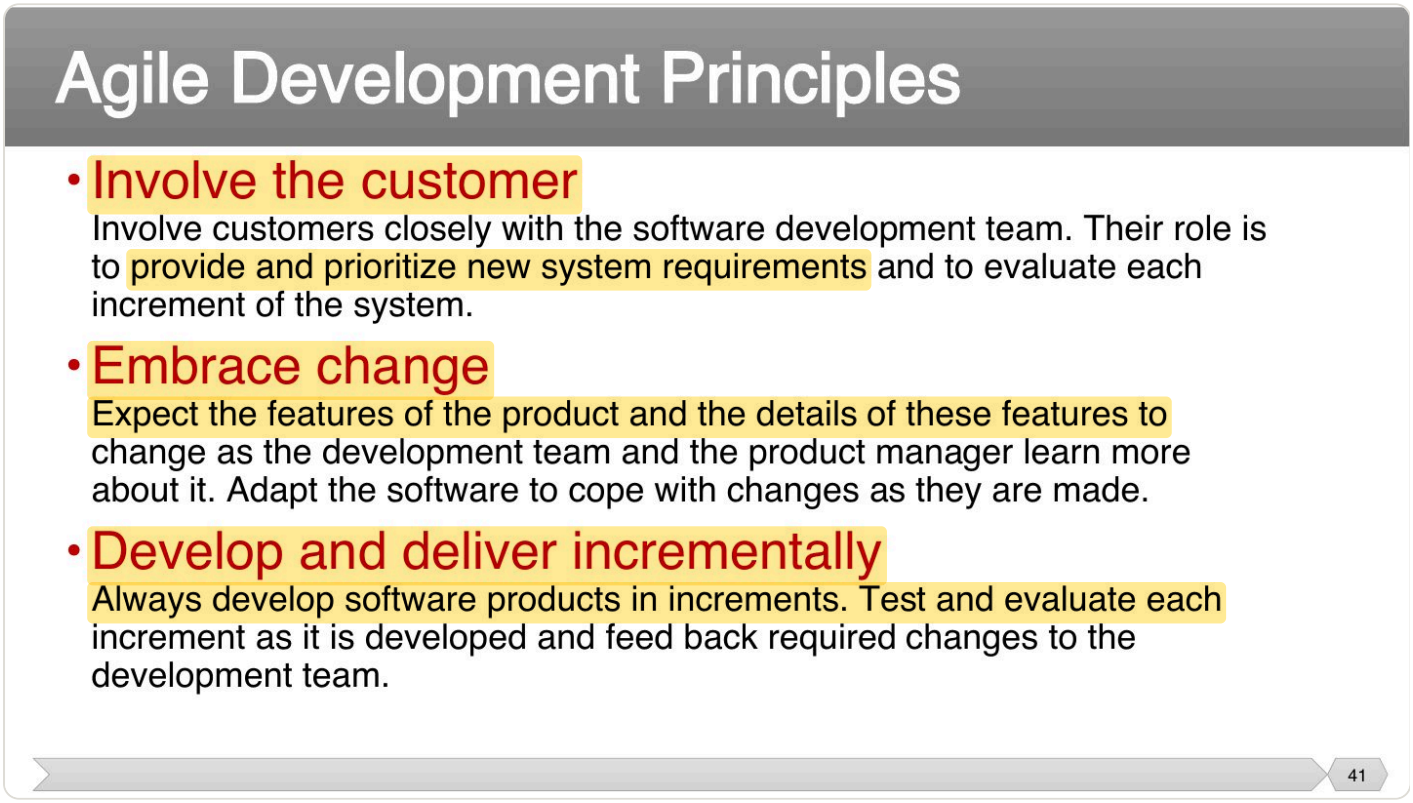
A Hours worked

B **Working software**

C Documents completed

D Lines of code

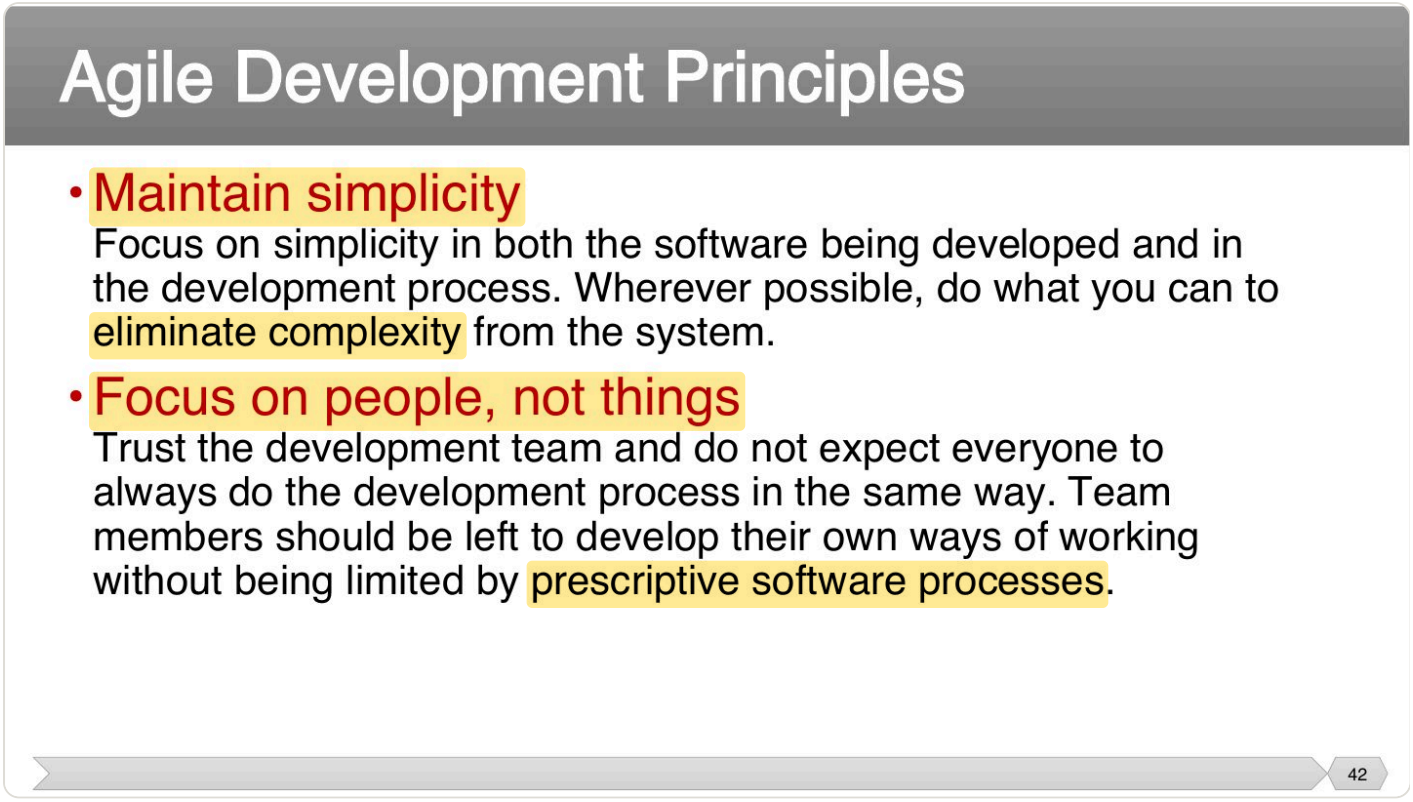
SLIDE 41 / 98 **Agile Development Principles**



NOTES

1. Nobody knows what customers want better than customers do. Having them **prioritize** and judge each increment prevents you building the wrong thing for months.
2. Don’t treat change as a failure of planning. You’ll learn more about what’s needed as you go, so **embrace change** and build software that’s easy to adjust.
3. Each piece is a chance to check you’re right. If you only **deliver incrementally**, a wrong turn costs one increment rather than the whole project.

 Agile Development Principles



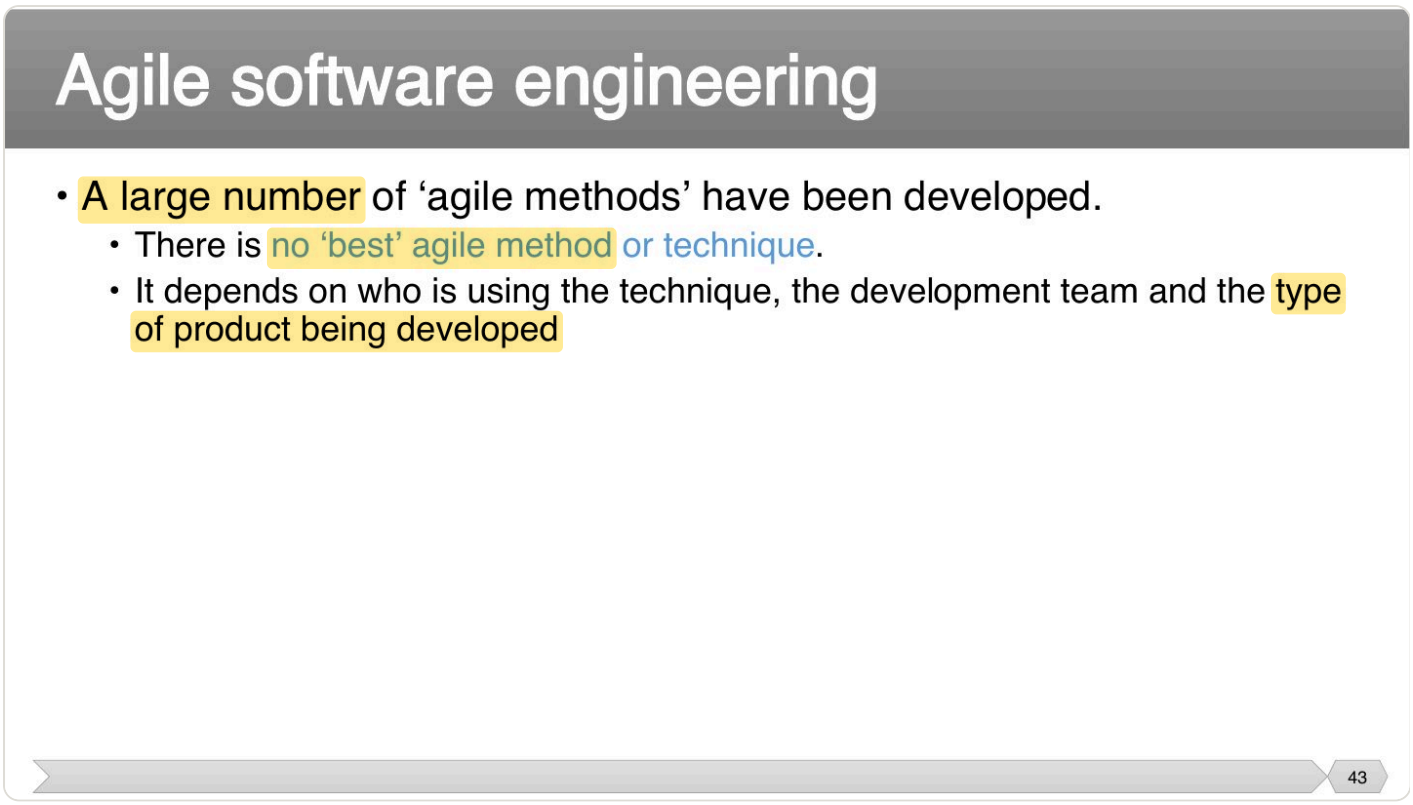
NOTES

1. Complexity is what makes software hard to change later, and extra process does the same. Ask “can this be simpler?” of the code and of how you work: **maintain simplicity**.
2. People differ, and forcing everybody into one way of working wastes their strengths. **Trust the team** to find what works for them, rather than imposing a rigid process.

TIPS & TRICKS

- Slides 41 and 42 together, **MIdFiEID**: Maintain simplicity, Involve the customer, Focus on people, Embrace change, Develop incrementally.

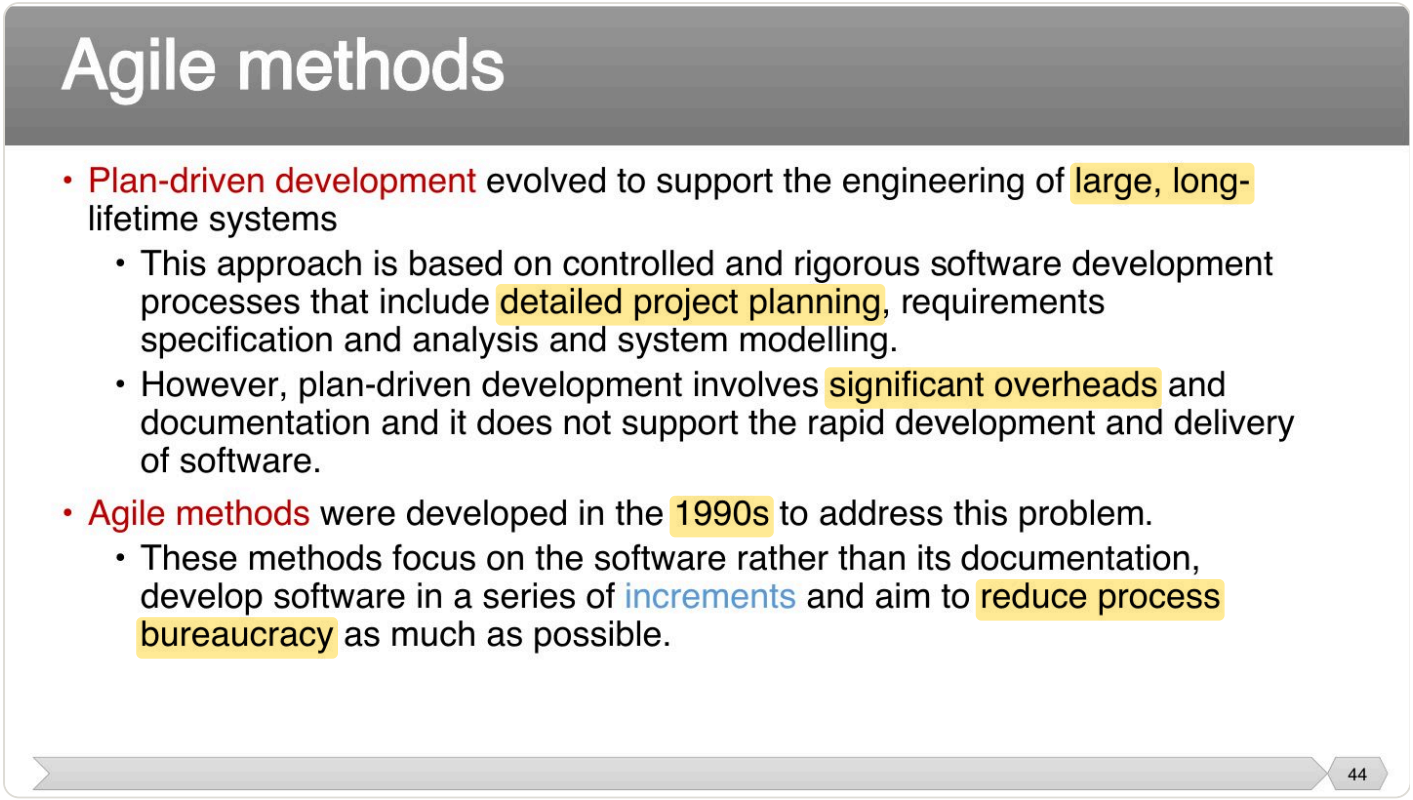
 Agile software engineering



NOTES

1. Don’t be overwhelmed by the names, since they share the same core ideas. The **many** methods mostly differ in emphasis...
2. ...and there is **no single best one**...
3. A small startup and a bank may need different things. So asking “what’s the best agile method?” is the wrong question, and you should ask what suits this **type of product** and team.

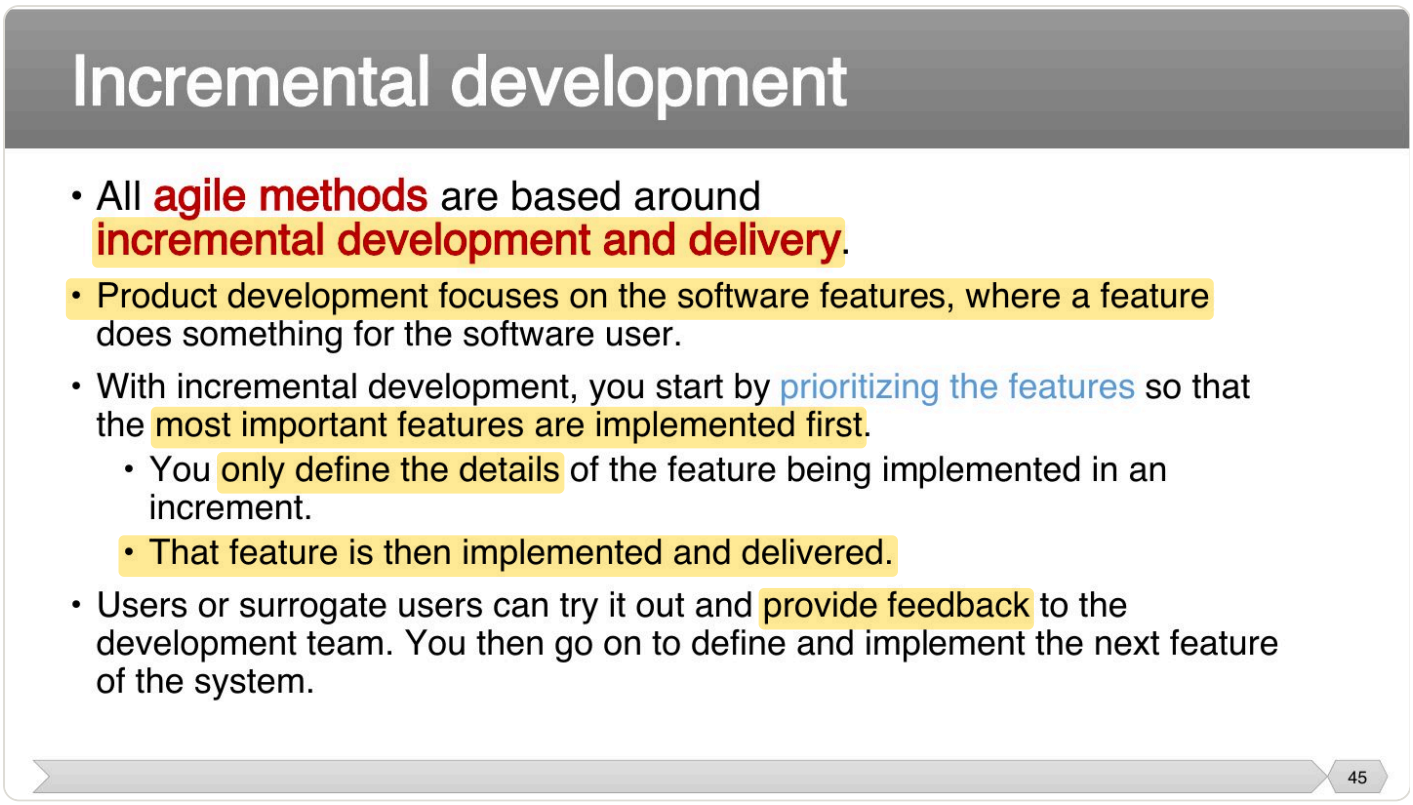
 Agile methods



NOTES

1. It isn't a bad approach, just built for a different problem. A system that must run for decades, built by hundreds, needs careful control, which is why it suits **large, long-lifetime** systems...
2. ...and that's what all the rigor is for. **Detailed planning** and system models let very large groups co-ordinate without constantly talking...
3. ...but for a small product in a fast market, all that **overhead** means you may arrive too late, as requirements can change before you've finished the documents.
4. Agile methods appeared in the **1990s** to fix this.
5. Time saved on paperwork goes into building. That's the trade: less **bureaucracy** and documentation, which is fine for a small team that can just talk to each other.

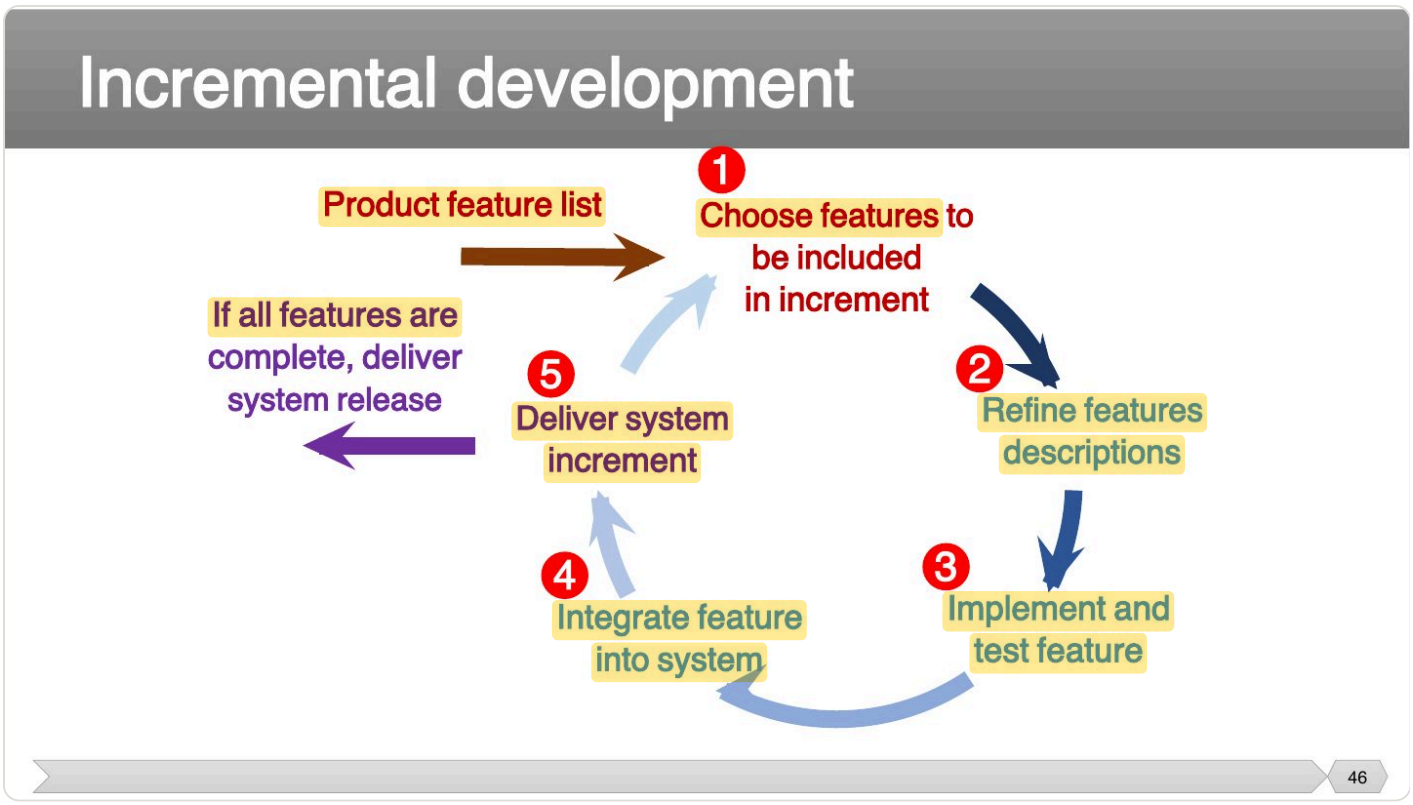
 Incremental development



NOTES

1. A feature is defined by what it does for the user, not how it's built. Organizing around features with **incremental delivery** keeps everybody focused on visible value.
2. If you run out of time or money, you want the most useful things already done. Doing **the most important first** means you lose only the least valuable work.
3. Only define the **details of the feature being built in this increment**; the rest stays rough until it is needed.
4. Real use reveals what discussions can't. Letting users **give feedback** after each feature means the next piece of work benefits from what you've just learned.

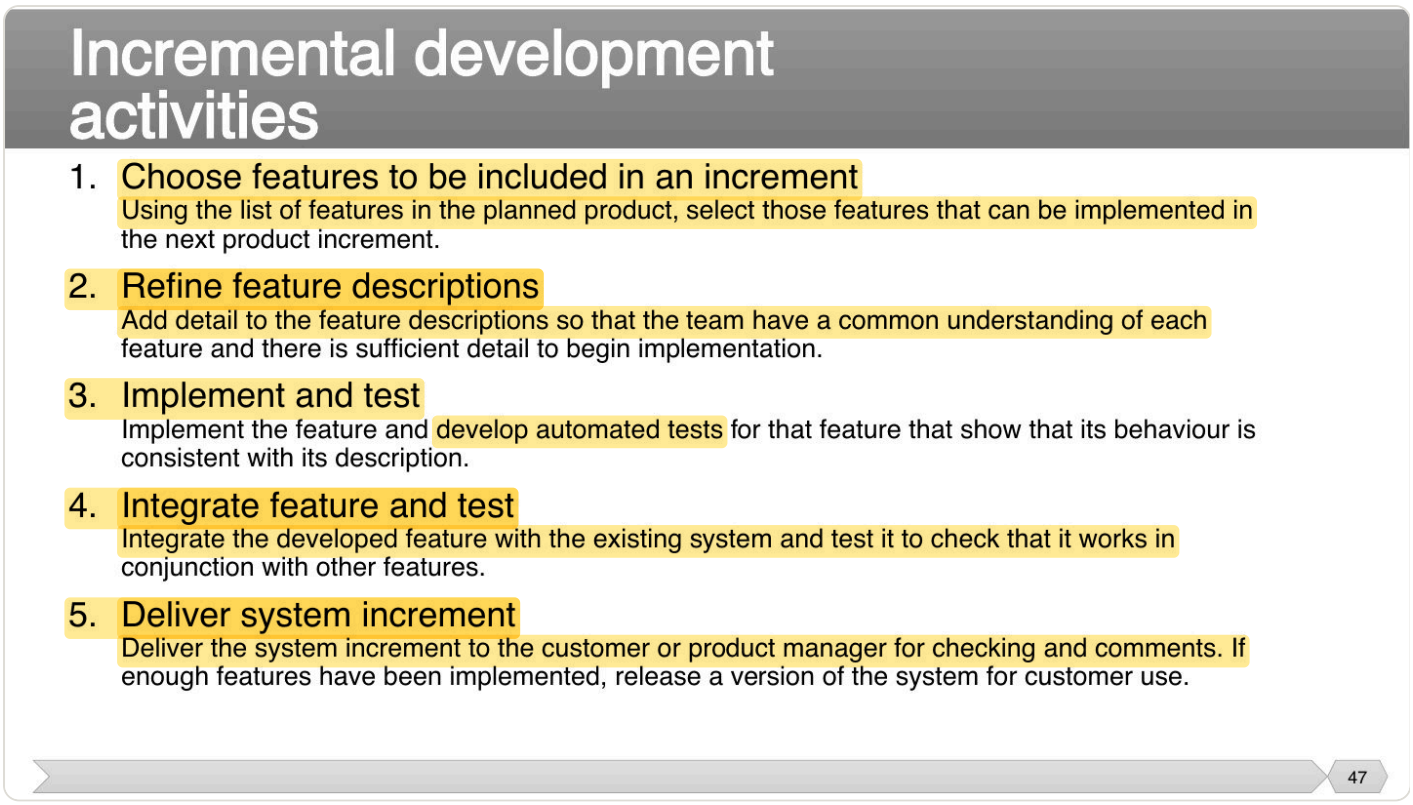
 Incremental development



NOTES

1. Start from the full list but pull in only what fits this round, so the team isn't **choosing** everything at once...
2. ...only now do you work out the details, since by this time you know most about each feature, so you **refine** only what's going to be built...
3. ...and build and check it together. If you **implement and test** at the same time, bugs get found while the code's still fresh...
4. ...then you check it works with everything else. **Integrating** each feature right away prevents the big painful merge at the end...
5. ...and put it in front of people. Each **delivery** is another chance to learn. Repeat until the list is empty.

 activities



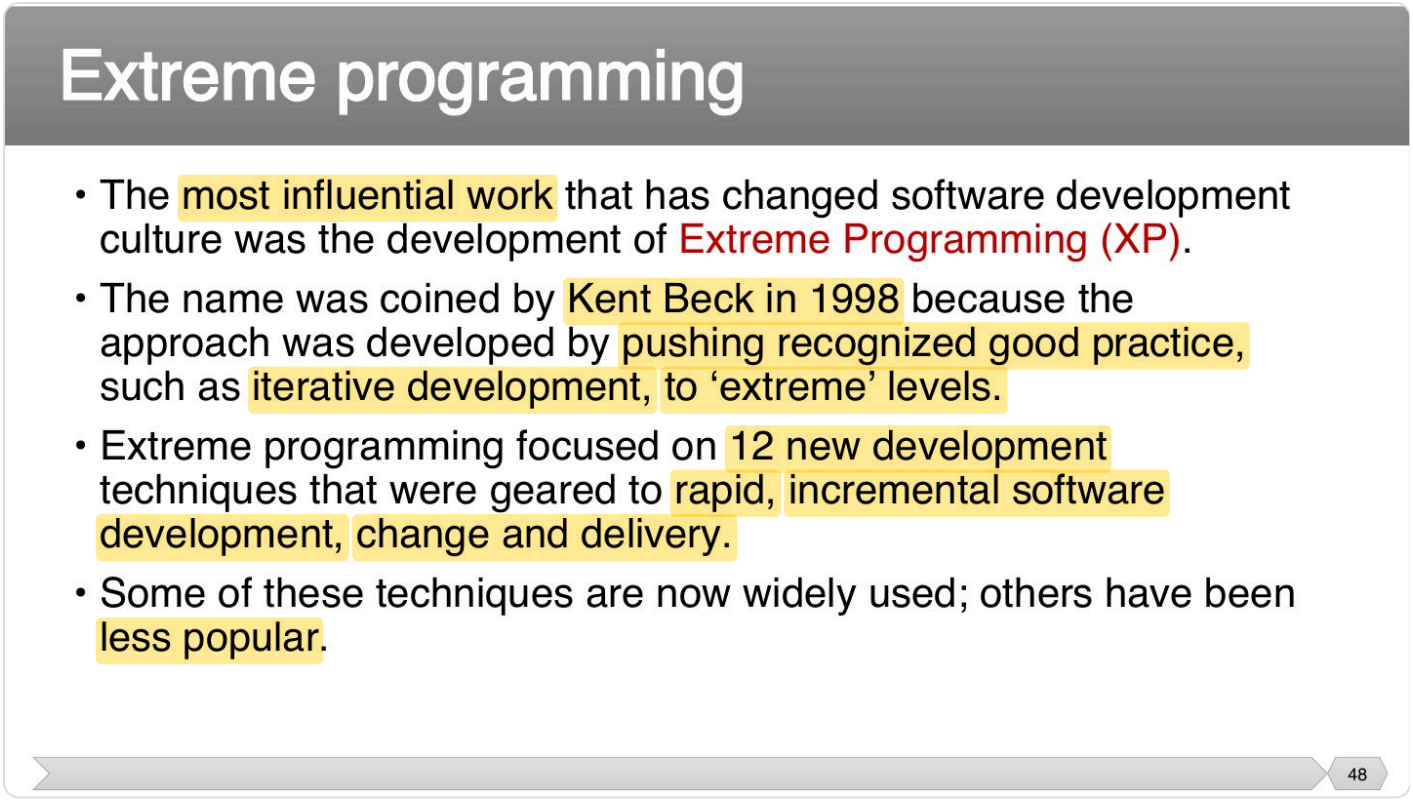
NOTES

1. Pick what can realistically be finished next. **Choosing** a small amount keeps each cycle short and its results visible.
2. Just enough detail for everyone to understand it the same way. **Refining** too early wastes effort if the feature changes, and too late means confusion.
3. Automated checks mean anyone can confirm that the feature still works, any time. **Automated tests** also stop later changes silently breaking it.
4. A feature can work alone and still fail with others. **Integrate and test** checks they work together, as the problems often live between the parts.
5. Someone has to judge it. Giving it to the customer or product manager brings comments that shape the next increment. A public release can wait until enough has been **delivered**.

TIPS & TRICKS

- The order matters here: **C**ooks **R**oast **I**n **I**ron **D**ishes. Choose features, Refine them, Implement and test, Integrate and test, Deliver.

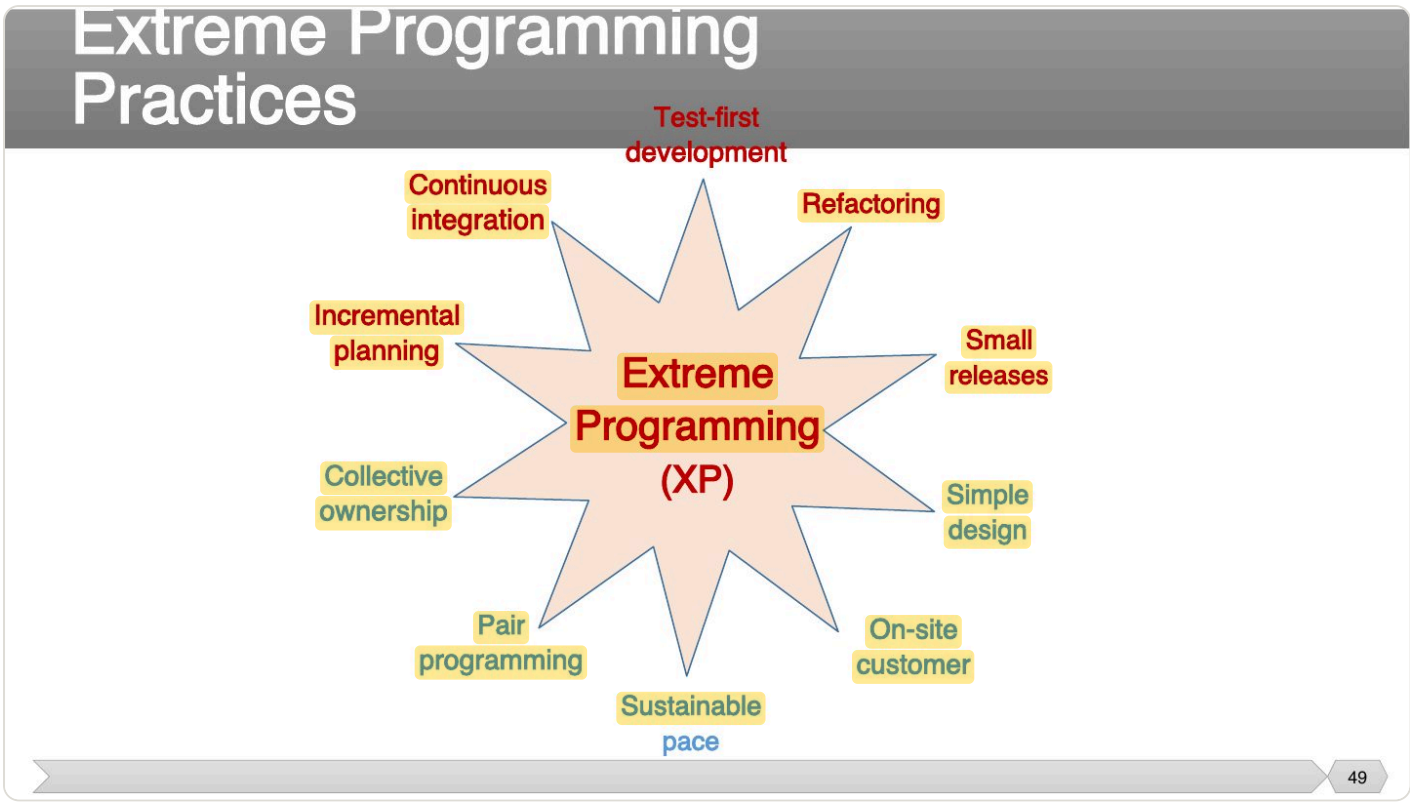
 Extreme programming



NOTES

1. Many everyday practices came from here, so it's why it's called **most influential**, even for teams who've never used XP as a whole.
2. If code review is good, why not review constantly? That's pair programming. If testing is good, why not test all the time? That's the **extreme** idea: take what works and push it further.
3. You don't need to memorize all **12**, and the next slides cover the ones in wide use. Together, they're built to make frequent change safe...
4. ...and teams picked what suited them. Pair programming all day is **less popular**, as it's tiring and costly, while tests and continuous integration became the norm.

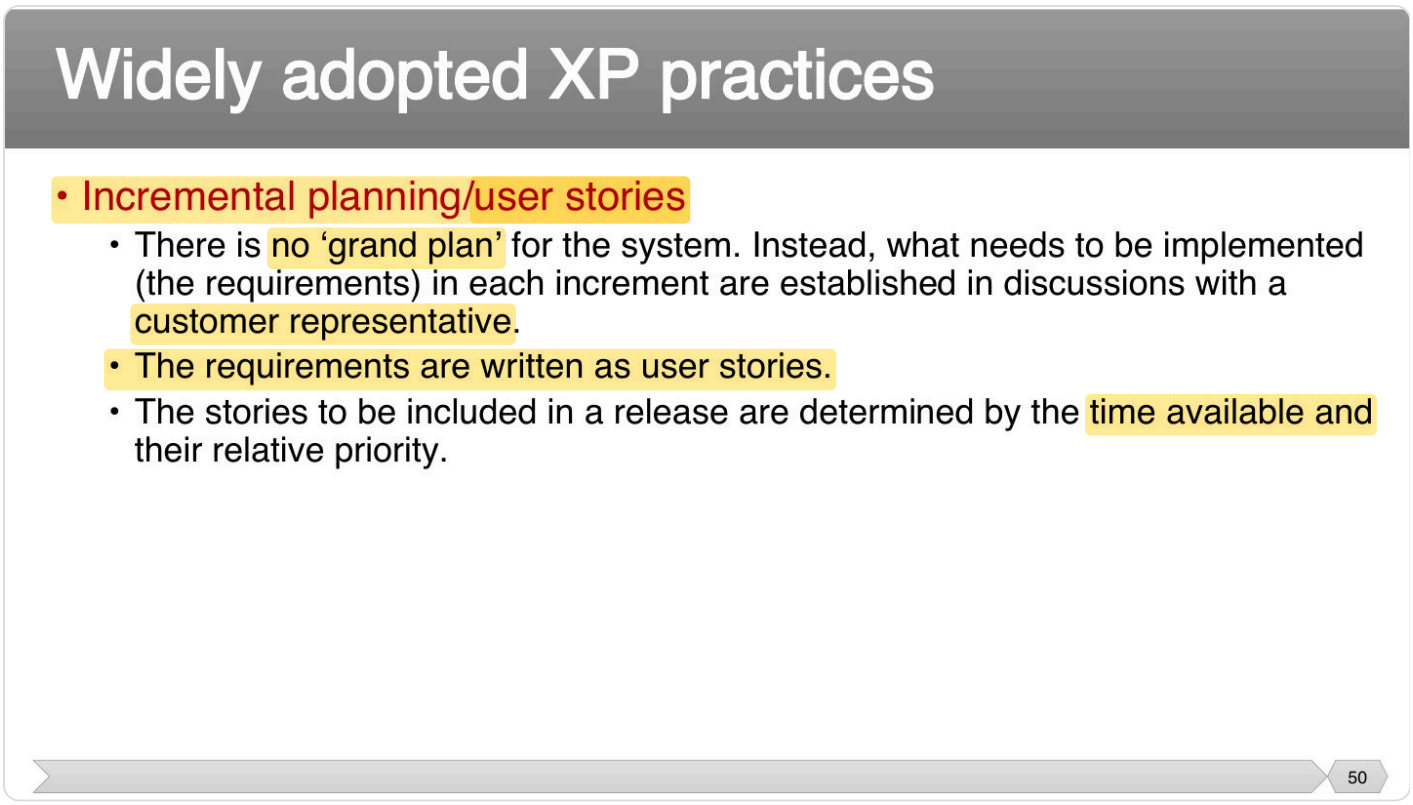
 Practices



NOTES

1. These all make frequent change safe: tests catch mistakes, **refactoring** keeps code clean, and small releases limit the damage if something goes wrong.
2. These tackle knowledge. With **pair programming** and shared ownership, no one person is the only one who understands a module, and an **on-site customer** gives instant answers.
3. **Sustainable pace**: no long overtime. The next slides cover the widely adopted ones.

 Widely adopted XP practices



NOTES

1. A big plan would be out of date before you got there. Deciding one increment at a time with a **customer representative** keeps the work tied to what’s needed now.
2. A **user story** states who wants something and why, not how to build it. The “so that” matters, as it tells you whether a simpler solution would do.
3. You can’t do everything by the deadline, so the two limits decide: what’s most valuable, and what fits in the **time available**. That’s how scope stays flexible.

TIPS & TRICKS

- Slides 50 to 54, **STRICt**: Small releases, Test-driven development, Refactoring, Incremental planning, Continuous integration.

Widely adopted XP practices

- Small releases
  - The minimal useful set of functionality that provides business value is developed first.
  - Releases of the system are frequent and incrementally add functionality to the previous release.

NOTES

1. Ask “what’s the smallest thing that’s useful?”, and build that first. A **minimal useful set** starts earning value and feedback much sooner than a full system.
2. Each release is small, so it’s low risk, and it’s quick to fix if wrong. **Frequent** releases also keep customers seeing progress.

Widely adopted XP practices

- Test-driven development
  - Instead of writing code then tests for that code, developers write the tests first.
  - This helps clarify what the code should actually do and that there is always a ‘tested’ version of the code available.
  - An automated unit test framework is used to run the tests after every change.
  - New code should not ‘break’ code that has already been implemented.

NOTES

1. It sounds backwards, yet writing the test first forces you to define “working” before you start. That’s the idea behind **test-driven development**.
2. If you can’t write a test for it, you may not understand what it should do yet, so the test **clarifies** the job. You’ll also always have a version known to pass.
3. Nobody has time to manually re-test everything after every change. **Automated** runs make that free, and that’s what makes frequent change safe…
4. …so if a change breaks an old feature, you find out within minutes, while it’s easy to see what caused it, rather than weeks later.

EXAM ANGLE

Prediction Q1

In test-driven development, developers:

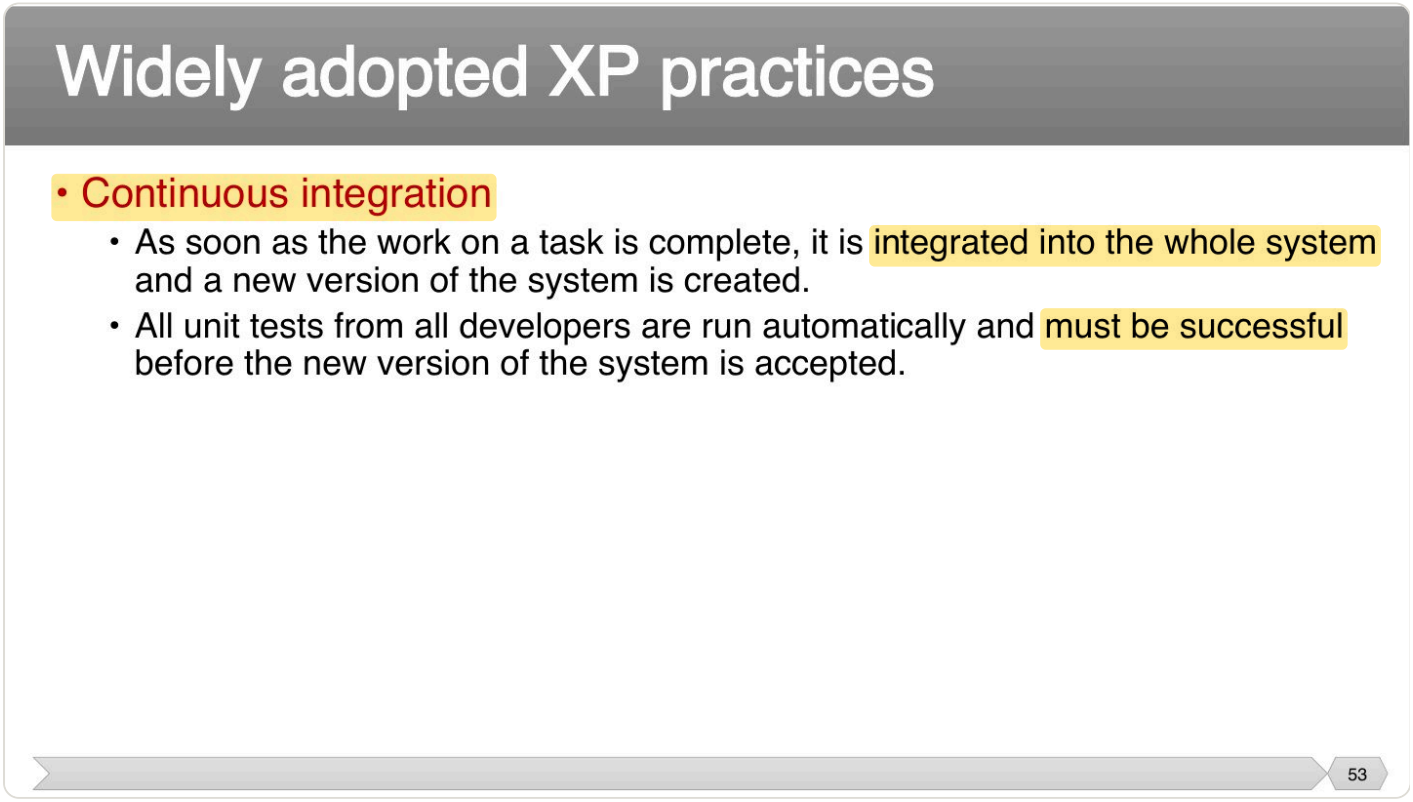
A Only test manually

B Write tests after release

C **Write tests before writing the code**

D Skip unit tests

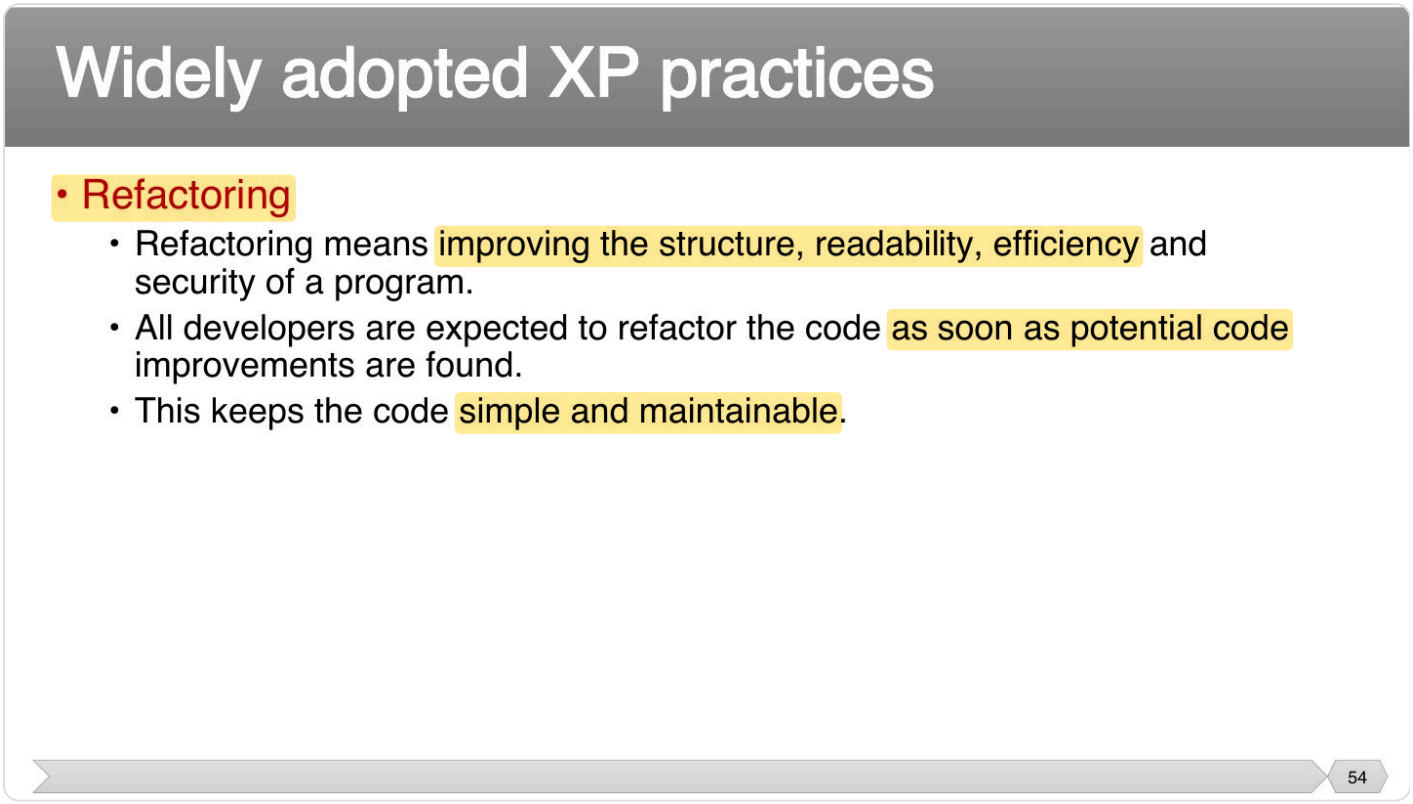
SLIDE 53 / 98 **Widely adopted XP practices**



NOTES

1. The longer code stays separate, the more painful the merge. **Integrating** each finished task right away keeps the differences tiny, and everyone works from the current system.
2. All unit tests from all developers run automatically and **must pass** before the new version is accepted, so integration problems appear in minutes instead of at the end.

SLIDE 54 / 98 Widely adopted XP practices



NOTES

1. Think of tidying a kitchen while the same meals still come out. Users see no difference, but **improving the structure** makes the next change easier and safer. Tests prove nothing broke.
2. Waiting for a “cleanup phase” means it never happens. Fixing small problems **as soon as** you see them stops them from building up, and this only works thanks to automated tests...
3. ...the payoff comes later. **Maintainable** code stays cheap to change, which is what lets a team keep moving quickly over the years.

SLIDE 55 / 98 **Widely adopted XP practices**

Widely adopted XP practices

- Incremental planning/user stories
  - There is no ‘grand plan’ for the system. Instead, what needs to be implemented (the requirements) in each increment are established in discussions with a customer representative.
  - The requirements are written as user stories.
  - The stories to be included in a release are determined by the time available and their relative priority.

NOTES

1 The same material appears here again: **incremental planning** with **user stories**. If it’s on two slides, it’s likely worth remembering.

SLIDE 56 / 98 **Scrum**

Scrum

- Software company managers need information that will help them understand how much it costs to develop a software product, how long it will take and when the product can be brought to market.
- Plan-driven development provides this information through long-term development plans that identify deliverables - items the team will deliver and when these will be delivered.
- Plans always change so anything apart from short-term plans are unreliable.
- Scrum is an agile method that provides a framework for agile project organization and planning. It does not mandate any specific technical practices.

NOTES

1. A business can't fund something without knowing roughly what it costs and when it arrives. Scrum doesn't remove this need for **how much**...
2. Traditional planning's answer sounds reassuring: a full schedule with dates. But a **long-term plan** gives confidence, not necessarily accuracy...
3. The further ahead a plan looks, the more it's just guessing. So when **plans always change**, trusting a detailed plan for next year is a mistake. Short-term plans are the ones you can rely on.
4. Scrum is about how a team organizes and plans, not how it writes code. So it **mandates no technical practices**, which is why it's often combined with XP. A common trap.

EXAM ANGLE

Prediction Q1

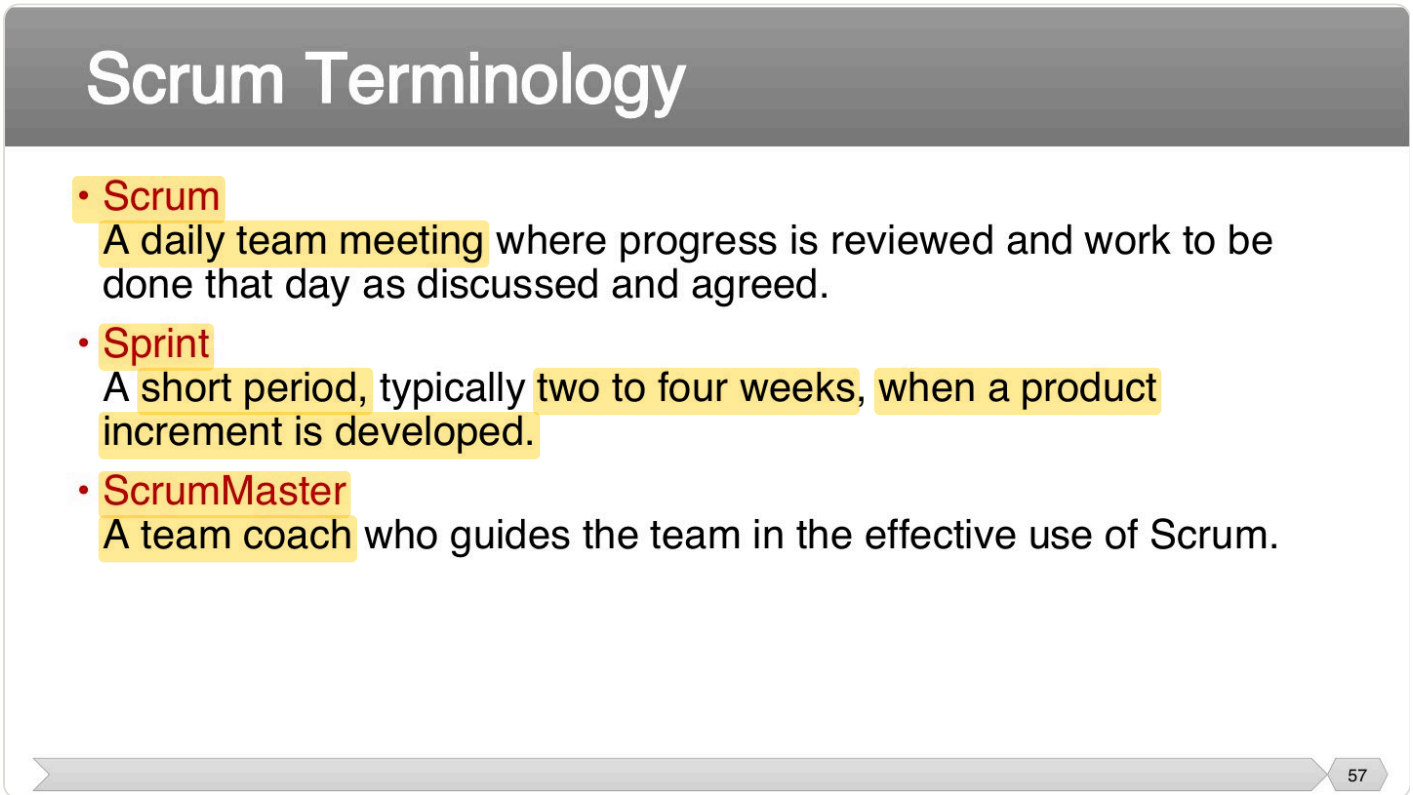
Scrum mandates specific technical practices such as pair programming.

T True

F False

> Scrum provides a framework for organization and planning only; XP defines the engineering practices.

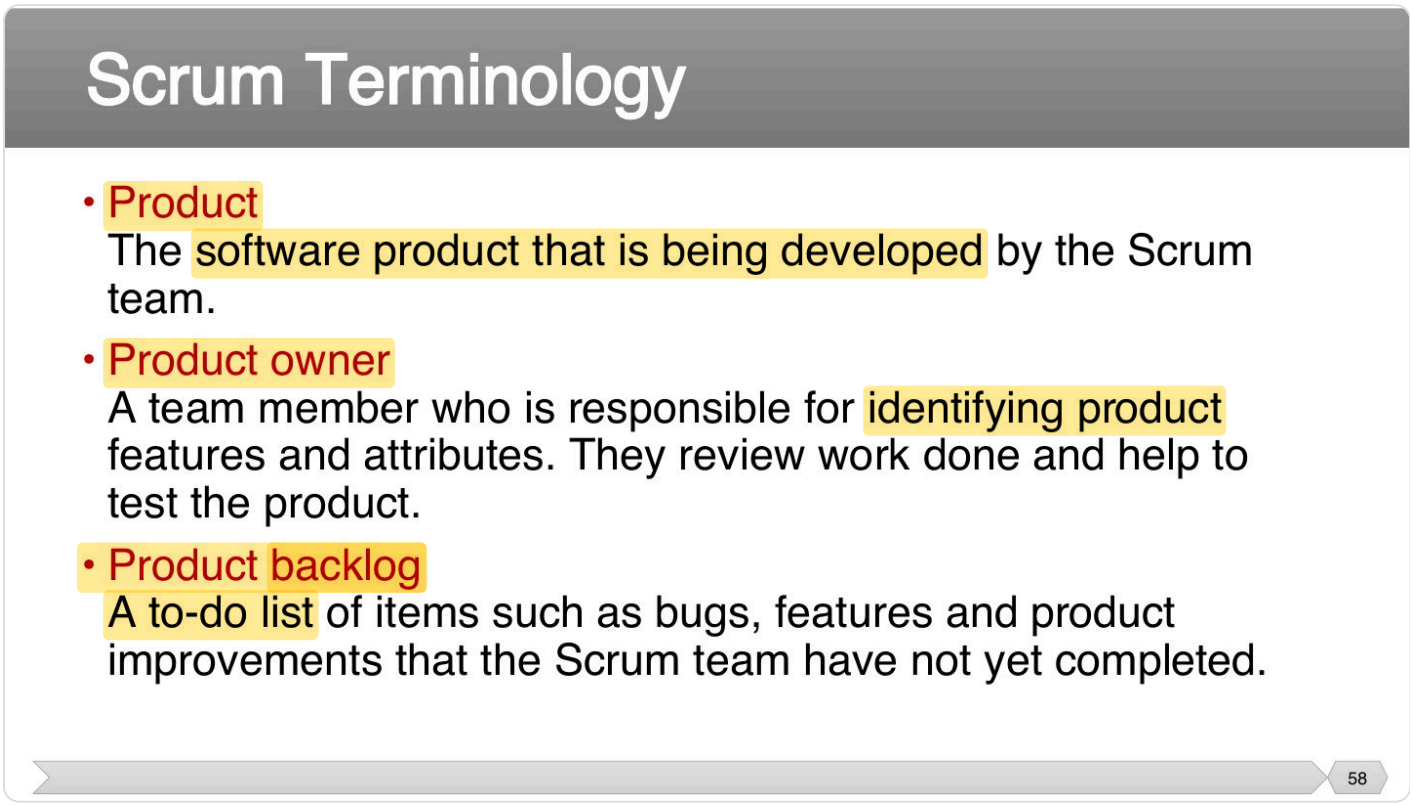
SLIDE 57 / 98 **Scrum Terminology**



NOTES

1. It keeps everyone in sync without long meetings. If someone's stuck, it's out in the open within a day. Careful: in this slide "**Scrum**" means the meeting, as well as the method's name.
2. A short, fixed period forces real progress and limits how long you can go wrong. **Sprint** length usually stays the same, so the team gets a steady rhythm.
3. The **ScrumMaster** is not a boss. They help the team use the method well and remove obstacles, so don't assume it's just a new name for project manager.

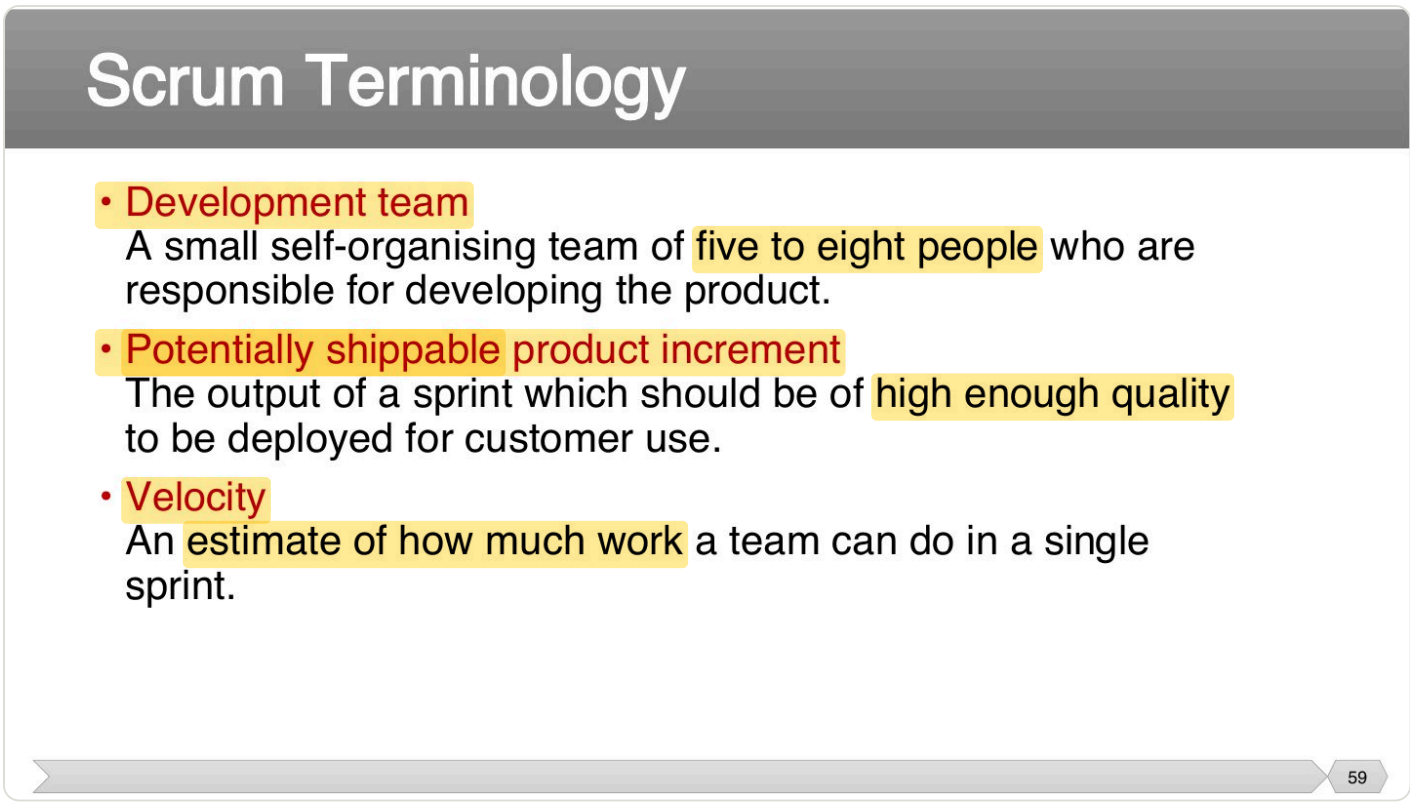
SLIDE 58 / 98 Scrum Terminology



NOTES

1. In Scrum, you focus on one **product** rather than a project with an end date. This keeps attention on what's being built over its whole life.
2. Someone has to decide what matters most. Having one **product owner** means developers get clear priorities, instead of conflicting requests from many people.
3. Everything that might get done lives in one place, so nothing is lost and it can be ordered. The **backlog** is the only source of work for the team.

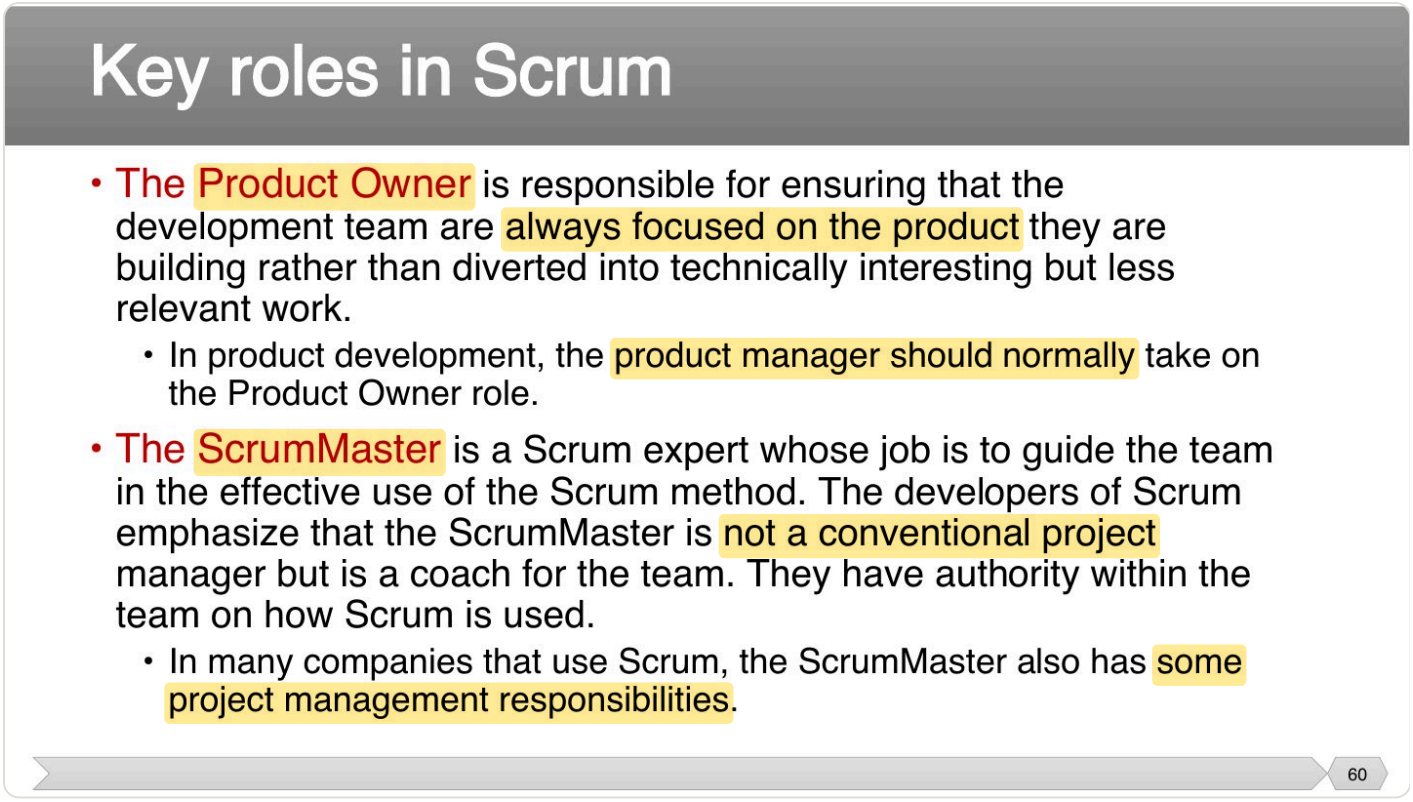
 Scrum Terminology



NOTES

1. Small enough that everybody can talk directly, and large enough for a mix of skills. **Five to eight** keeps communication paths manageable.
2. This sets the bar: “mostly finished” doesn’t count. A **potentially shippable** result means there’s no hidden work left, so you could release it if you chose to.
3. **Velocity**: an **estimate of how much work** the team can do in one sprint, used to plan the next sprints.

 Key roles in Scrum



NOTES

1. Engineers love interesting technical problems, which isn't always what customers need. The **Product Owner** keeps asking "does this help the product?" so the work stays useful.
2. Who's best placed to do this? Someone who already understands the market and the customers. So the **product manager** normally takes this role.
3. This trips people up: the **ScrumMaster** doesn't assign tasks or manage people. They're responsible for how the method is used, and they're there to coach.
4. Theory and practice differ. Someone still has to handle budgets and reports, so in many firms this person also takes on **project management** duties. Slide 70 lists these.

TIPS & TRICKS

- **PoST**: Product owner, ScrumMaster, Team.

EXAM ANGLE

Prediction Q1

Who keeps the team focused on the product instead of technically interesting but less relevant work?

A Sponsor

B Tester

C ScrumMaster

D **Product Owner**

Prediction Q2

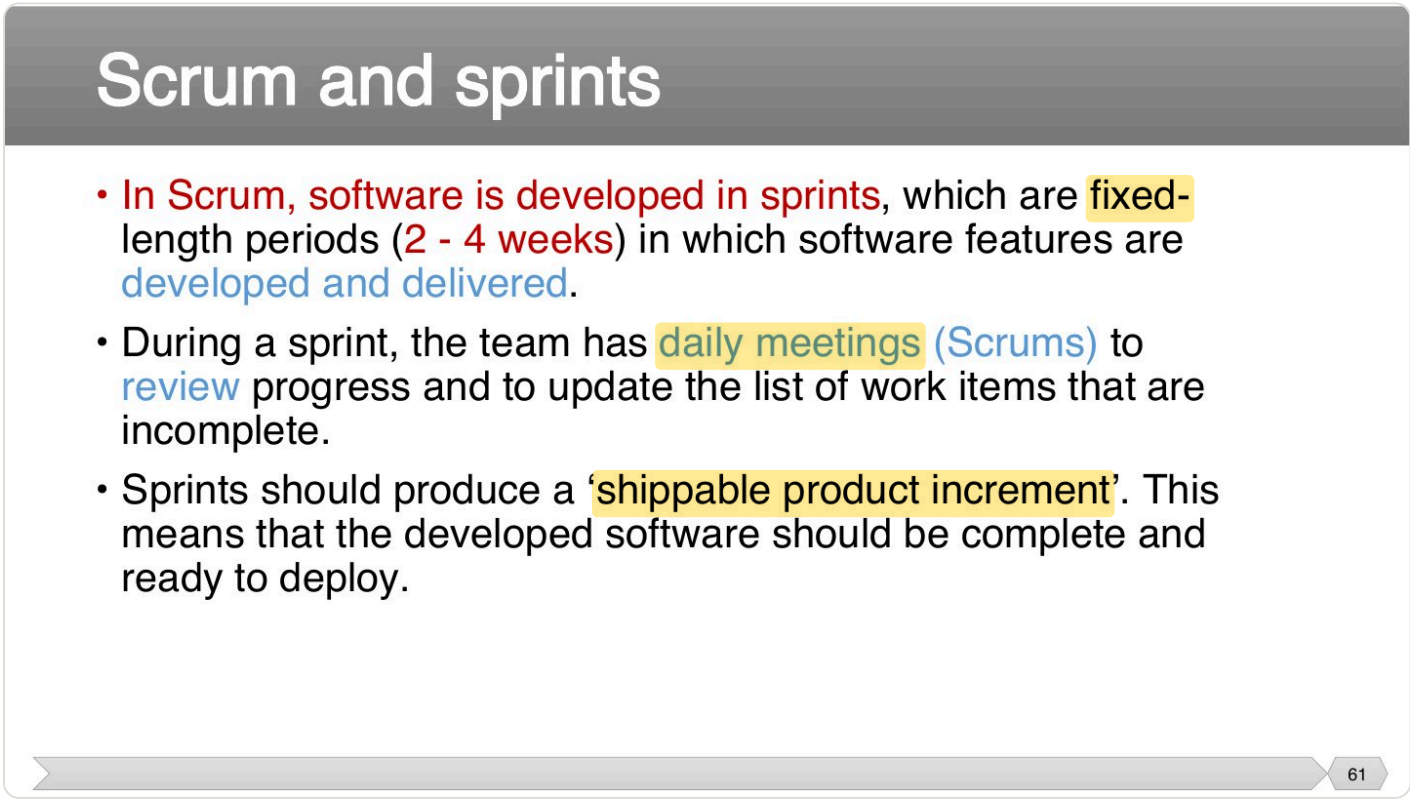
The ScrumMaster is a conventional project manager.

T True

F **False**

> The ScrumMaster is a coach for the team.

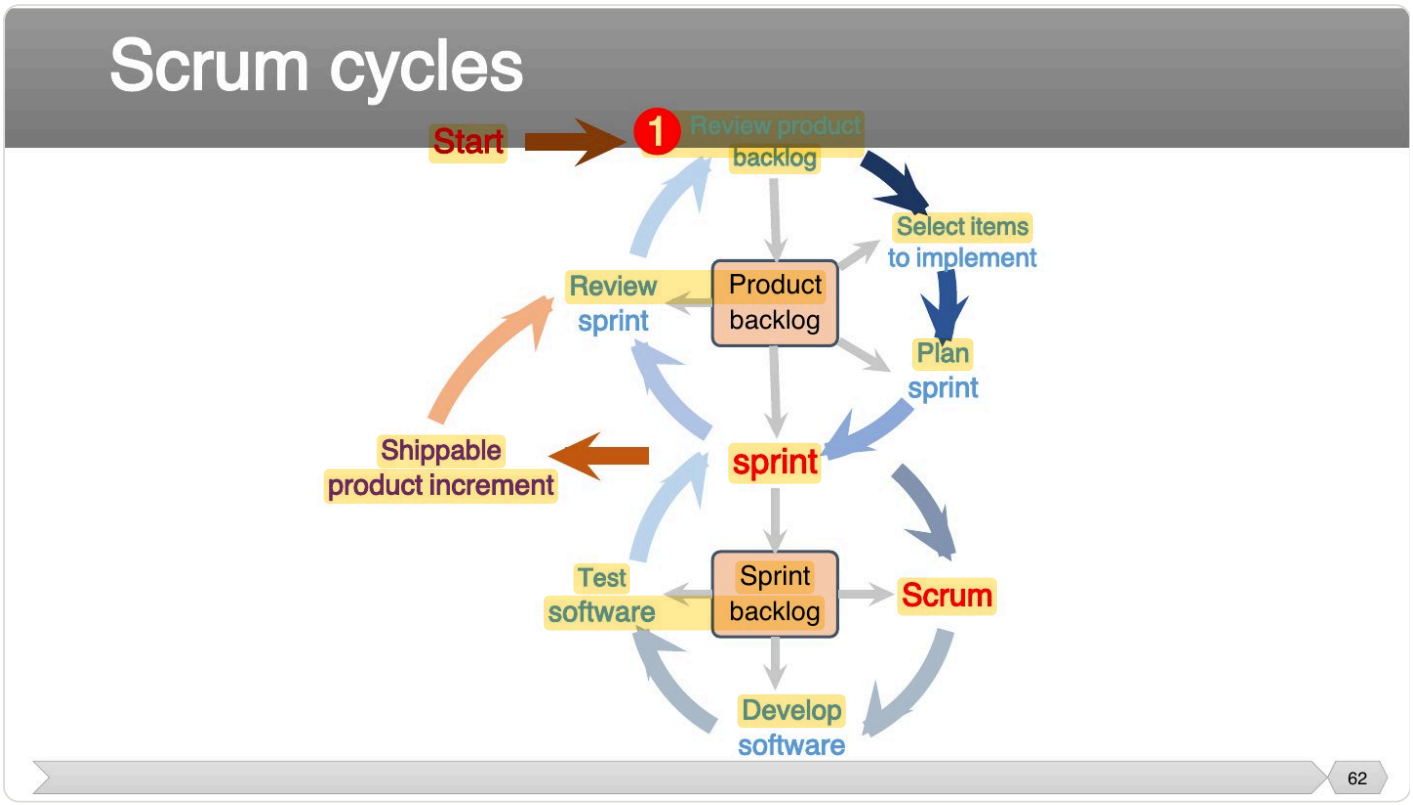
SLIDE 61 / 98 **Scrum and sprints**



NOTES

1. Since the length is fixed, a slipping task doesn't extend the **sprint**. Work gets dropped instead, and this makes delivery dates predictable.
2. A problem is visible within a day, and teammates can help right away. That's the value of **daily meetings**: short feedback, not status reporting to a boss.
3. "Shippable" stops the team from piling up unfinished work to fix later. Each sprint's **increment** really is complete, so you always have something you could release.

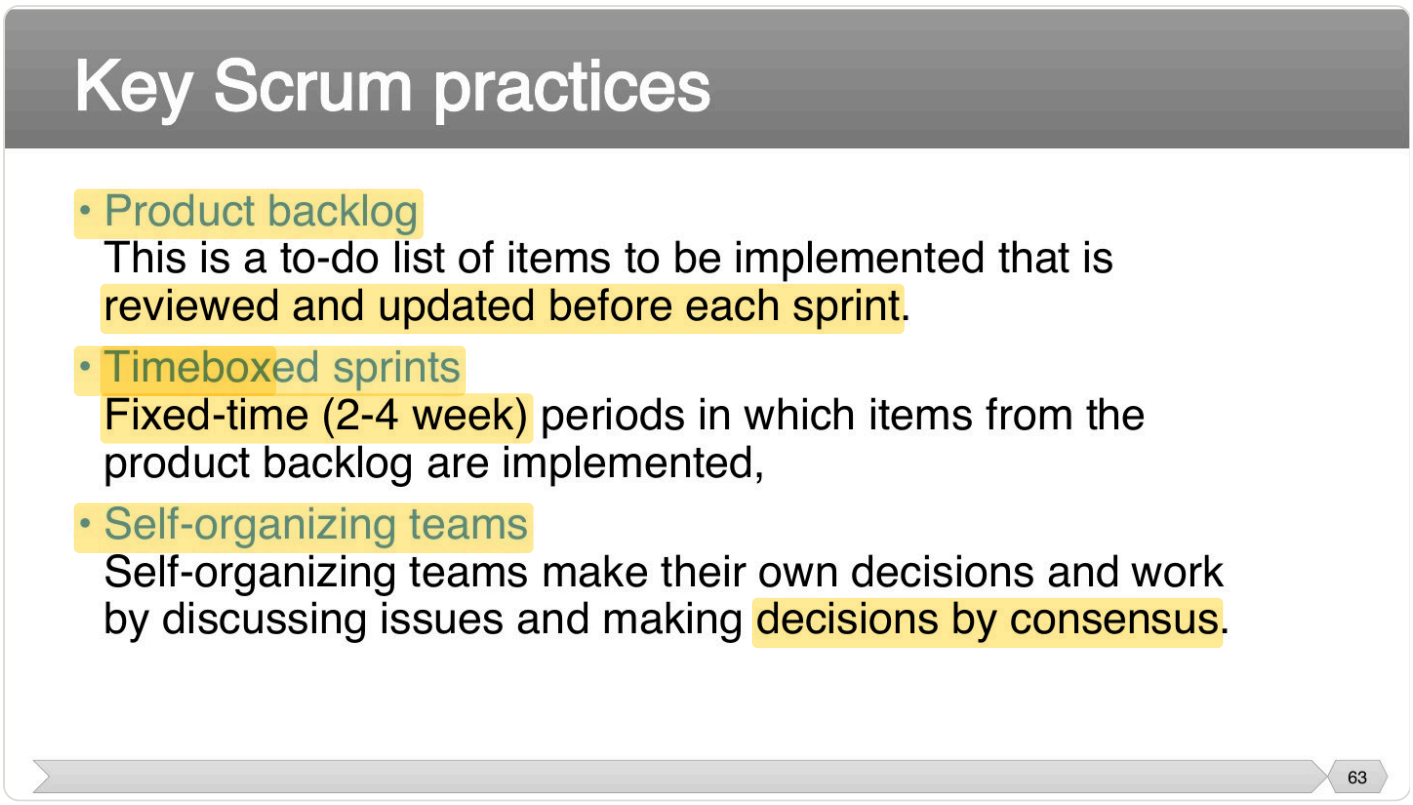
SLIDE 62 / 98 **Scrum cycles**



NOTES

1. The team pulls the most valuable items it believes it can finish. Because it's chosen by the team, the **sprint plan** is more realistic than if someone else had set it.
2. During the sprint the team **develops and tests** software, with a daily Scrum.
3. A sprint's result is seen by stakeholders in the review, and what they say feeds the next one. Repeating this makes the **shippable increment** a regular way of learning.

 Key Scrum practices



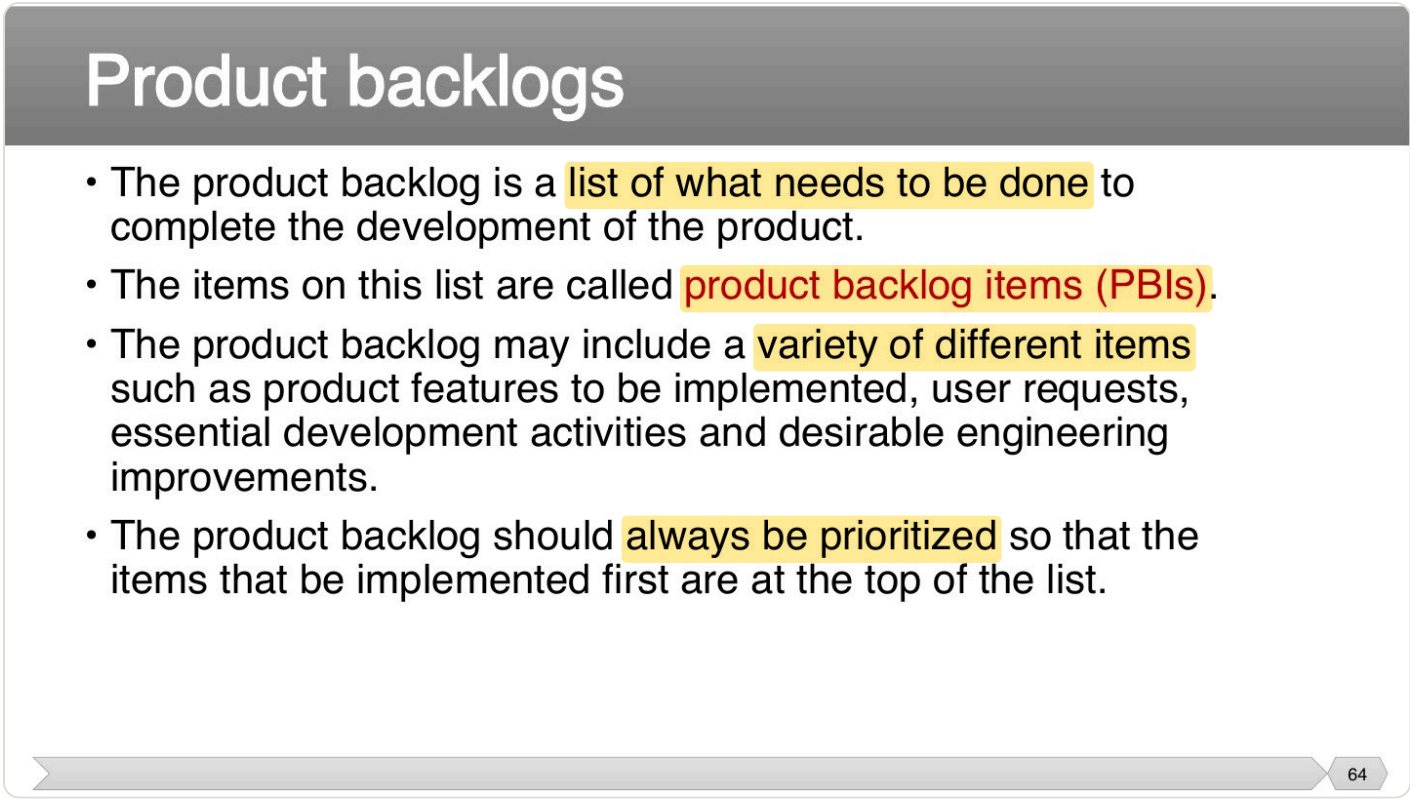
NOTES

1. The list changes constantly as you learn. Updating it **before each sprint** means the team always works on what's most valuable right now.
2. Fixing the time forces prioritizing: you can't fit everything, so what's important has to go first. That's what makes a **timebox** powerful.
3. They're closest to the work, so they often decide best, and people commit to their own decisions. **Consensus** also spreads knowledge across the team.

TIPS & TRICKS

- **STeP**: Self-organizing teams, Timeboxed sprints, Product backlog.

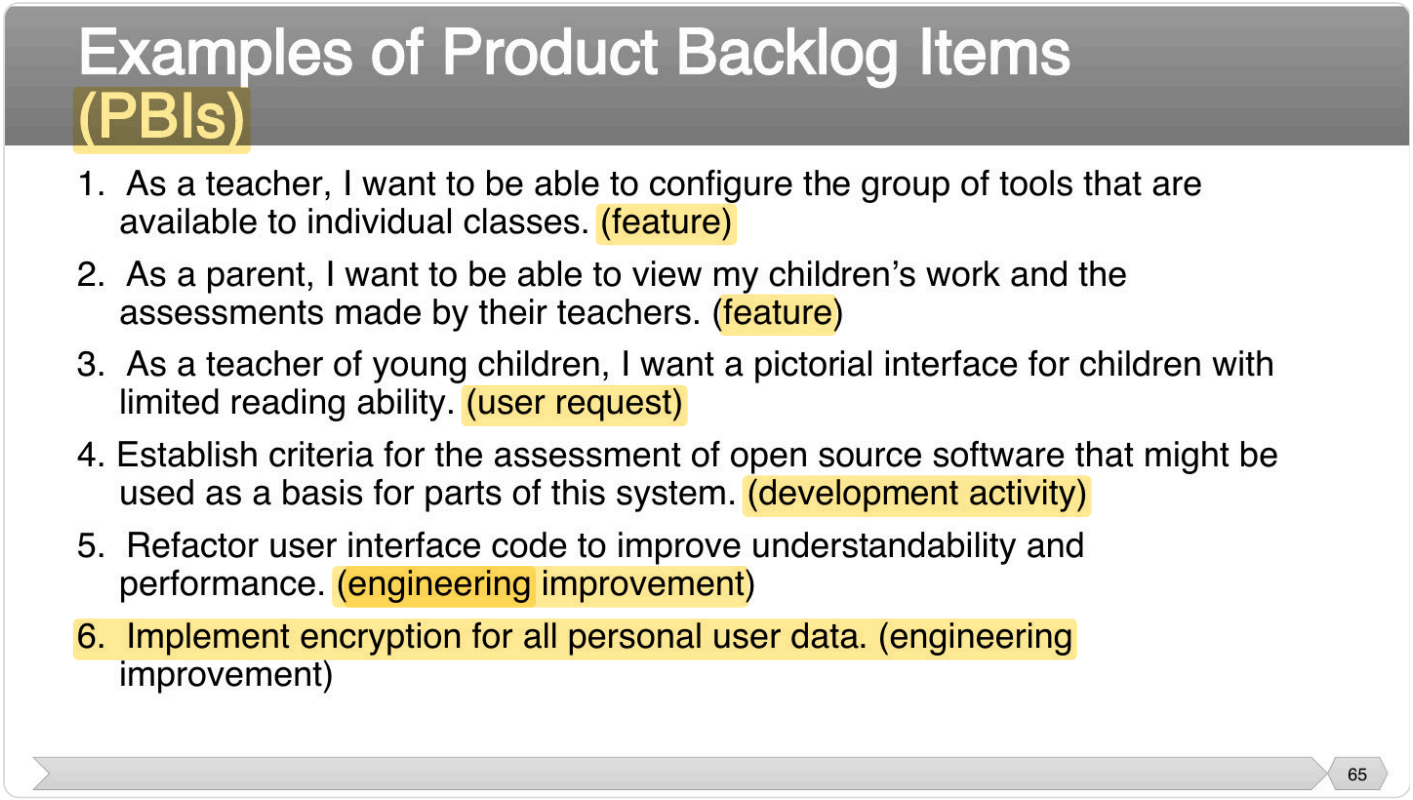
 Product backlogs



NOTES

1. If it's not on the list, it doesn't get done. This gives one place to see all remaining work, so the **product backlog** is what gets prioritized and discussed.
2. A **PBI** is just the name for a single entry. You'll see the short form in exam questions, so don't be thrown by it.
3. It's not only features! Engineering work needs a place too, or it never gets scheduled. Keeping all **varied** work on one list forces honest trade-offs.
4. Order matters, since the team always takes from the top. An **always prioritized** list means whoever is available starts on what's most valuable next.

 Examples of Product Backlog Items



NOTES

1. Notice the form: who wants it and what they want. A **feature** is written from the user's point of view, so everyone can see who's helped by it.
2. This came directly from what users said they need. A **user request** may become a feature, but it starts as somebody's stated need.
3. No user will ever see this, and it's still needed. Choosing which open-source software to rely on is a **development activity** that keeps the team able to build.
4. Users won't notice this directly either, but it protects them or keeps the system healthy. An **engineering improvement** is easy to neglect if it isn't on the backlog.

EXAM ANGLE

Prediction Q1

"Refactor user interface code to improve performance" is which kind of PBI?

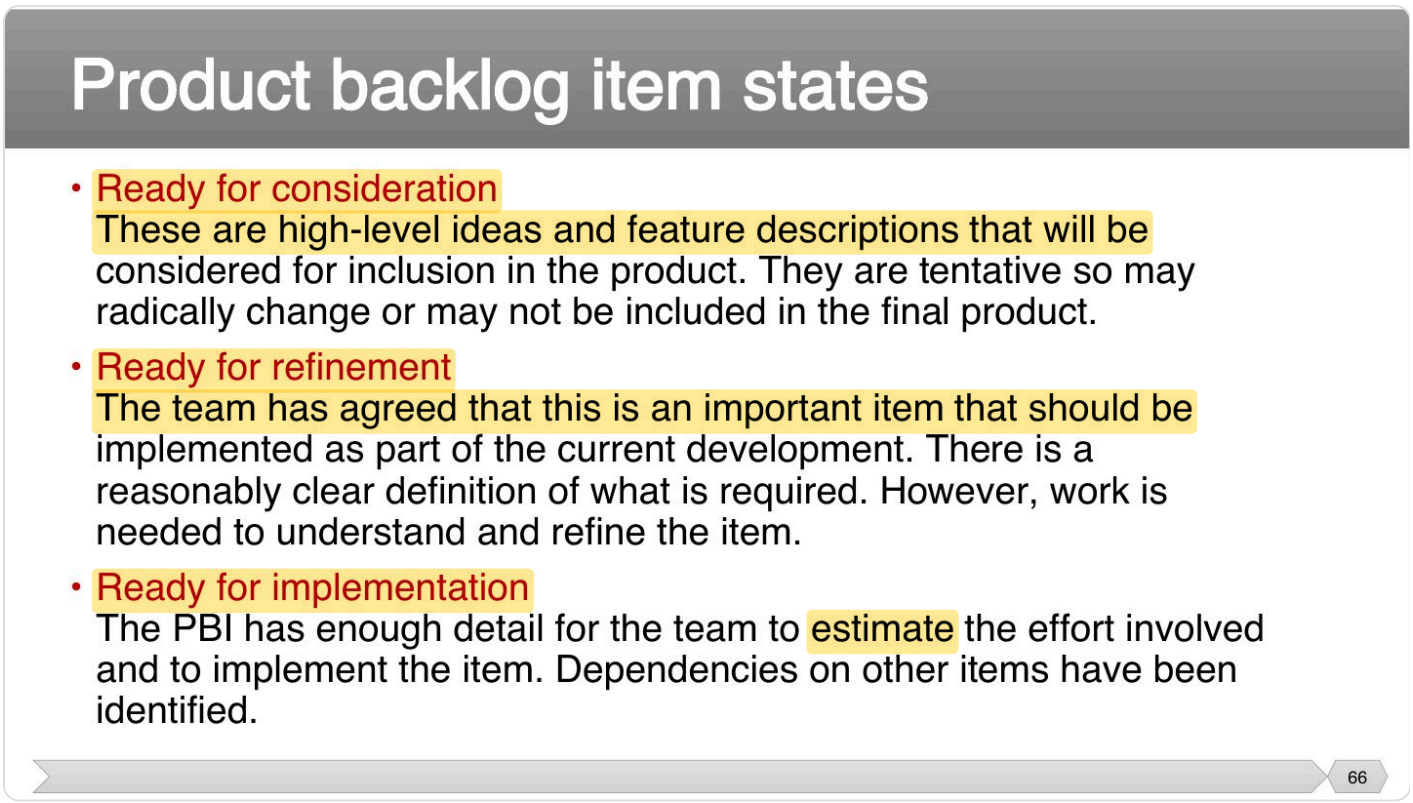
A Development activity

B **Engineering improvement**

C User request

D Feature

 Product backlog item states



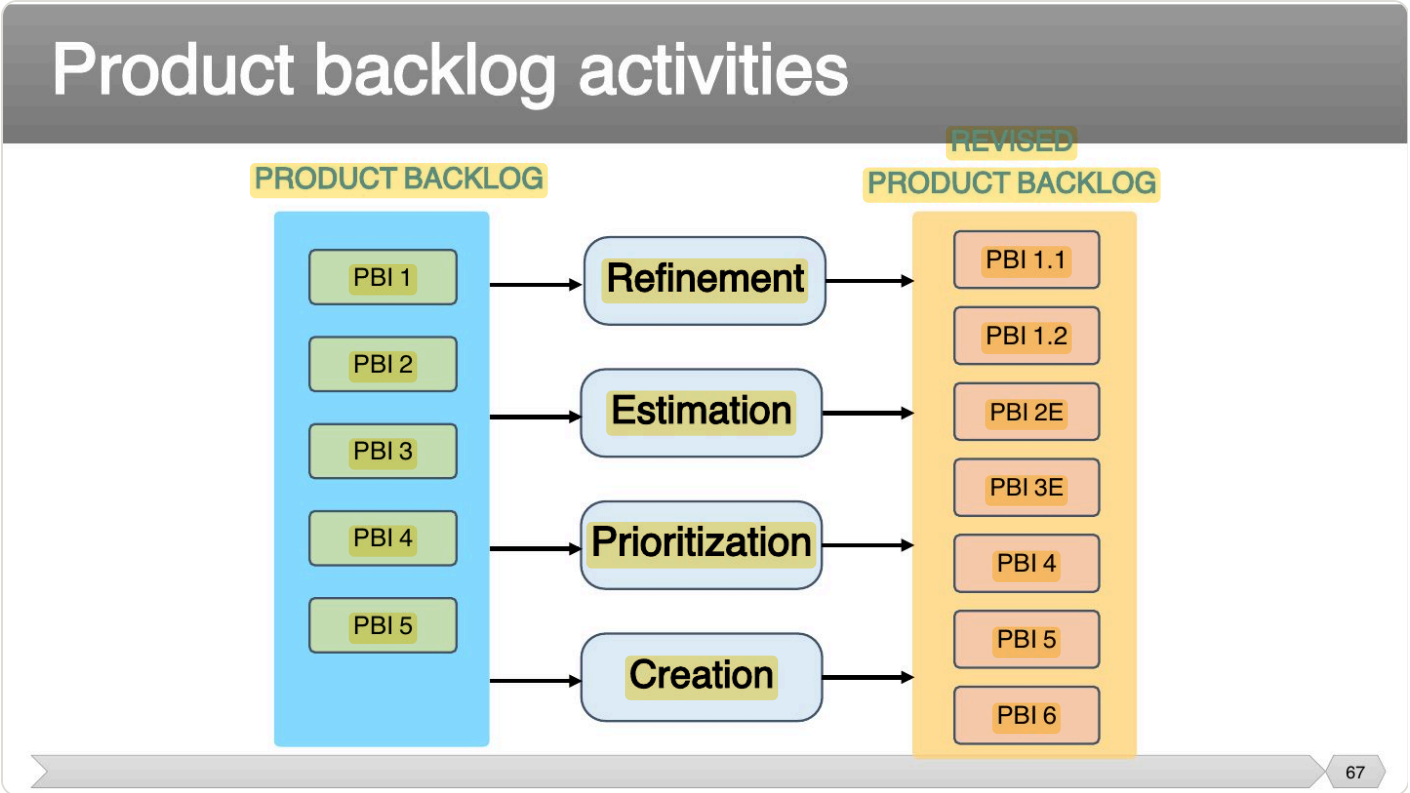
NOTES

1. Don’t spend time detailing something that may never happen. At **consideration**, a line or two is enough, and dropping it costs nothing.
2. Now it’s worth investing some thought, as you’ve decided it matters. **Refinement** answers the open questions, so that it can be built.
3. The test for this state is whether the team could start tomorrow. If it’s possible to **estimate** it and nothing is blocking it, the item is ready.

TIPS & TRICKS

- **CRIb**, already in order: Consideration, Refinement, Implementation.

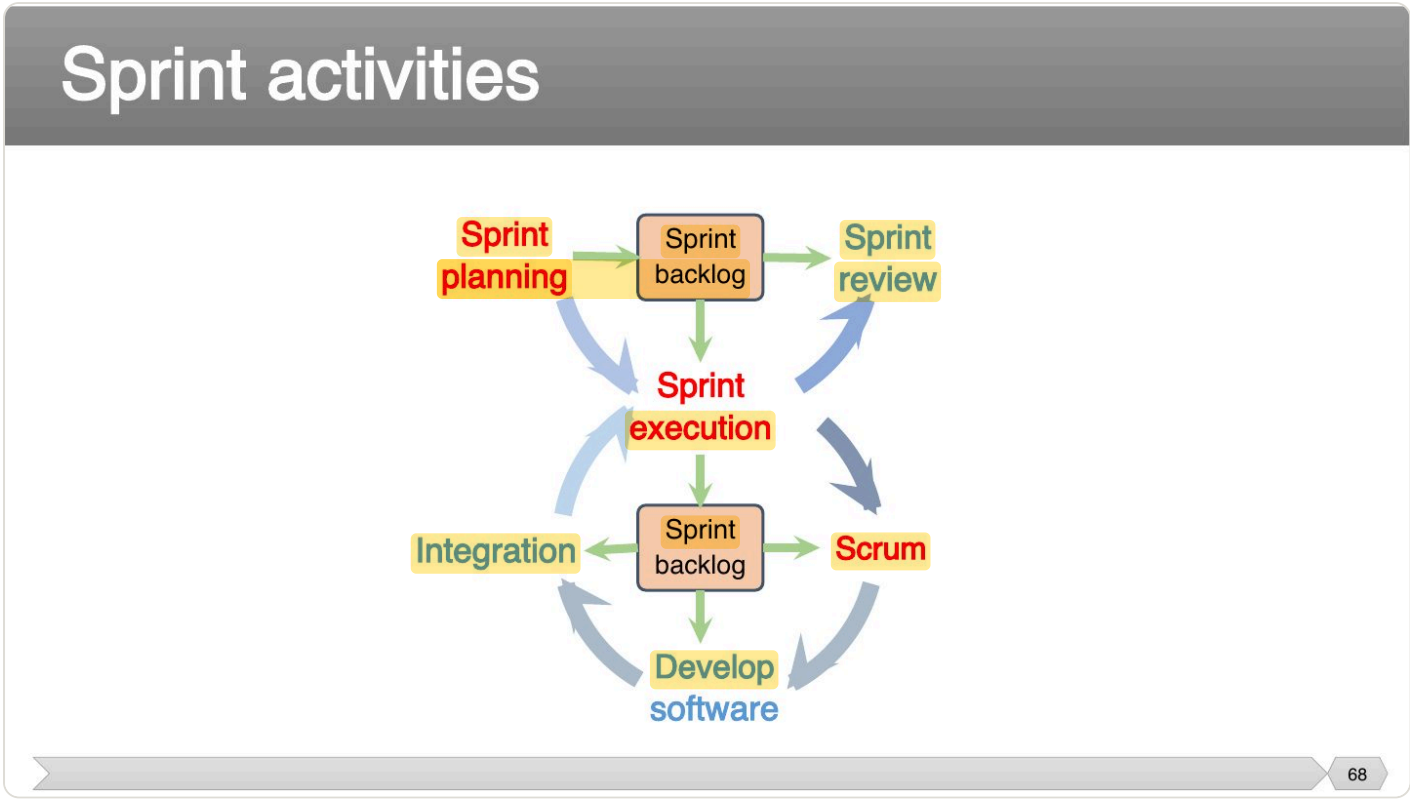
 Product backlog activities



NOTES

1. **Refinement**: break PBIs into smaller, clearer items (PBI 1 becomes 1.1 and 1.2).
2. **Estimation**: estimate the effort for items (PBI 2E, 3E).
3. **Prioritization**: reorder the backlog by value.
4. **Creation**: add new items as they are discovered (PBI 6). The result is a revised backlog.

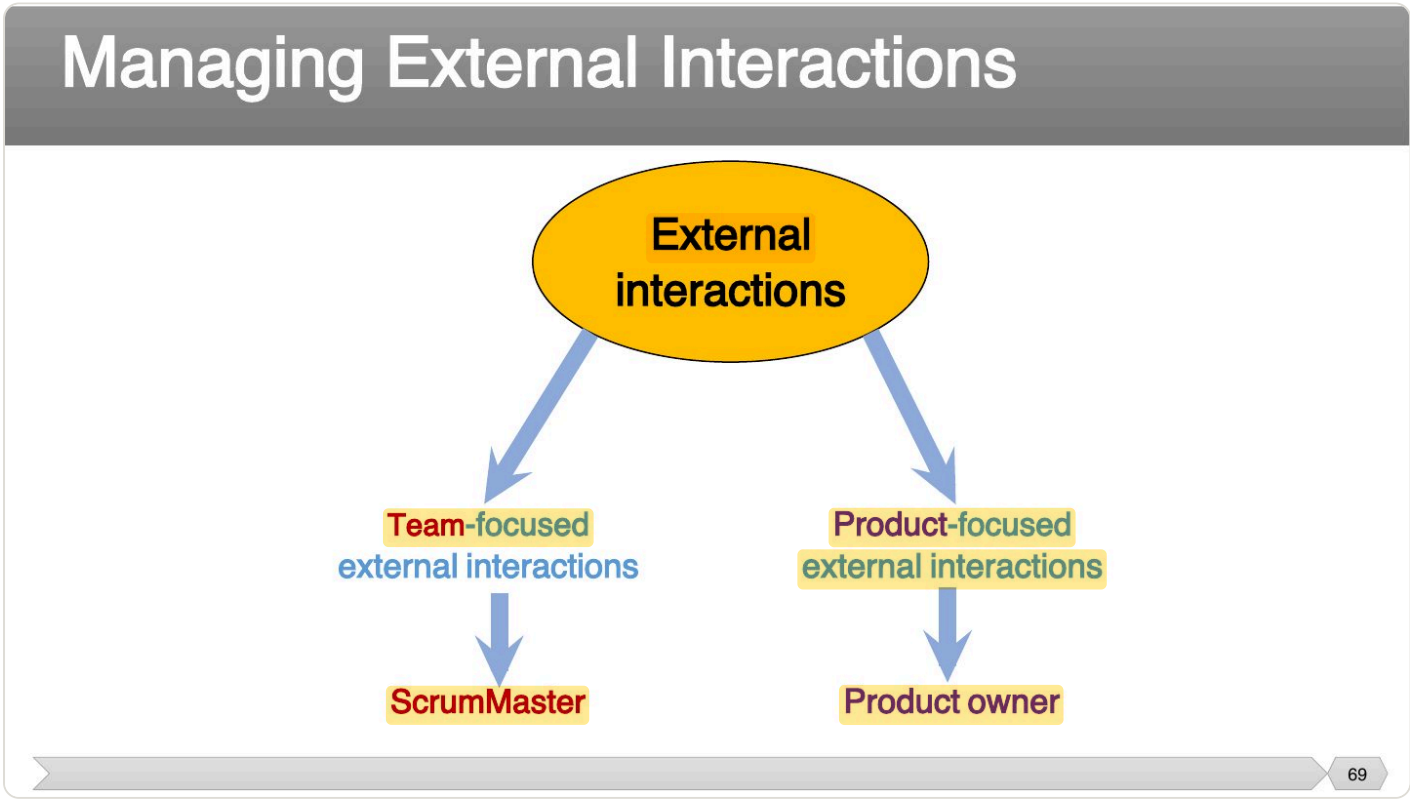
SLIDE 68 / 98 **Sprint activities**



NOTES

1. Planning sets what this sprint is for, then **execution** is the work. The team checks in daily, so a problem can't hide for long.
2. Work is continuously **integrated**, and the **sprint review** closes the sprint.

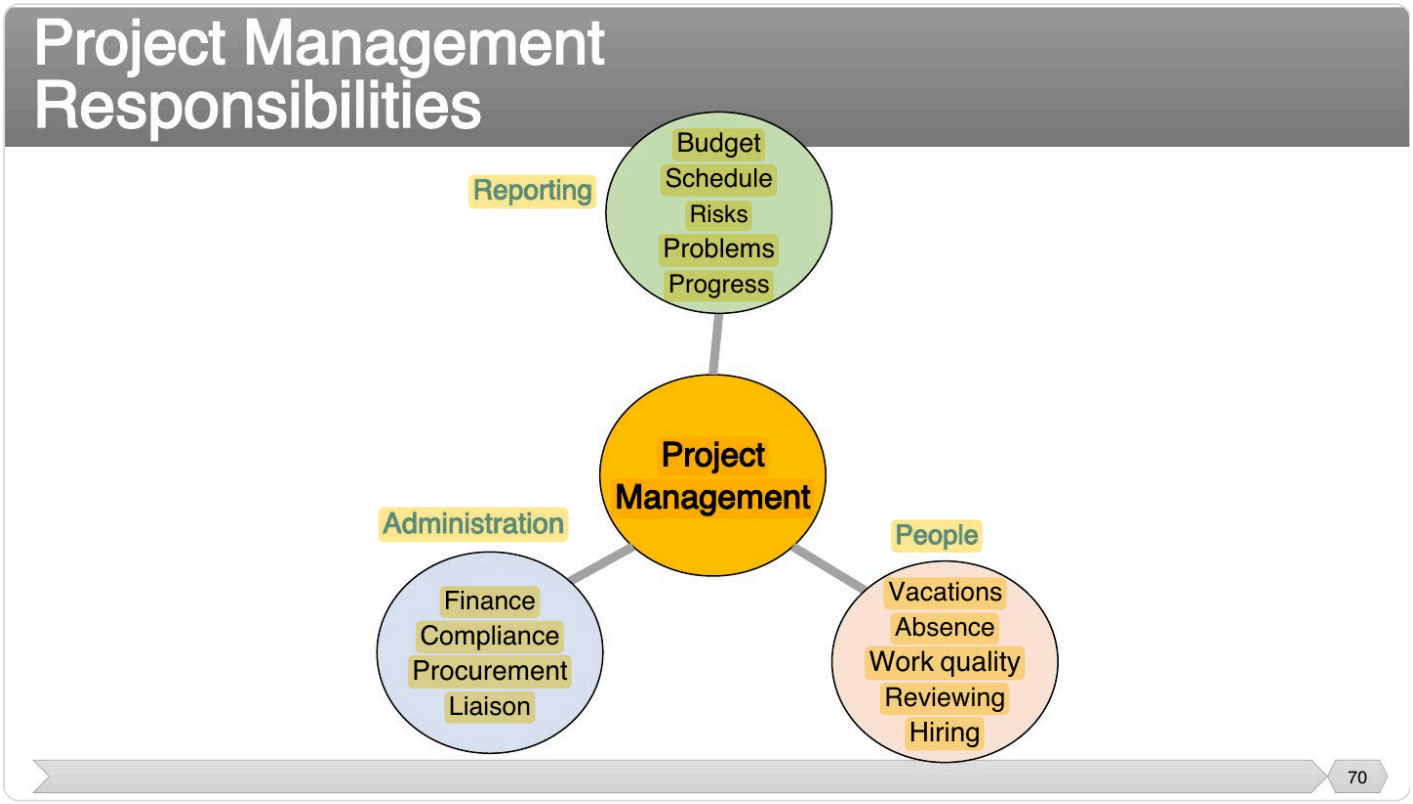
SLIDE 69 / 98 **Managing External Interactions**



NOTES

1. If everybody can interrupt developers, they stop getting work done. So the **ScrumMaster** handles requests about the team, like reporting and HR, and shields them.
2. **Product-focused** ones (customers, marketing, sales) go through the **Product Owner**. Either way, the developers are shielded from distractions.

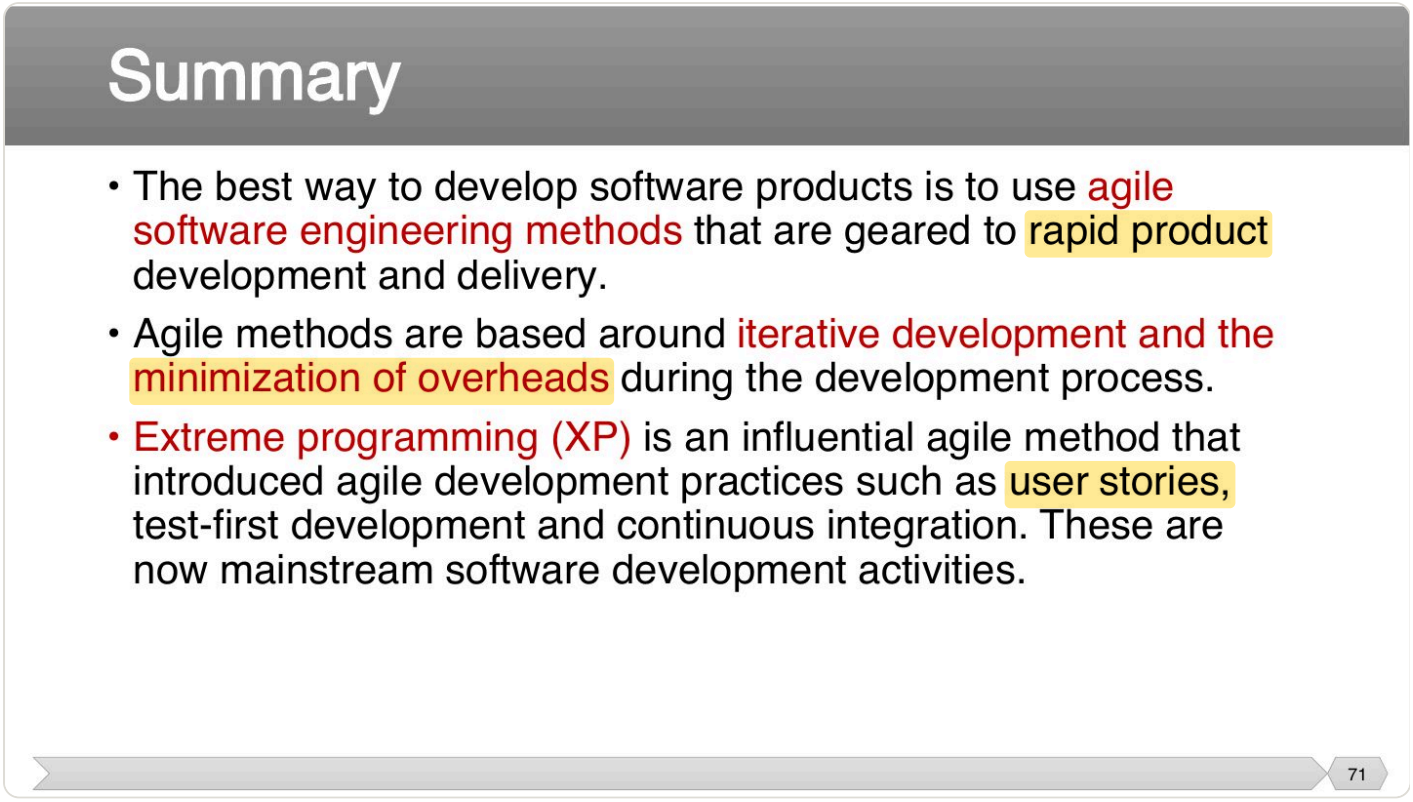
SLIDE 70 / 98 Responsibilities



NOTES

1. Scrum doesn't remove the business's need for information. Someone still has to **report** on money and progress, even if the team runs itself.
2. Rules still apply and bills still need paying, however agile the team is. So **administration** doesn't disappear, and Scrum doesn't say who handles it.
3. Scrum says little about holidays or hiring, but they still need handling. So **people** tasks often fall to a ScrumMaster or PM. Hence "ScrumMaster isn't a PM" is only partly true in practice.

 Summary



NOTES

1. "Best way" is the slide's claim for products, where speed matters. For **rapid product** development, agile fits because it's built for frequent delivery...
2. ...and how? By working in short cycles and cutting whatever doesn't help, so **minimal overhead** leaves more time for building...
3. You'll meet **XP's** practices even in teams that have never heard of XP, since **user stories** and continuous integration became standard everywhere.

Summary

- Scrum is an agile method that focuses on agile planning and management. Unlike XP, it does not define the engineering practices to be used. The development team may use any technical practices that they believe are appropriate for the product being developed.
- In Scrum, work to be done is maintained in a product backlog – a list of work items to be completed. Each increment of the software implements some of the work items from the product backlog.

NOTES

1. Keep the two straight. XP says how to write the code, and **Scrum** says how to organize the work. So they fit together, and Scrum **does not define engineering practices**.
2. One list holds all the work, and each sprint takes some of it. The **product backlog** is what links long-term plans to daily work.

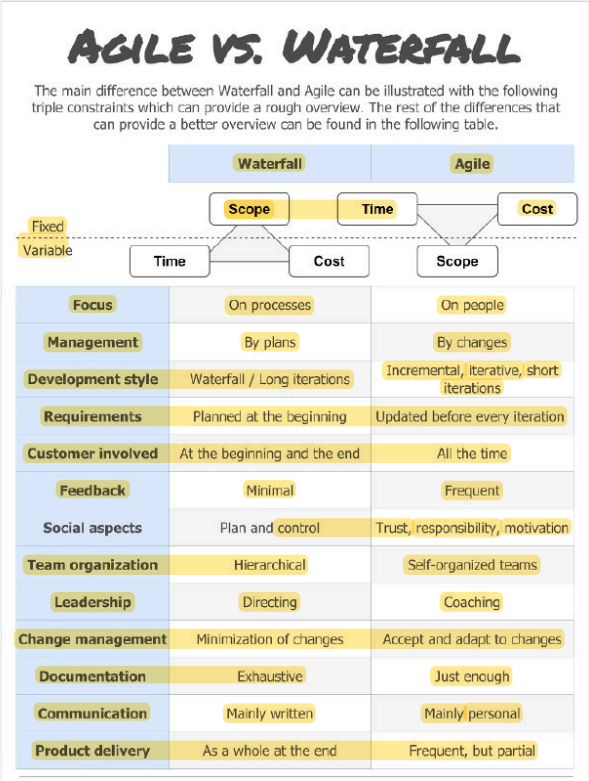
Summary

- Sprints are fixed-time activities (usually 2–4 weeks) where a product increment is developed. Increments should be 'potentially shippable' i.e. they should not need further work before they are delivered.
- A self-organizing team is a development team that organizes the work to be done by discussion and agreement amongst team members.
- Scrum practices such as the product backlog, sprints and self-organizing teams can be used in any agile development process, even if other aspects of Scrum are not used.

NOTES

1. No “we’ll finish it next sprint”. **Potentially shippable** means the increment is really done, so progress is real and not just reported.
2. The team decides together who does what. That only works with skilled people who trust each other, so **self-organizing** isn’t something you can just announce.
3. You don’t have to adopt all of Scrum. Many teams use just a backlog and sprints, which is why **any agile process** can borrow these.

SLIDE 74 / 98 Agile vs. Waterfall



NOTES

1. The key line in the triangle: **waterfall fixes scope** and lets time and cost vary; **agile fixes time and cost** (the timebox and team) and lets scope vary.
2. The table compares the rest: waterfall focuses **on processes**, manages by plans, plans requirements up front, involves the customer at the start and end, and delivers everything at the end; agile focuses **on people**, manages by change, updates requirements every iteration, involves the customer all the time, uses self-organized teams and coaching leadership, and delivers frequently in parts.

EXAM ANGLE

Prediction Q1

In agile, which of the triple constraints is typically allowed to vary?

A Scope

B None of them

C Cost

D Time

Past exam · Midterm 251 · V1 · Case Q2 (also V2) Q2

Given the evolving requirements (student usability, advisor requests, IT integration rules), which development approach (Agile, Waterfall, or Hybrid) would you recommend, and why would it be the best fit for this project?

Answer: Agile (Scrum) (0.5), because it handles the evolving requirements from students, advisors and IT, delivers iteratively so each sprint gets feedback, and integrates with DevOps pipelines for frequent tested releases (0.5). Justify it against the slide criteria: the requirements are reasonably clear but not stable, so waterfall's "clear, unambiguous, stable requirements" condition fails.

SLIDE 75 / 98 **DevOps**

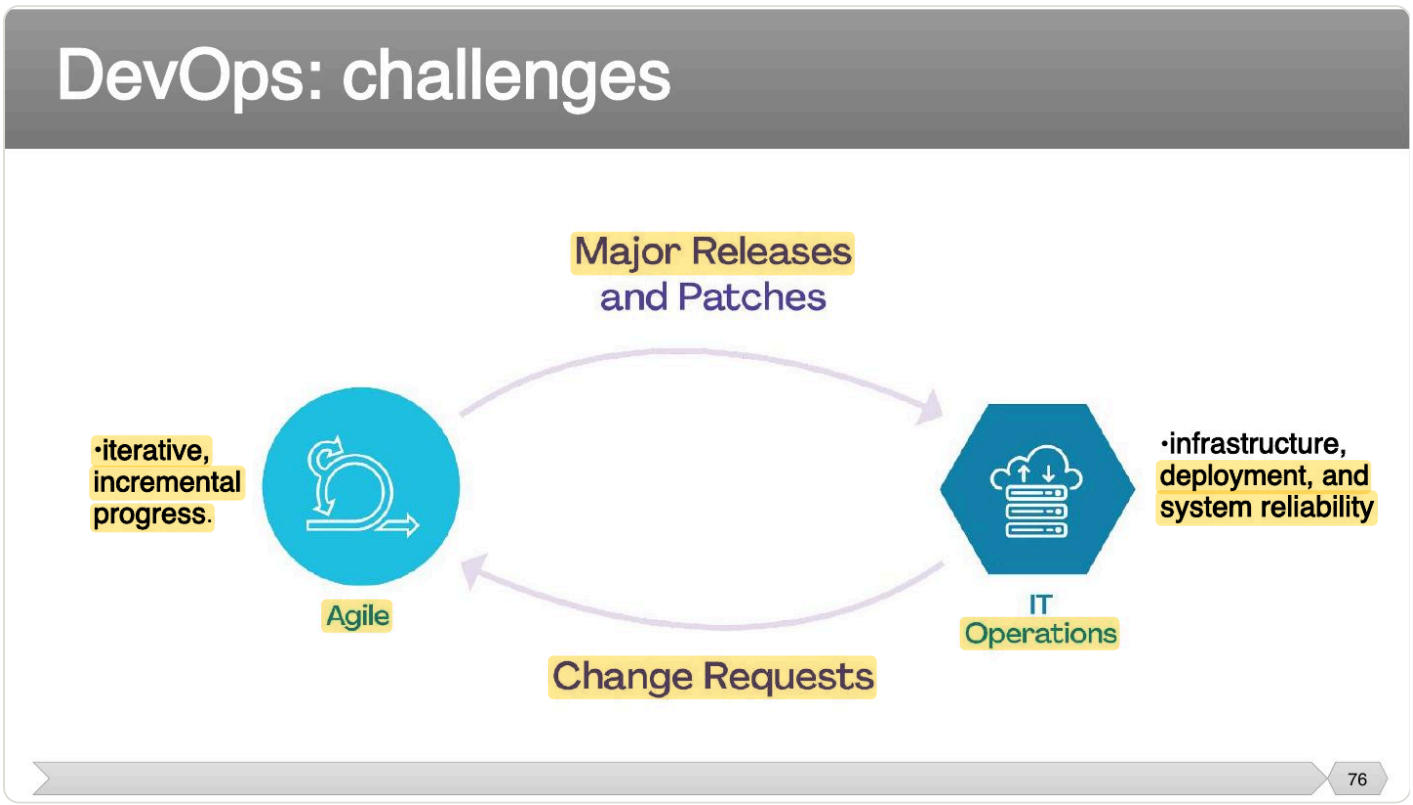


75

NOTES

1. Section divider: **DevOps** = **dev**elopment and **op**eration**s**.

SLIDE 76 / 98 DevOps: challenges



NOTES

1. The **DevOps gap**. Agile development delivers **iterative, incremental** progress and keeps sending change requests...

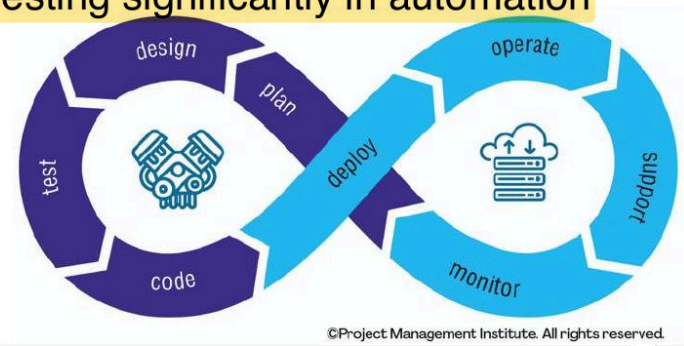
2. ...while IT operations care about infrastructure, deployment and **system reliability**, and resist frequent major releases and patches. Fast change and stability pull against each other.

 DevOps

DevOps

h

- When organizations address these challenges by removing the barriers that inhibit effective collaboration (operations <-> dev), they close the DevOps gap.
- Organizations do this by: adopting a mindset that promotes collaborative; learning-centric ways of working that are supported by agile practices; and very often investing significantly in automation
- On the development side they tend to adopt a continuous delivery (CD) approach and their operations efforts become streamlined and more automated.



©Project Management Institute. All rights reserved.

77

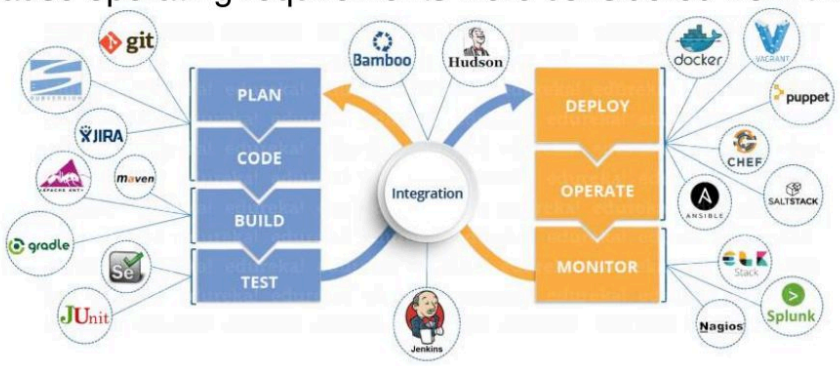
NOTES

1. The two sides often have opposed targets: one is judged on shipping, the other on stability. **Removing the barriers** starts with shared goals, not just talking more...
2. ...and a mindset of blame prevents this. If an outage leads to learning, rather than punishment, people share information. **Automation** then makes frequent releases safe to do...
3. Developers can ship often once **continuous delivery** is in place, and operations gets predictable, automated deployments. That way neither side has to give up its goal.

 DevOps

DevOps

- DevOps is an approach to software development that extends the principles of agile development by integrating both the development and operations units of the IT team into one homogenous unit.
- This integration enables a more seamless flow between the traditionally siloed sides of the software development lifecycle and leads to fewer defects in production because operating requirements were considered from the outset of the project.



78

NOTES

1. Agile joined business and developers. **DevOps** extends that to the people who run the software. So it isn't a replacement for agile but an extension, which is why it's **one unit**.
2. When developers know how their software will run, they design for it from the start. That's why there are **fewer production defects**: operating needs aren't an afterthought.

EXAM ANGLE

Prediction Q1

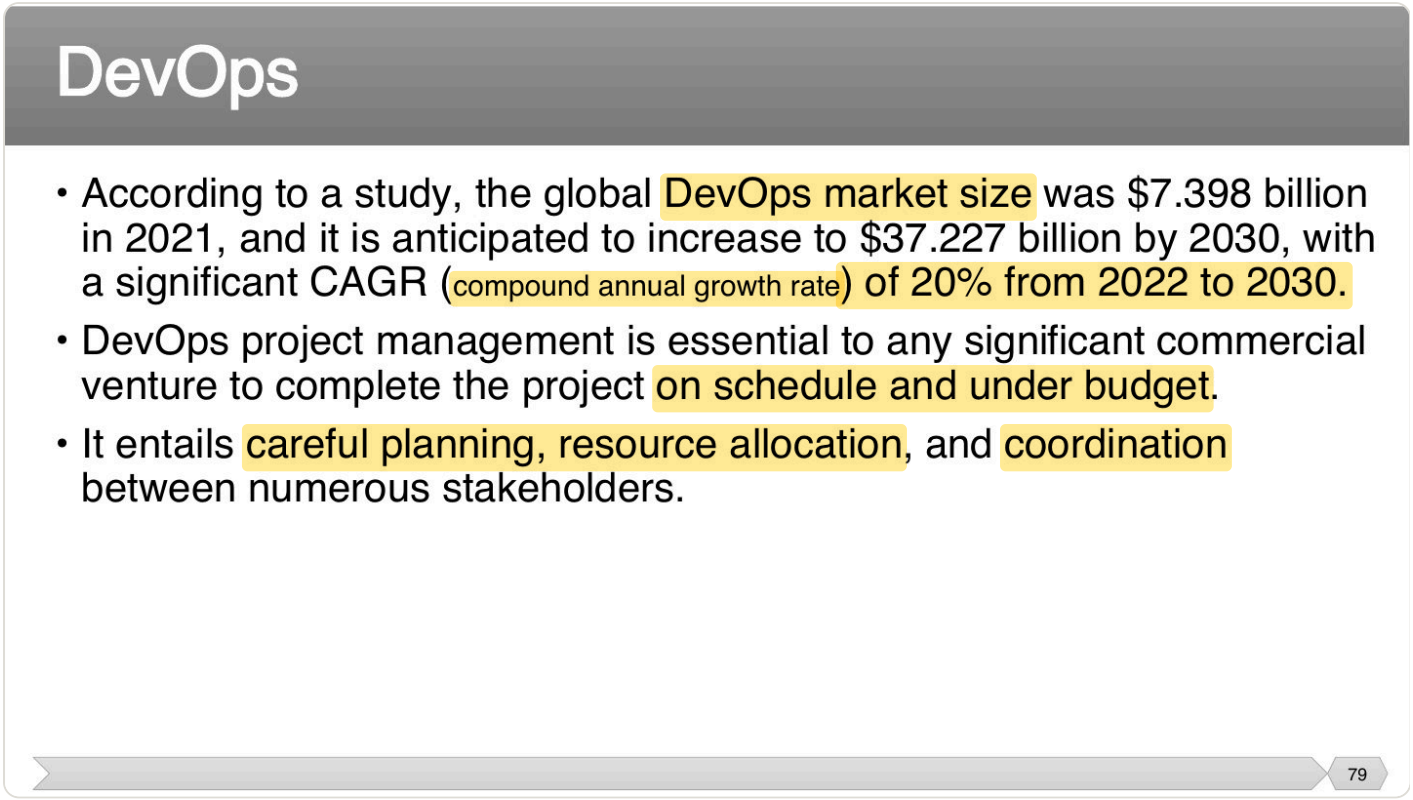
DevOps extends agile principles by:

A Removing testing

B Separating developers from operations

C Fixing scope up front

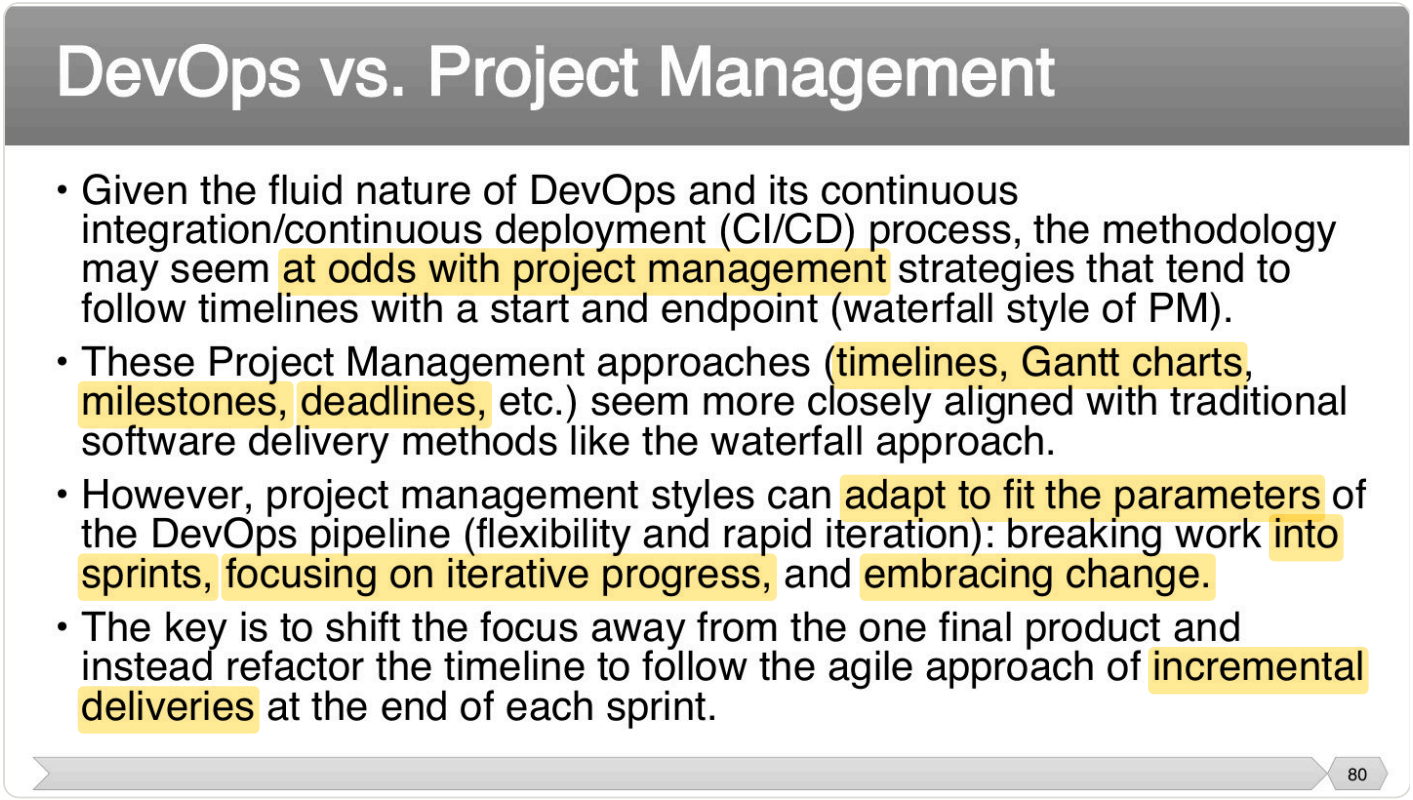
D **Integrating development and operations into one unit**



NOTES

1. The market figures are context only (not exam material): DevOps is growing fast.
2. Continuous delivery can make deadlines seem unimportant, but a business still has budgets and commitments. So DevOps still needs someone focused on **on schedule and under budget**...
3. ...and with many teams and tools involved, **coordination** becomes a real job. Fast-moving work needs this more, not less.

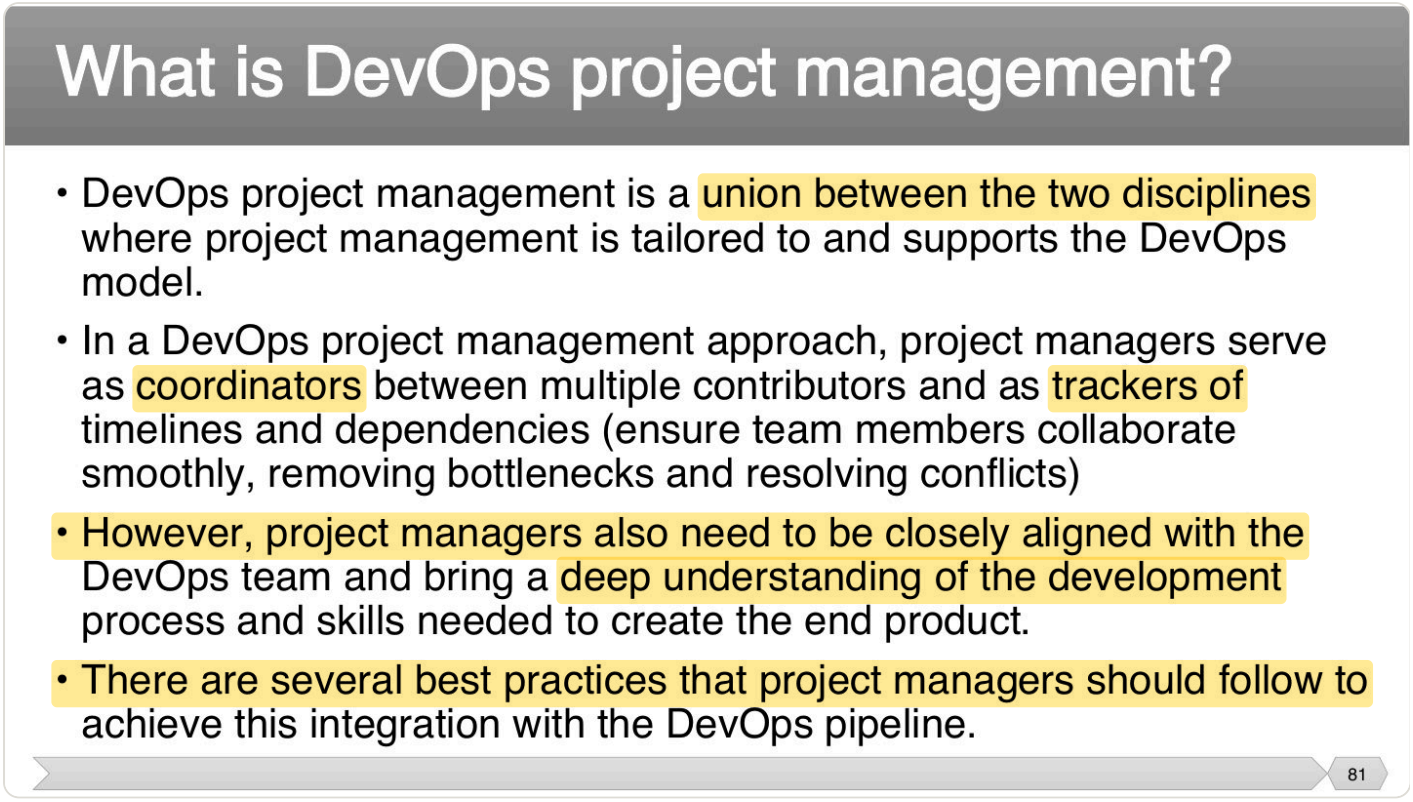
 DevOps vs. Project Management



NOTES

1. Traditional PM assumes a project starts and ends, while DevOps never really stops. That's the apparent conflict: where's the finish line, if it seems **at odds with project management**?
2. A Gantt chart assumes that you know the tasks and order well in advance. **Timelines and milestones** suit predictable work, and they're harder to apply when everything keeps changing.
3. The tools change, while the job remains: know what's happening and help it go well. So PM can **adapt** by planning in short cycles.
4. Each sprint's output becomes the milestone, rather than one distant end date. So PMs plan **incremental deliveries**, which keeps the benefit of tracking without the fixed end.

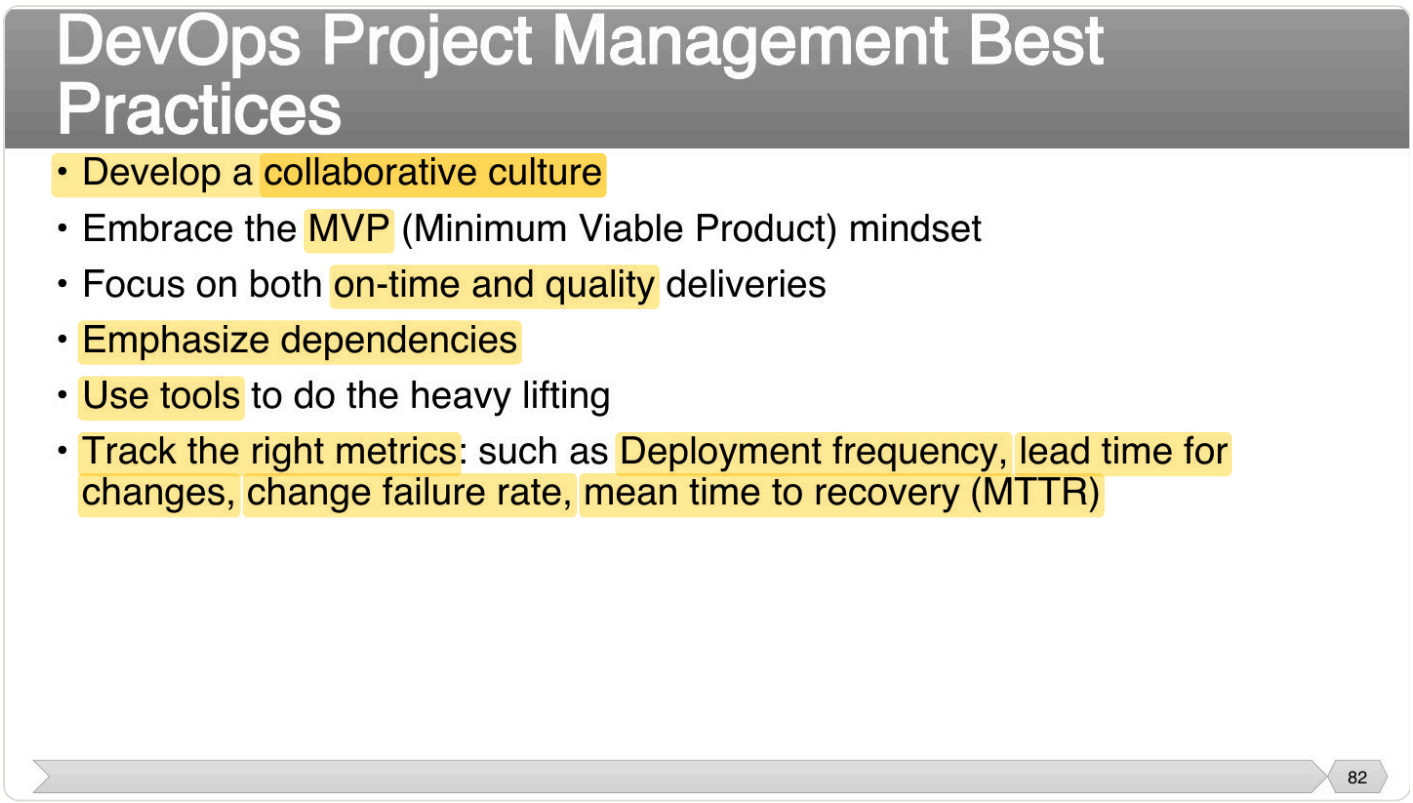
 What is DevOps project management?



NOTES

1. It's not choosing one over the other. **DevOps project management** keeps PM's planning and tracking, adjusted to fit a continuous way of working.
2. With so many people and tools, someone has to see the whole picture. A **coordinator** notices when one team's delay will affect another, and clears the way.
3. A PM who doesn't understand how code gets built and released can't spot problems or set sensible expectations. That's why a **deep understanding** of development is needed here.

 Practices



NOTES

1. Everything else depends on this: tools can't fix teams that blame each other. So **collaborative culture** comes first on the list...
2. ...shipping the smallest useful thing keeps feedback coming, so the **MVP** mindset fights the urge to wait until it's perfect...
3. ...speed and quality aren't opposites if testing is built in, but you do need to watch both. So **on-time and quality** are listed together...
4. ...with frequent releases, a blocked task can hold up many others quickly. **Dependencies** need tracking all the time, rather than once at the start...
5. ...people shouldn't do by hand what a machine does reliably. **Tools** free up attention for the problems that actually need thinking...
6. ...and **track the right metrics**: deployment frequency, lead time for changes, change failure rate and mean time to recovery (MTTR), the four DORA metrics.

TIPS & TRICKS

- **COMMiTteD**: Collaborative culture, On-time and quality, MVP mindset, Metrics, Tools, Dependencies.

EXAM ANGLE

Past exam · Final 251 · Q1.4 Q1

Give an example of a project management tool or technique that you might use to manage a complex software development project, and explain how it could help you ensure project success.
Answer: A Work Breakdown Structure: it decomposes the complex system into smaller deliverables and work packages that can be estimated, assigned and tracked, so nothing is forgotten, ownership is clear, and it is the base for the schedule, budget and risk analysis. Other valid answers, each with its benefit explained: a Gantt chart, CPM, a risk register, or a Scrum board with a burndown chart and Jira/CI dashboards for real-time visibility (slide 87).

Prediction Q2

Which is one of the DevOps metrics listed on the slide?

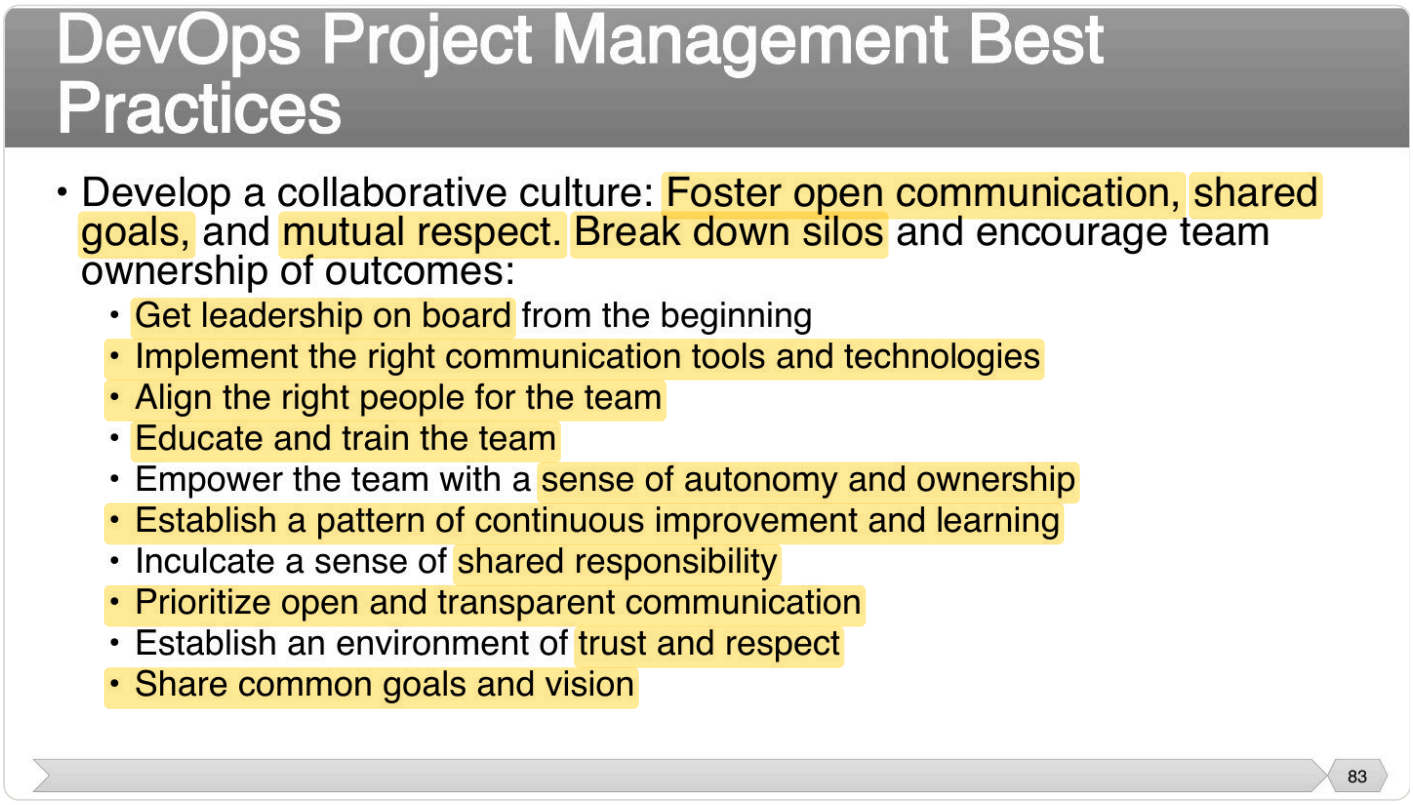
A **Mean time to recovery (MTTR)**

B Number of meetings

C Lines of code per day

D Cost per printed page

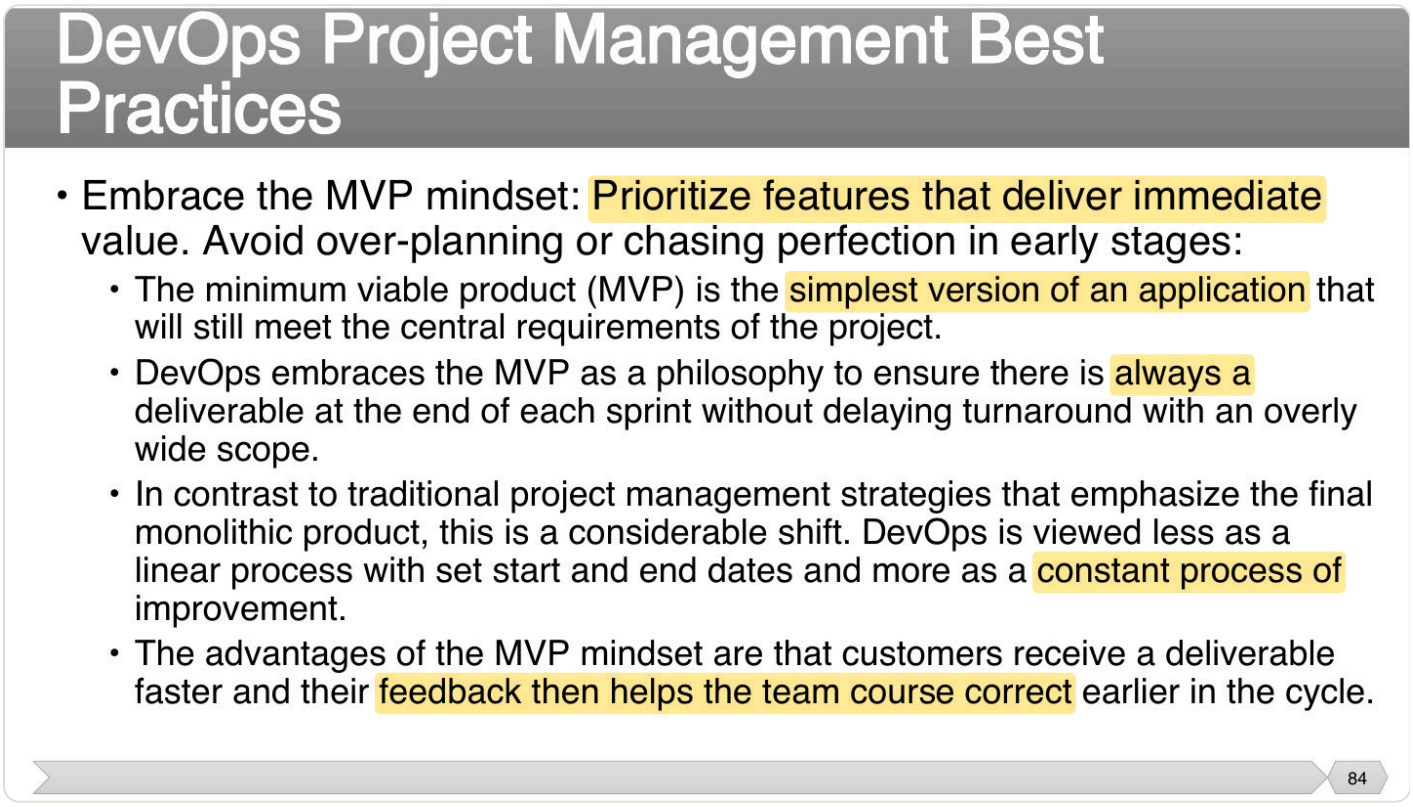
SLIDE 83 / 98 **Practices**



NOTES

1. A **silo** is a team that protects its own area. DevOps can't work with these, because a problem belongs to everyone, so shared goals matter more than any tool.
2. Culture change fails without backing from the top, since old targets keep rewarding old behavior. So get **leadership on board** first. People need new skills too, so **train** them.
3. People take care of what they own. Giving them real **autonomy** means they fix what they see, and **shared responsibility** stops the old "not my problem" reaction.
4. Without **trust**, people hide mistakes, and you can't learn from what you can't see. A common vision gives everybody a reason to help one another.

SLIDE 84 / 98 **Practices**



NOTES

1. Perfect is the enemy of shipped. **Immediate value** is what to prioritize, since early feedback may change what "perfect" even means.
2. Not a poor product, but the smallest useful one. **Simplest** still has to meet the central need: a car without air conditioning counts, four wheels and no engine doesn't.
3. A wide scope might mean nothing's ready when the sprint ends. Keeping scope small means there's **always a deliverable**, so stakeholders always see something real.
4. Traditional PM aims for one final result, while DevOps never quite "finishes". So it's a **constant process** of improving, which changes how success gets judged.
5. You'd rather hear "this isn't what we need" after two weeks than after a year. Early **feedback** makes changing course cheap.

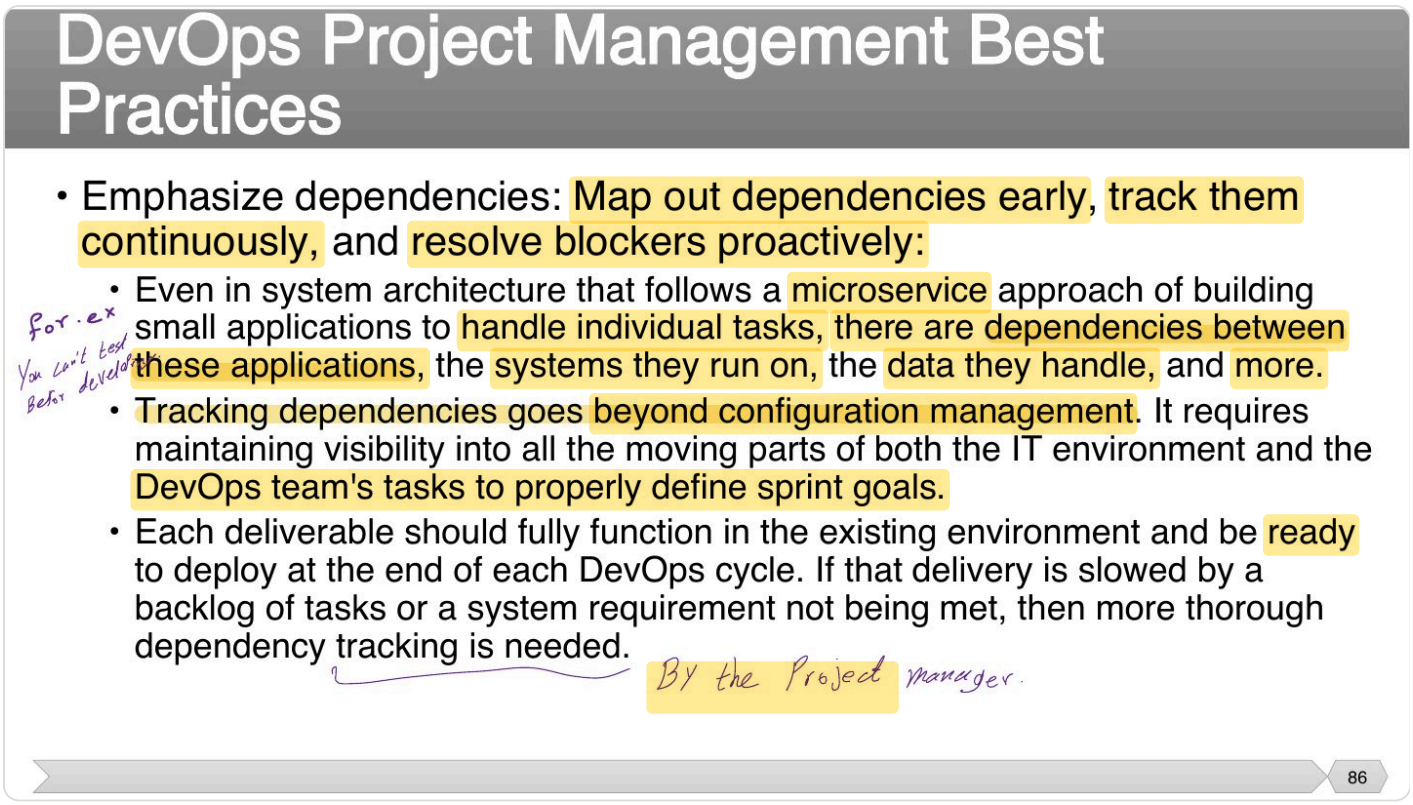
DevOps Project Management Best Practices

- Focus on both on-time and quality deliveries: Set realistic timelines, monitor quality metrics (e.g., defect rates, rollback frequency), and ensure testing is embedded in the pipeline:
  - Project management places a heavy emphasis on meeting deadlines. While this is important, many of the metrics DevOps teams use to measure success focus on more qualitative factors to gauge if the customer is meeting their goals with the application.
  - This means that quick turnaround times are important. So is allocating enough time and resources to ensure the deliverable quality meets a high standard. Project managers need to weigh both these factors when deciding on the goals for a sprint and assigning tasks.
  - Part of an effective delegation of responsibilities is understanding the dependencies for each task.

NOTES

1. Teams under deadline pressure tend to skip quality, so measure it directly. **Rollback frequency** shows how often releases had to be undone, a clear sign of rushing.
2. Shipping on time means little if customers can't achieve anything with it. **Qualitative** measures ask if it actually helped, not just whether it arrived.
3. If you only reward speed, people cut corners, and if you only reward quality, nothing ships. So the PM sets sprint goals that allow **enough** time for both.
4. You can't hand out work sensibly if you don't know what must happen first. Knowing **dependencies** tells you who can start now and who's waiting.

 Practices



NOTES

1. A blocker found late can stop several people. **Map them early**, and you can reorder work or warn someone before it becomes a problem.
2. Don’t assume that small, separate services mean independent ones. Even with **microservices**, one service’s change can break others that rely on it.
3. Tracking versions isn’t enough. You also need to see what people are working on, as well as the systems, so **beyond configuration management** means both together.
4. If deliveries keep getting stuck, treat that as a signal. Each deliverable has to be **ready to deploy**, so repeated blocking means your dependency tracking needs work.

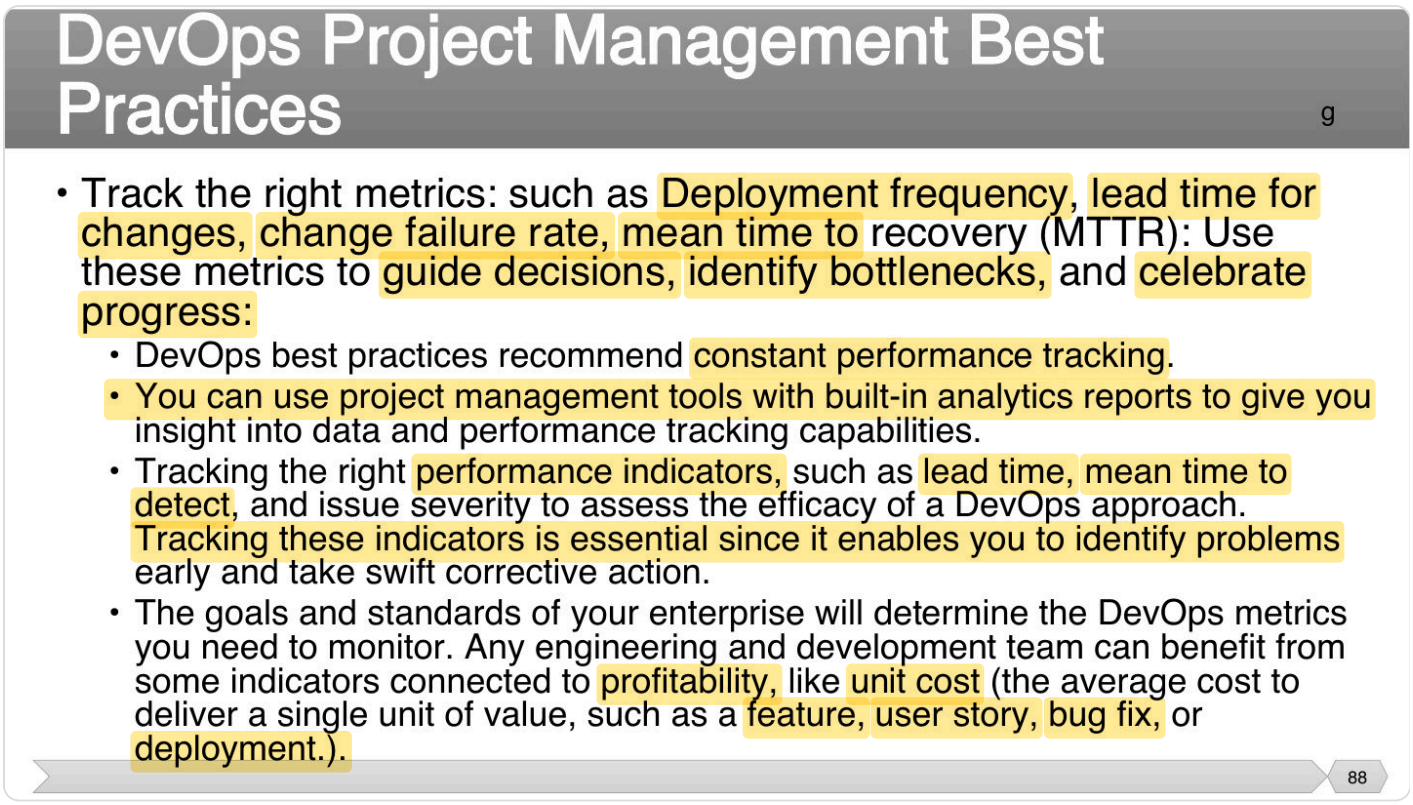
DevOps Project Management Best Practices

- Use tools to do the heavy lifting: Leverage dashboards, issue trackers, CI/CD platforms, and communication tools to streamline workflows and gain visibility:
  - Automation is a major focus in DevOps to scale the impact of a team's effort, and DevOps project management is not different.
  - PM tools provide a single source of truth for how work has been assigned, how the project is proceeding, and what issues have occurred along the way. This information is valuable for the entire team to provide greater context and foster more informed decision-making.
  - These applications will overlap and integrate with the tools the DevOps team uses in their day-to-day work so that both project managers and developers are on the same page (Example: A developer updates a ticket in Jira, and the PM sees the status instantly. A failed deployment in Jenkins triggers an alert that both teams can act on.)
  - Dashboards can provide real-time metrics on how the team is performing against their goals and help PMs detect potential issues to address earlier in the cycle.

87

NOTES

1. A team that does everything by hand can't release often. **Automation** and shared dashboards let the same people do much more, and see what's going on.
2. When different people keep different records, you waste time arguing about which is right. A **single source of truth** means everyone trusts the same information.
3. Nobody has to write a status report if the tools share it automatically. When a developer **updates a ticket**, the PM sees it at once, without interrupting anyone.
4. Checking last month's report means learning too late. **Real-time** metrics let a PM spot a problem today, and act while it's still small.



NOTES

1. These four cover speed and stability together. Fast deployment that keeps breaking isn't success, so you measure **deployment frequency** and failure rate together. They're worth learning by name.
2. Things change fast here, so an occasional check isn't enough. **Constant tracking** comes built into tools, which makes it almost free to collect.
3. The sooner you know, the less damage. **Mean time to detect** asks how long a problem exists before anyone notices, while severity tells you what to fix first.
4. Measure what the business actually cares about. **Unit cost** links engineering to money, which makes it easier to justify spending on improvements.

 outcomes.

DevOps project management leads to better outcomes.

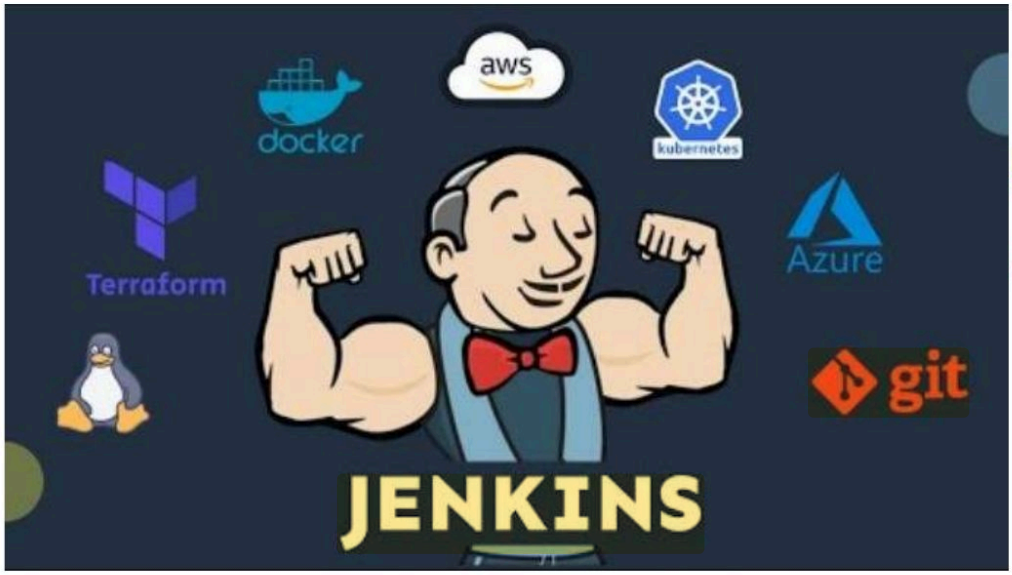
- By combining the practice of DevOps (CD/CI, automation, etc.) with the principles of project management (planning, task definition, progress tracking, etc.), you achieve greater coordination between individual team members and the larger goals of the sprint.
- This leads to better outcomes by ensuring that task scopes are properly defined and dependencies are tracked so the application is ready to deploy.
- DevOps project management tools (Jira, github, gitlab, github actions, Jenkins, etc.) simplify this process and promote visibility between project managers and team members so everyone is on the same page and working toward the same goals.

NOTES

1. DevOps moves fast, while PM keeps everything pointed the same way. Combining **CI/CD** with planning means fast work is also the right work.
2. A task can be unclear, or depend on something unfinished, and that blocks deployment. Good planning avoids this, so **dependencies** get handled before they stop a release.
3. You don’t need to know these tools in depth. They matter because they make the work visible to both sides, such as **Jira** for tasks and **GitHub** for code, without extra reporting.

SLIDE 90 / 98 DevOps tools (Jenkins)

DevOps project management leads to better outcomes (Jenkins)





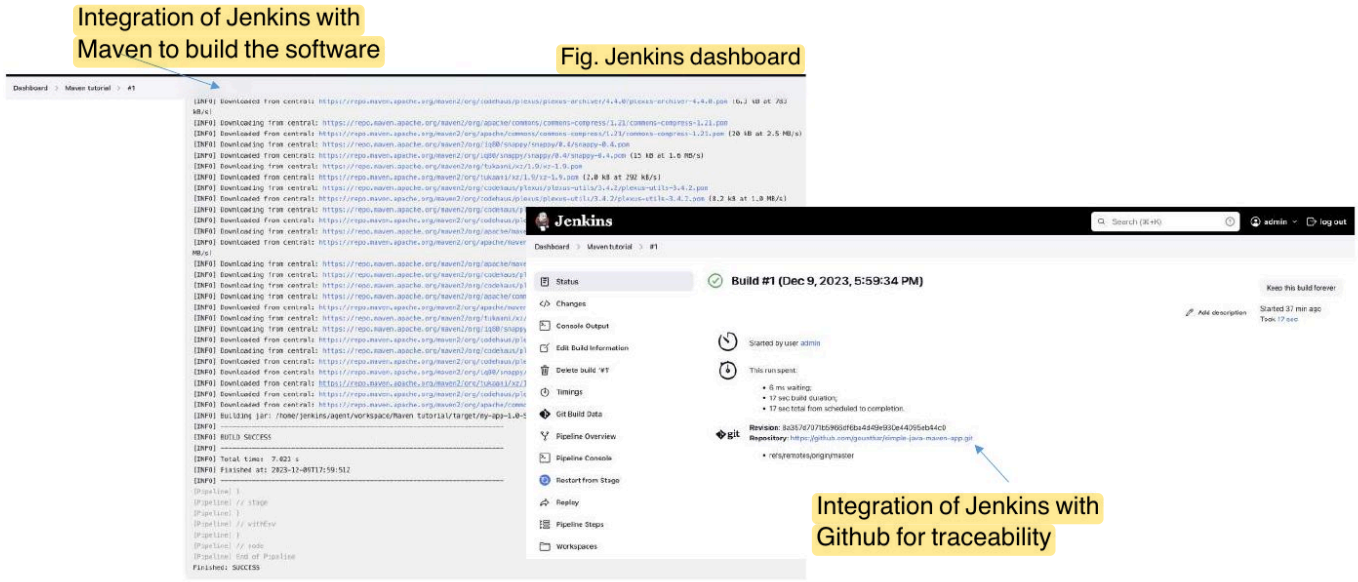
90

NOTES

1 **Jenkins**, an open-source automation server, at the center of a DevOps toolchain (Docker, Kubernetes, AWS, Azure, Terraform, Git). It automates build, test and deployment.

SLIDE 91 / 98 outcomes (Jenkins)

DevOps project management leads to better outcomes (Jenkins)

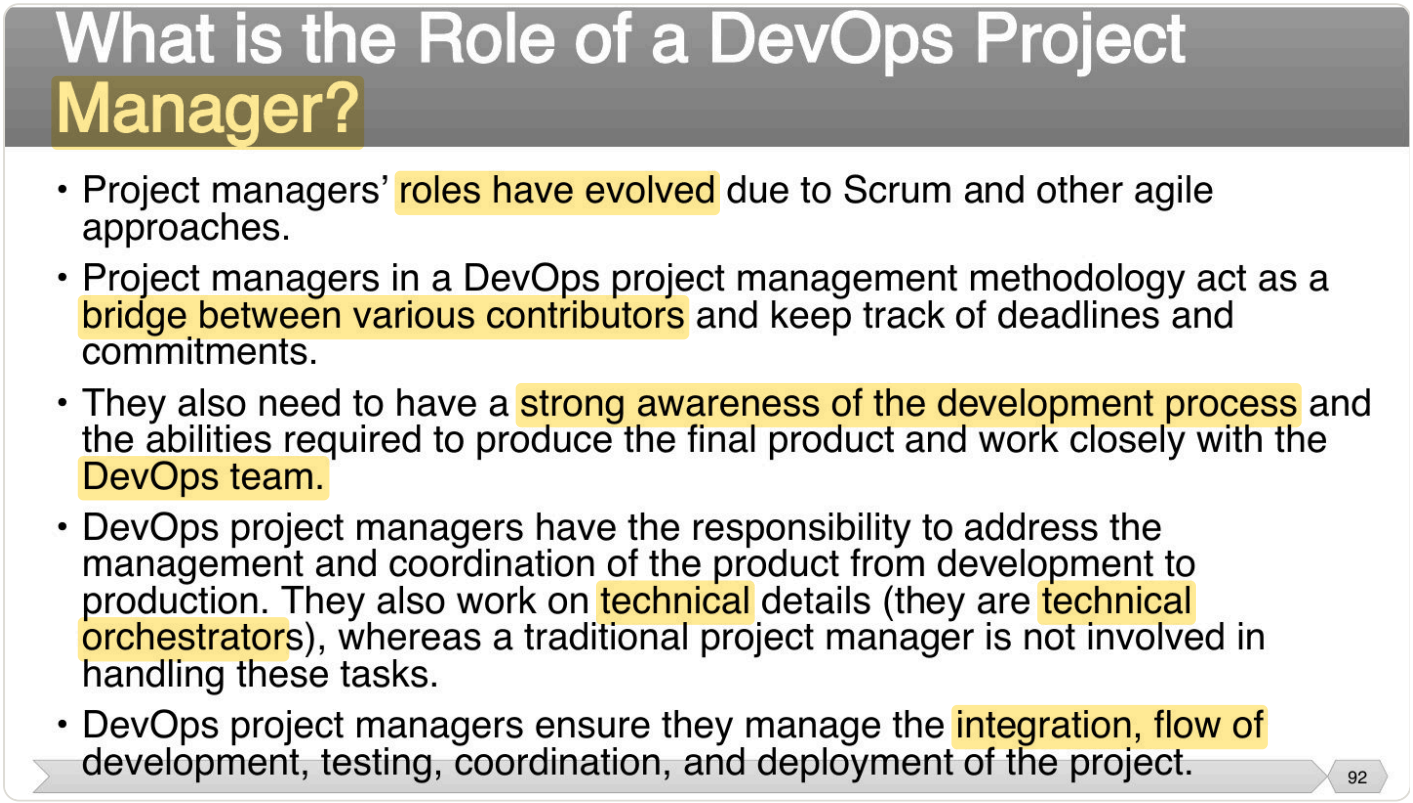


91

NOTES

1. Nobody has to remember how to build the software correctly each time. Linking Jenkins with **Maven** makes the build repeatable and automatic...
2. ...and with **GitHub for traceability**: each build links back to the commits that triggered it.

SLIDE 92 / 98 What is the Role of a DevOps Project



NOTES

1. Not only the methods changed, the job did too. With self-organizing teams, a PM gives fewer orders and helps more, so **roles have evolved**.
2. Developers, operations and business people have different concerns, and someone has to connect them. A PM as a **bridge** keeps commitments visible to everybody.
3. You can't coordinate work you don't understand. **Strong awareness of the development process** lets the PM spot a real risk rather than just asking for status.
4. A traditional PM may leave technical details to others. Here, the PM gets closer to them, as a **technical orchestrator**, and follows the product all the way into production.
5. Compare with development alone: this covers the whole path to release. **Integration, testing and deployment** all have to fit together, and managing that flow is the job.

EXAM ANGLE

Prediction Q1

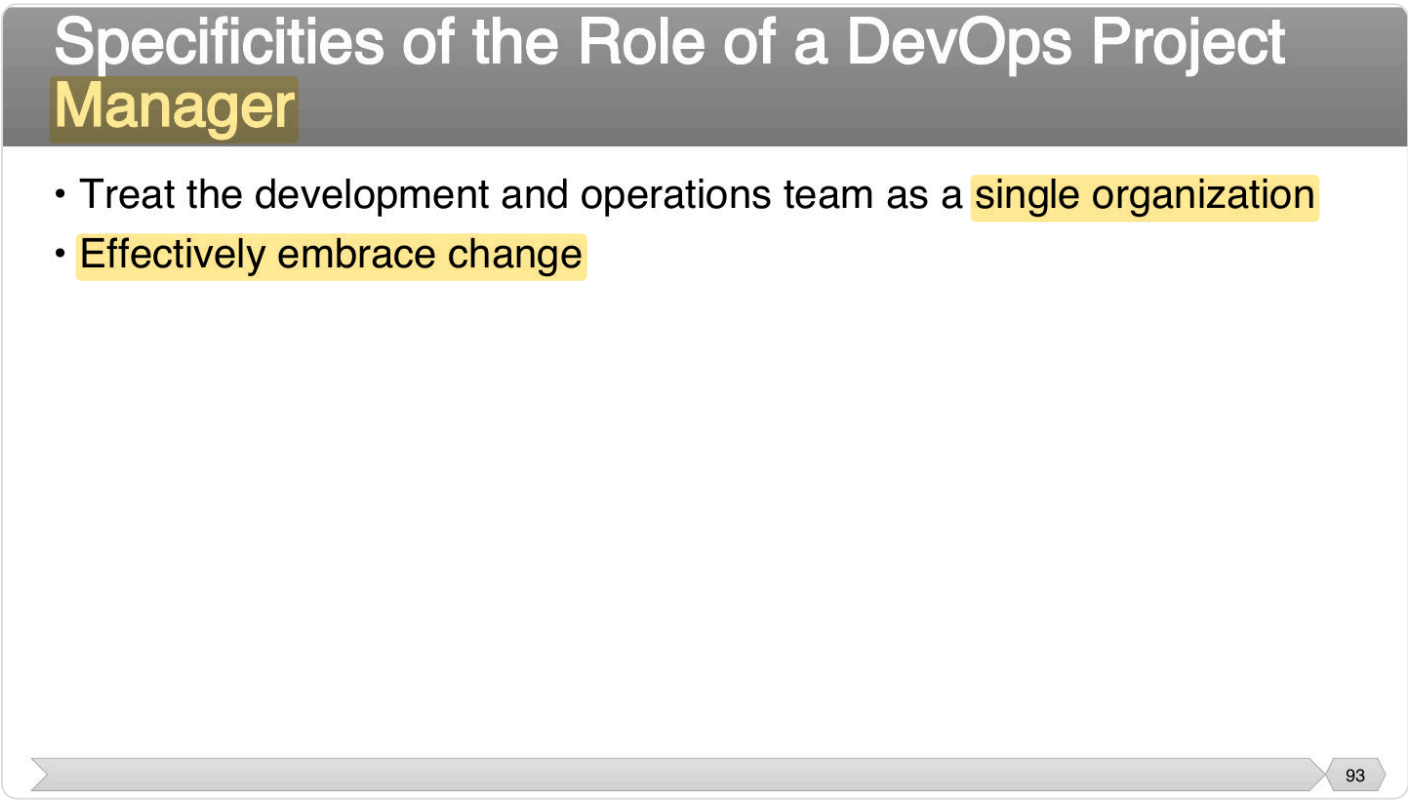
A DevOps project manager stays away from technical details, like a traditional project manager.

T True

F False

> DevOps PMs work on technical details as technical orchestrators.

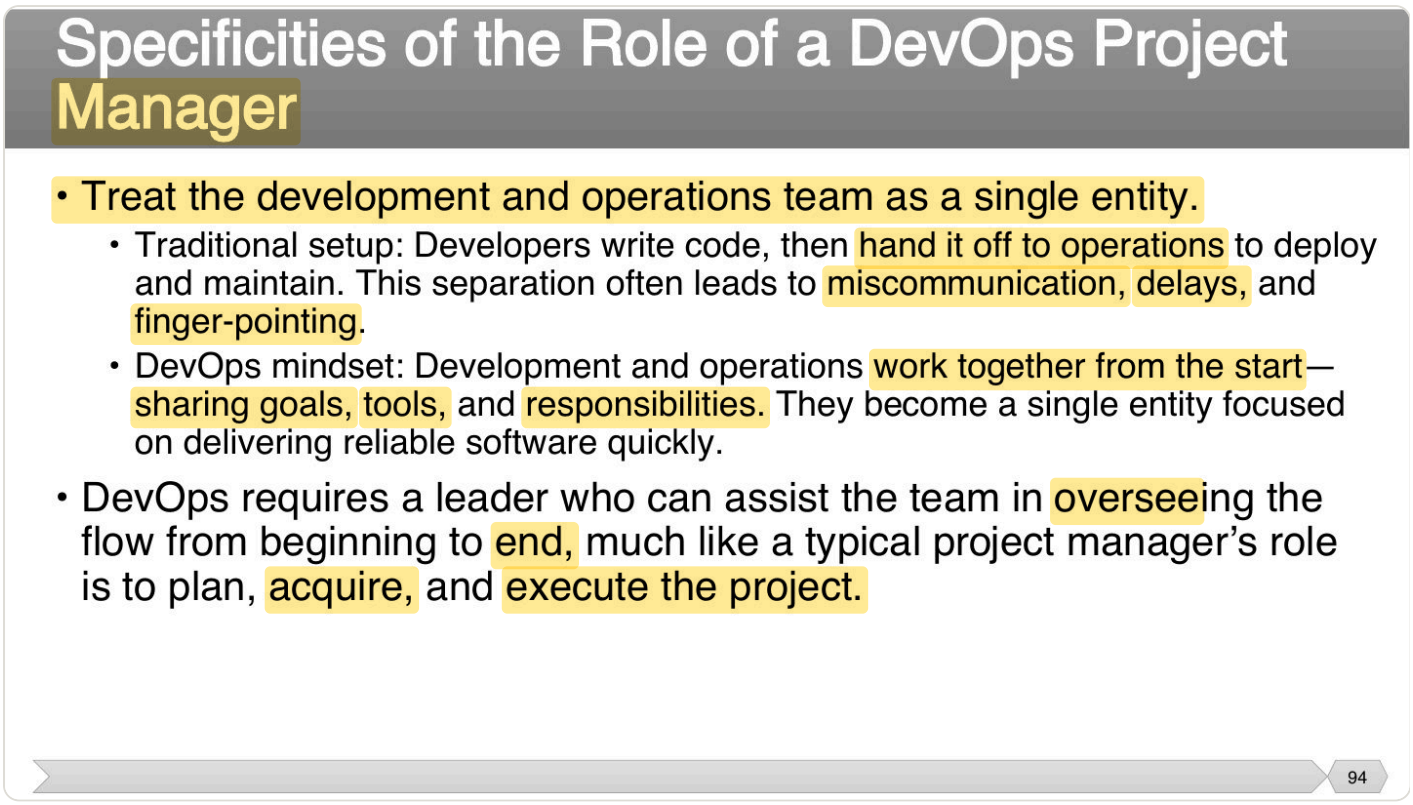
SLIDE 93 / 98 **Specificities of the Role of a DevOps Project**



NOTES

1. If they stay as two groups with different targets, nothing else on these slides works. So **single organization** is a precondition, not an extra...
2. ...change is constant in DevOps, so a PM who treats each change as a problem will be exhausted. To **embrace change** means making it routine.

 Specificities of the Role of a DevOps Project



NOTES

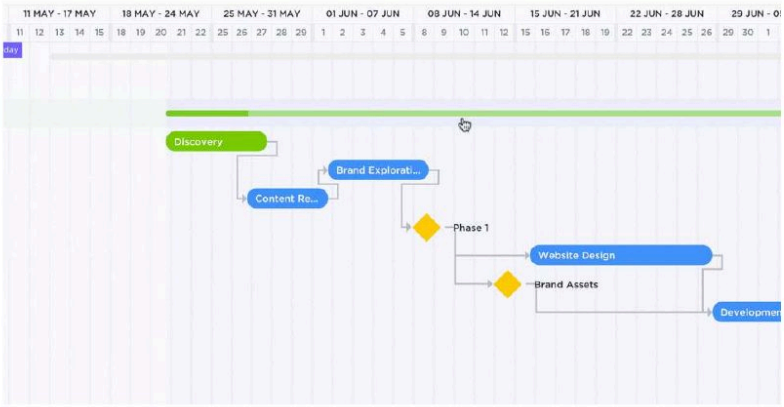
1. Developers want new features out, while operations want stability, and each blames the other when something breaks. **Hand-offs** are where information gets lost.
2. When both are responsible for the outcome, blaming stops being useful. **Working together from the start** means problems in operating the software get raised while they're cheap to fix.
3. Joined teams still need someone watching the whole flow, or gaps appear between steps. So a leader who can **oversee** it end to end is what makes this work.

 Specificities of the Role of a DevOps Project

Specificities of the Role of a DevOps Project Manager

- Treat the development and operations team as a single entity
  - Creating value stream maps should be a natural extension of the project manager's Gantt Chart planning abilities.

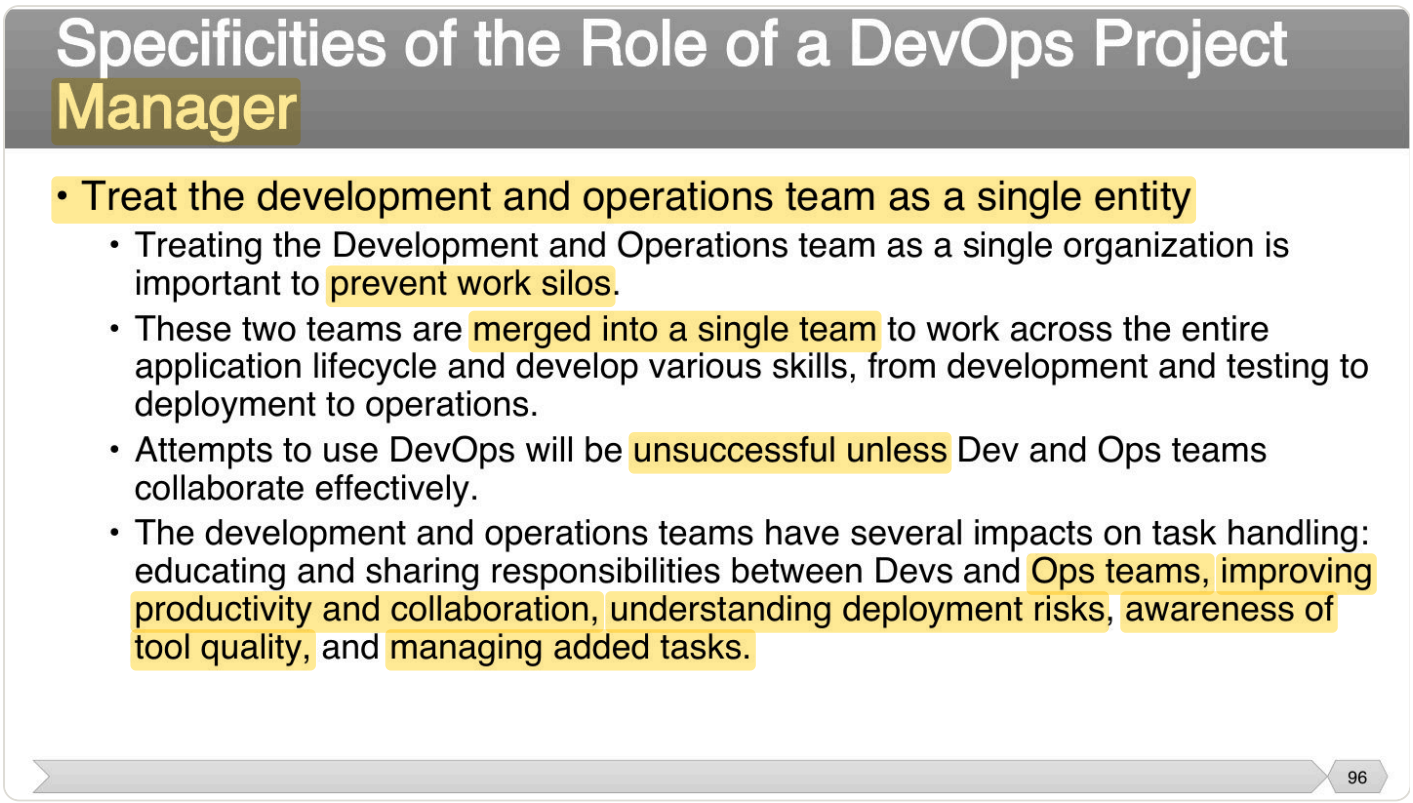
•Traditional project managers use these to plan tasks, timelines, and dependencies.
•Value stream maps visualize the entire flow of work—from idea to deployment—highlighting where value is added and where waste or delays occur.
•For project managers familiar with Gantt charts, creating value stream maps is a logical next step. It shifts the focus from task scheduling to process optimization.



NOTES

1. If you can read a Gantt chart, you're most of the way there, because both show work over time. **Value stream maps** just ask a different question about it...
2. ...rather than "when will this happen?", they ask "where does work get stuck?". If a change waits five days for one approval, that's **waste** you'd never see on a task list.
3. A schedule tells you when, but it can't tell you if the process is any good. This shift from **task scheduling to process optimization** is the point of the slide.

 Specificities of the Role of a DevOps Project



NOTES

1. If you only change the names but keep separate goals, nothing changes. A **silo** forms whenever people are judged only on their own part of the work.
2. When people do a bit of everything, they see each other's problems, and they're less likely to create them. That's what **merged** teams gain: a wider view.
3. Buying tools isn't adopting DevOps. Without real collaboration, you get the same conflict with newer tools, which is why it's **unsuccessful unless** they truly work together.
4. A developer who's paged at night writes more careful code. **Understanding deployment risks** is a benefit of sharing responsibilities: people see the consequences of what they do.

SLIDE 97 / 98 **Specificities of the Role of a DevOps Project**

Specificities of the Role of a DevOps Project Manager

- Effectively embrace change
  - The agile project management methodology clarifies that embracing change rather than sticking to the plan is crucial.
  - It takes a lot of communication and teamwork to manage change and release features incrementally. Communication is a project manager's primary skill. Enterprises ought to make use of this potential.

NOTES

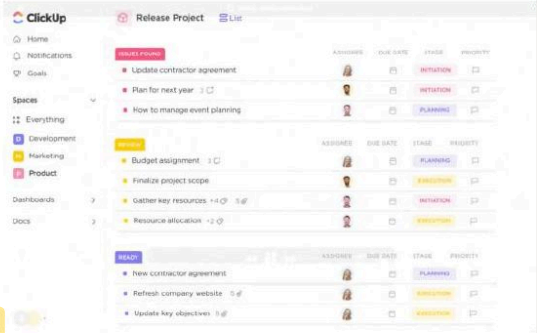
1. A plan is useful until it stops matching reality. Sticking to it anyway means building what nobody needs, so **embracing change** is what keeps a project useful.

2. When things keep changing, people need to know what changed and why. So a PM's **communication** is what stops frequent change becoming chaos, and it's their strongest tool here.

SLIDE 98 / 98 **Specificities of the Role of a DevOps Project**

Specificities of the Role of a DevOps Project Manager

- Effectively embrace change
  - Change isn't just inevitable—it's a catalyst for growth. To thrive in fast-paced DevOps environments, project managers must evolve their mindset and practices:
- Begin with small projects to test ideas and reduce risk
- using the MVP approach: to deliver value early and often
- using the right agile planning tools: to stay adaptive and transparent
- eliminating silos: by fostering cross-functional collaboration
- reducing project handoffs: to streamline workflows, minimize delays
- creating real-time project visibility: with shared dashboards and live updates
- reducing project overhead: by simplifying processes and focusing on value delivery
- managing change collaboratively: Involve your team in planning and adapting to change



NOTES

1. A small project limits the damage if the idea fails, and teaches you cheaply. **Start small**, then an **MVP** gets real results quickly, while you learn how the new way of working fits.
2. Every hand-over is a chance to lose information and wait in a queue. So **reduce hand-offs**, and work moves faster with fewer misunderstandings.
3. People make better decisions when they can see what's going on without asking. **Real-time visibility** also makes unnecessary status meetings go away.
4. People resist changes done to them far more than changes they helped shape. **Managing change collaboratively** turns those affected into people who make it work.

EXAM ANGLE

Prediction Q1

Your organization wants to move a waterfall team to DevOps. As the PM, what would you change first and why?

Answer: Treat development and operations as one team with shared goals and tools, to stop hand-offs and finger-pointing. Start with a small project and an MVP so there is a deliverable every sprint; build a CI/CD pipeline with automated tests; map and track dependencies; use shared dashboards (Jira, Jenkins) for real-time visibility; and track metrics such as deployment frequency, lead time, change failure rate and MTTR to show improvement.

Original slides from the SE423 lecture deck, with study notes. Check exact wording against the original deck.
Notes and questions are study aids. Made by Shoug Alomran.